

第6回

手嶋毅志 (Takeshi Teshima)

質問と回答

加対象の質問

今回までの授業で様々なモデル式が出され、「0 に近づいたものを探す式」が多かったように思います。そこで質問なのですが、これまでの授業で扱われた式は、実際どのくらい0 に近づけばよいのでしょうか？例えば、0.5や0.3などは十分に近づいたことになるのでしょうか？今回取り扱っていたダイバージェンスの式の計算も途方もない事になりそうなのですが、実務だとどうなのか知りたいです。

加算対象ではないその他の質問・コメント

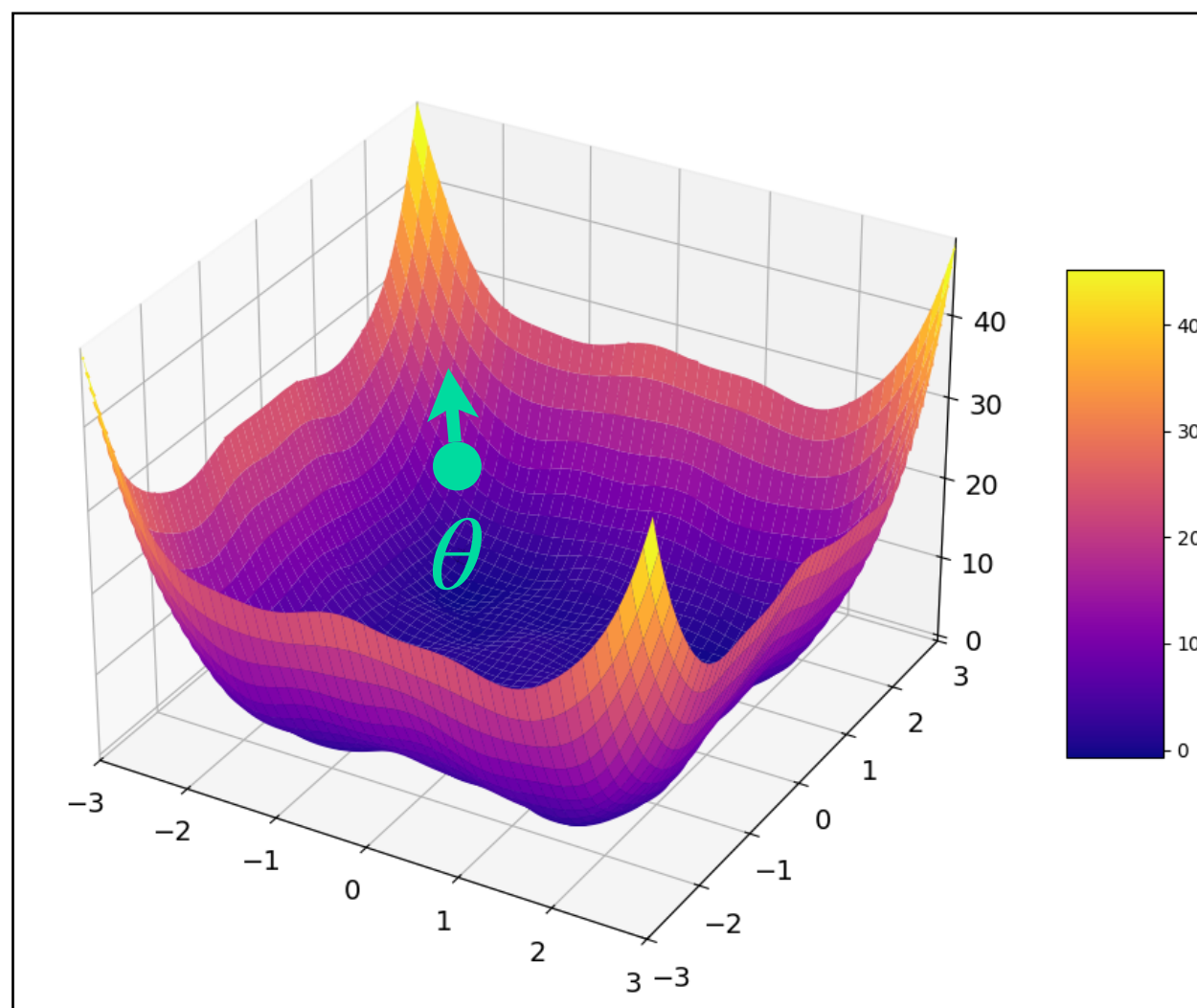
授業で扱う内容を実務で使っている方々というは、どのようにして習得していったのでしょうか？ 正直、3回目ぐらいからモデル式や理論などに追いつけなくなってきました。もし、1人で実践・練習などできる教材・アプリ・方法があれば教えてほしいです。

2クラスの確率論的分類（前回の残り）

基本知識の確認＞勾配のもつ意味

勾配は、最も上昇する方向（steepest ascent direction）を表す

- 多次元の場合も同じ。微小距離だけ θ を移動させるとき、
 - θ を $\nabla L(\theta)$ の方向に移動すれば、 L は最も増加する
 - θ を $-\nabla L(\theta)$ の方向に移動すれば、 L は最も減少する



勾配降下法による最適化＞アルゴリズム

勾配降下法（gradient descent）により目的関数の最小化を目指す

- パラメーター θ を適当に初期化 ($\hat{\theta} \leftarrow \theta$)
- 以下を必要なだけ繰り返す
 - 目的関数 L の現在のパラメーター $\hat{\theta}$ における勾配 $\nabla L(\hat{\theta})$ を計算
 - 勾配の反対方向にパラメーターを少し更新（学習率 $\eta > 0$ ）

$$\hat{\theta} \leftarrow \hat{\theta} - \eta \nabla L(\hat{\theta})$$

- 早期停止条件を満たしていたら中断
- 最終的な $\hat{\theta}$ を使用する

勾配降下法による最適化＞詳細

基本はシンプルだが、細かく見ていくと色々な項目を決める必要がある

- パラメーター θ を適当に初期化 ($\hat{\theta} \leftarrow \theta$)
 - ← 初期化の方法は色々ある
(ハイパーパラメーター)
- 以下を必要なだけ繰り返す
 - ← 繰り返しの最大回数 T は決め打ち
(ハイパーパラメーター)
 - 目的関数 L の現在のパラメーター $\hat{\theta}$ における勾配 $\nabla L(\hat{\theta})$ を計算
 - **勾配の反対方向** にパラメーターを少し更新 (学習率 $\eta > 0$)
 - ← 繰り返しの t 回目には $\eta = \eta_t$ など
徐々に小さくすることもある
(ハイパーパラメーター)
 - 早期停止条件を満たしていたら中断
 - ← 例えば検証損失の改善幅が
 S 回にわたって ϵ に満たなかったら
早期停止 (ハイパーパラメーター)
- 最終的な $\hat{\theta}$ を使用する

$$\hat{\theta} \leftarrow \hat{\theta} - \eta \nabla L(\hat{\theta})$$

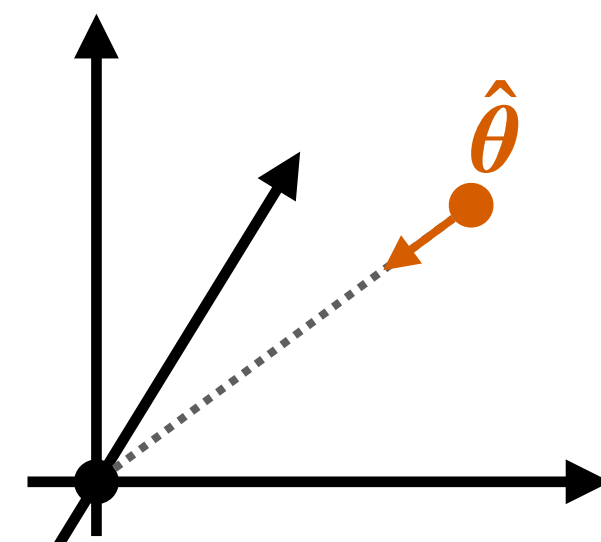
勾配降下法による最適化 > 詳細 > 重み減衰

勾配法で最適化する場合のL2正則化は，重み減衰とも呼ばれる

■ 重み減衰 (weight decay)

- 勾配法で最適化をする場合の，L2正則化の呼称
- L2正則化項があると，勾配法の更新方向は $-\nabla L(\hat{\theta}) = -\nabla \hat{R}(g_{\hat{\theta}}) - 2\lambda\hat{\theta}$
- 単に $R(g_{\theta})$ を小さくするだけの方向と比べて， $-2\lambda\hat{\theta}$ だけズレる

■ これは，更新の度に少しずつパラメーターを原点に向けて縮小すると解釈できる

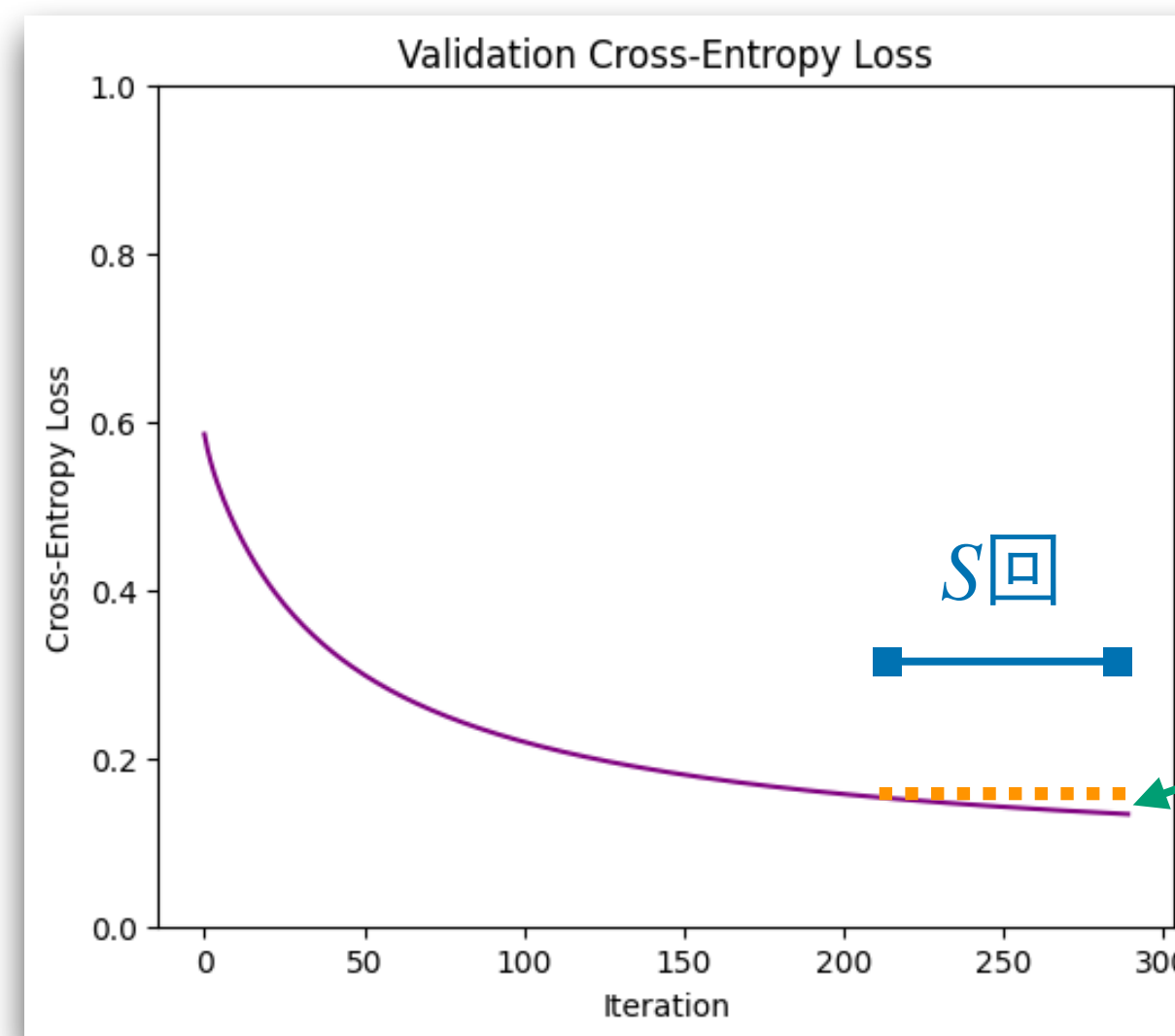


【用語】ここでの「重み」は「パラメーター」と同義（重み付き和の係数に由来する用語）。ただし，「重み」といったとき，定数項に対応するパラメーターは含まないこともある。後者の場合，定数項に対応するパラメーターはバイアスと呼ばれ，バイアスを除くパラメーターにだけ重み減衰が適用される。

勾配降下法による最適化 > 詳細 > 早期停止

何らかの指標に基づいて、早期停止（early stopping）をすることが多い

- 例）検証データ上で直近 S 回にわたって ε 以上の改善がみられなかったら終了



検証損失値

ε に満たない改善幅
(停滞している)

- 「更新回数」（iteration）というハイパーパラメーターの選択にほかならない

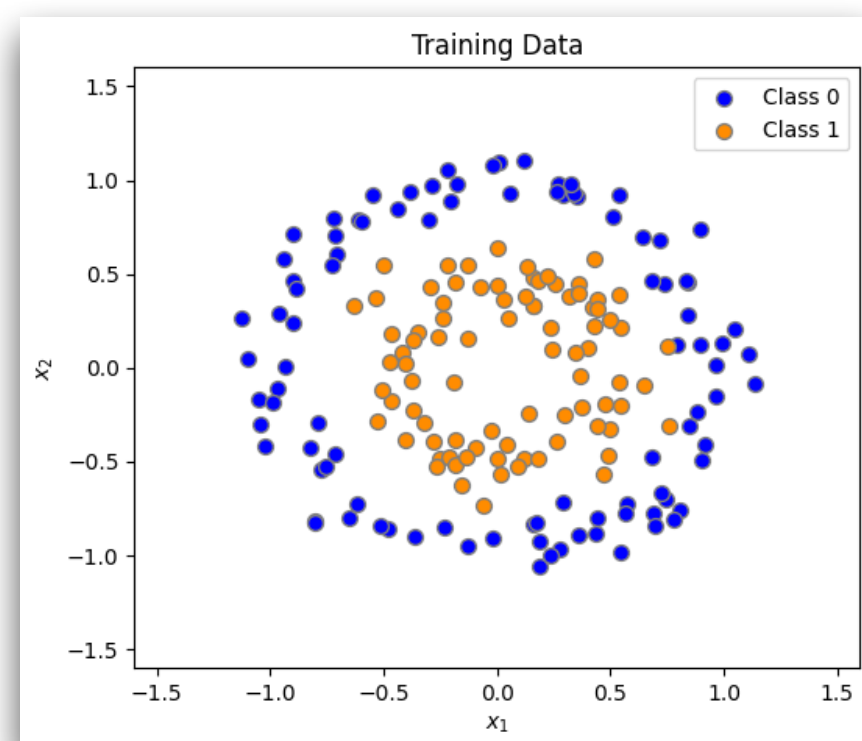
確率論的二値分類 > 実験 > 設定

ロジスティックモデルをL2正則化付き最尤法で学習（勾配降下法で最適化）

タスク・学習戦略

分類タスク・教師付き学習

特徴量	ラベル
x_1	y_1
x_2	y_2
$\vdots$	$\vdots$
$\{(X_i, Y_i)\}_{i=1}^n \stackrel{\text{i.i.d.}}{\sim} p$	



訓練データ

モデル

ロジスティックモデル

$$g_{\theta}(x) = \sigma(\theta^{\top} \phi(x))$$

- 中身は30次の多項式

目的関数

負の対数尤度+正則化

$$L(\theta) := \frac{1}{n} \sum_{i=1}^n \ell((x_i, y_i), g_{\theta}) + \lambda \|\theta\|^2$$

- ハイパーパラメーター選択
- モデル選択指標は、早期停止時点の検証損失値
- 正則化係数 λ は $\{10^{-3}, 10^{-2}, 10^{-1}, 10^0\}$ から選択

最適化手法

勾配降下法

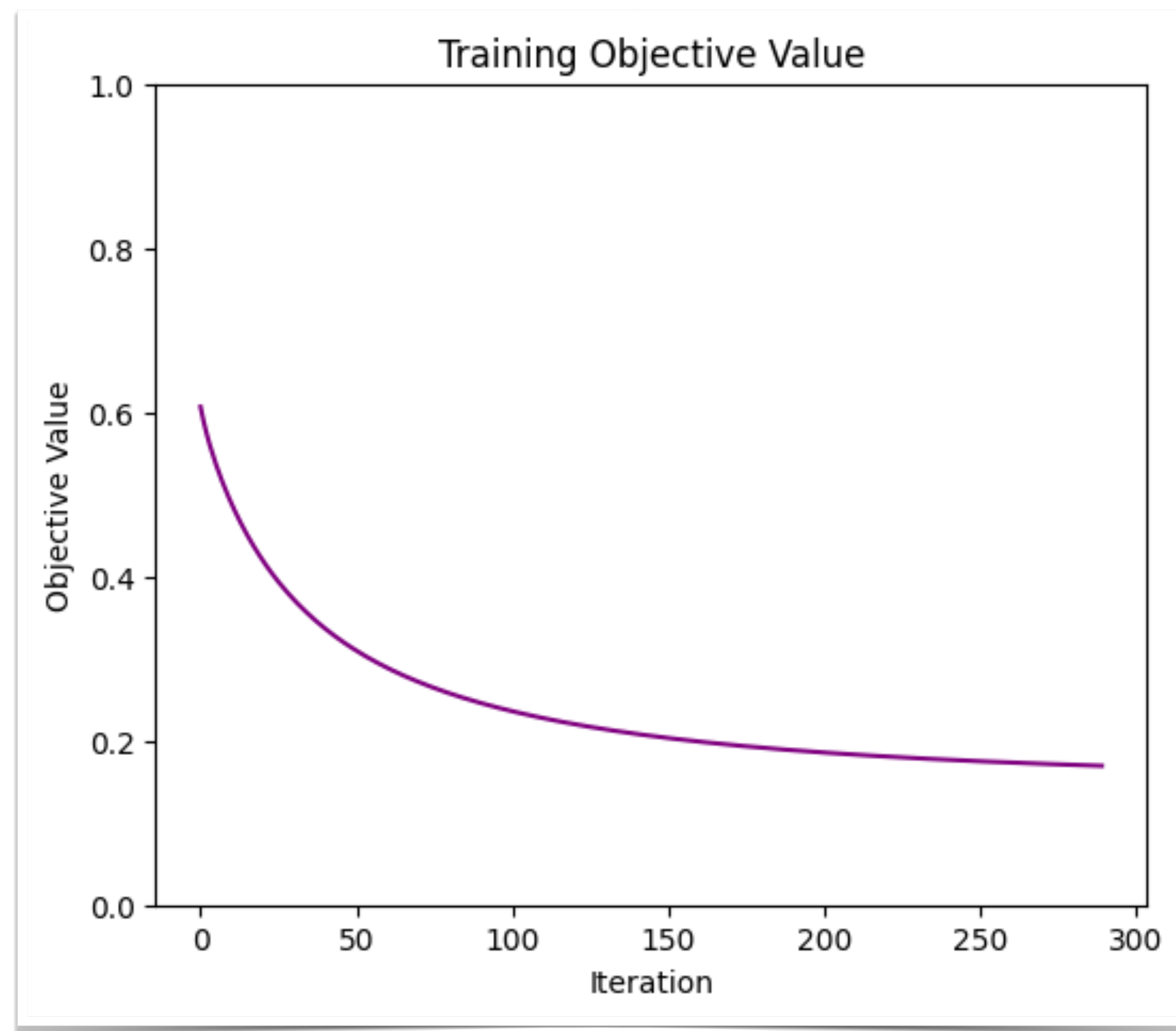
$$\hat{\theta} \leftarrow \hat{\theta} - \eta \nabla L(\hat{\theta})$$

- 学習率 η は 0.5
- 更新回数上限10000回
- 早期停止あり（直近5更新の改善幅が 10^{-3} 未満で停止）

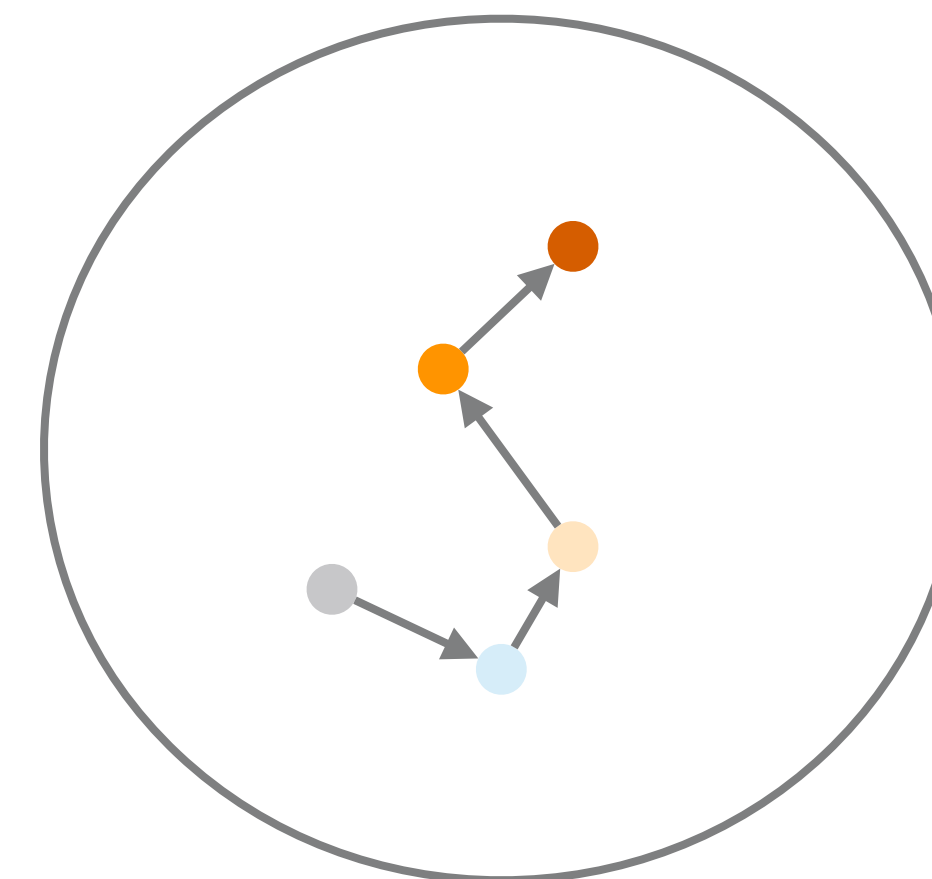
確率論的二値分類＞実験＞結果（学習過程）

更新の繰り返しにより，徐々に目的関数値が小さいパラメーターを探索

縦軸：目的関数値



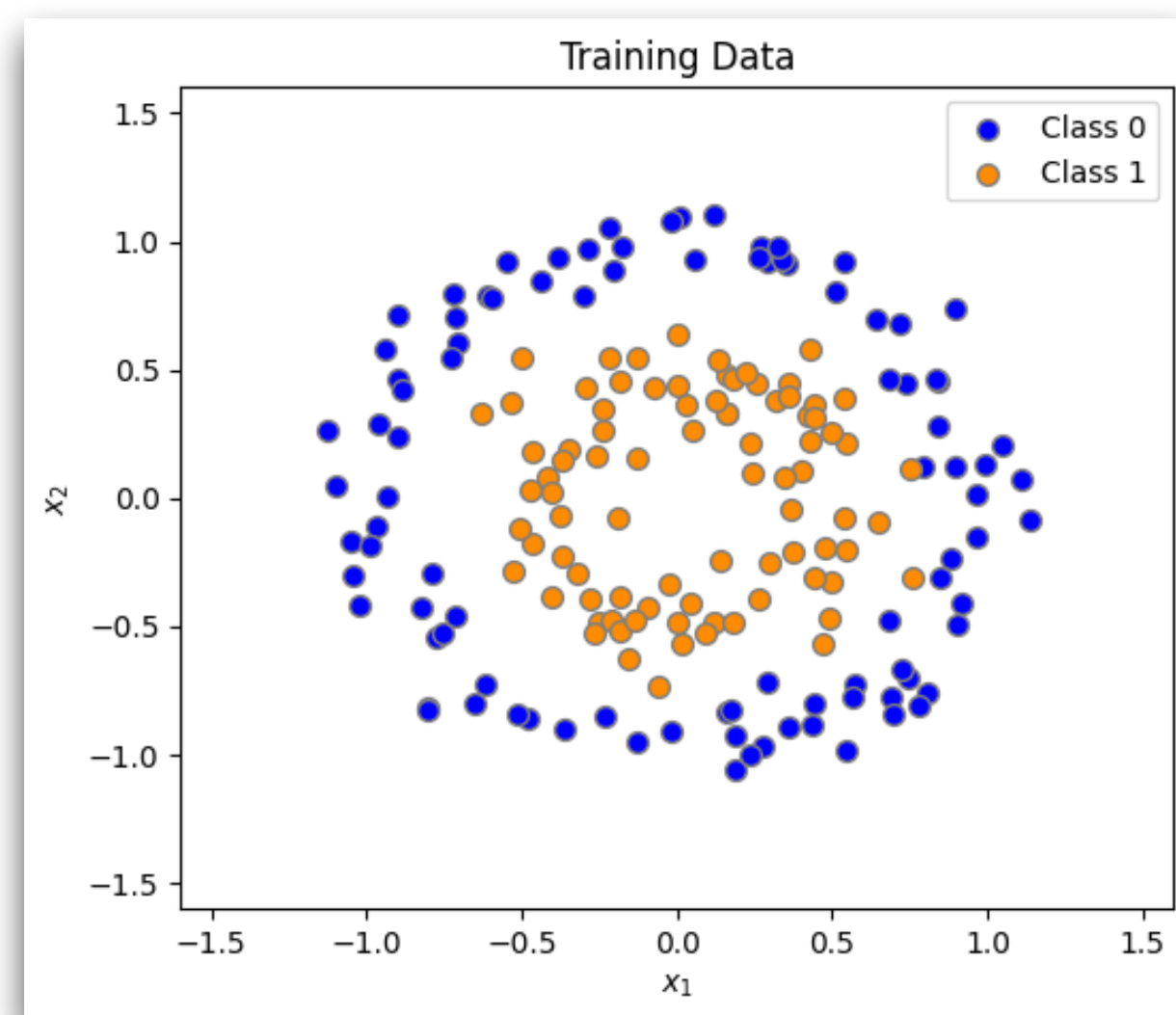
横軸：更新回数



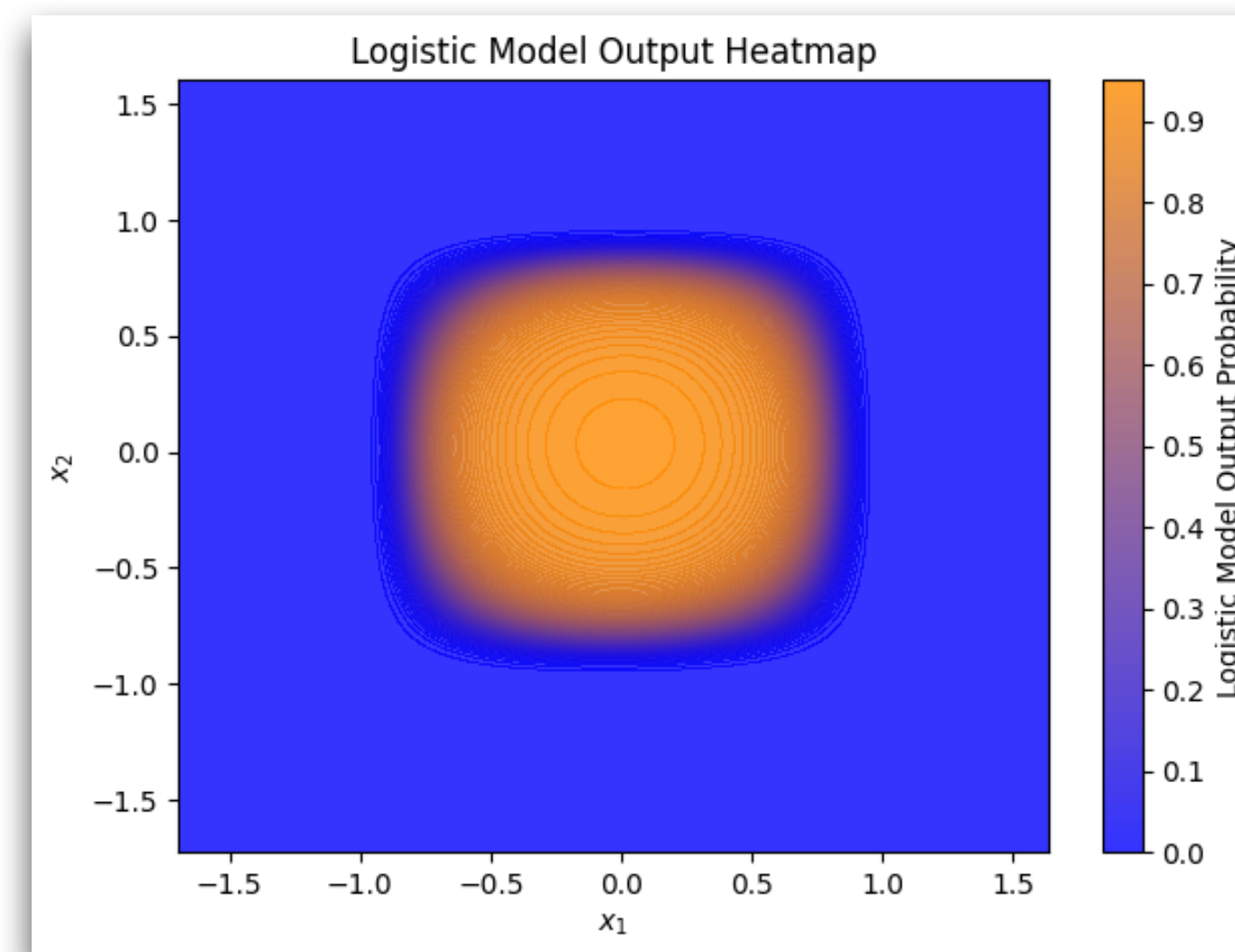
確率論的二値分類 > 実験 > 学習結果

妥当そうなスコアリングモデルが学習できている

- 正例のあたりでは 1 に近く，負例のあたりでは 0 に近いスコア
 - その境目付近ではグラデーションがついている



訓練データ

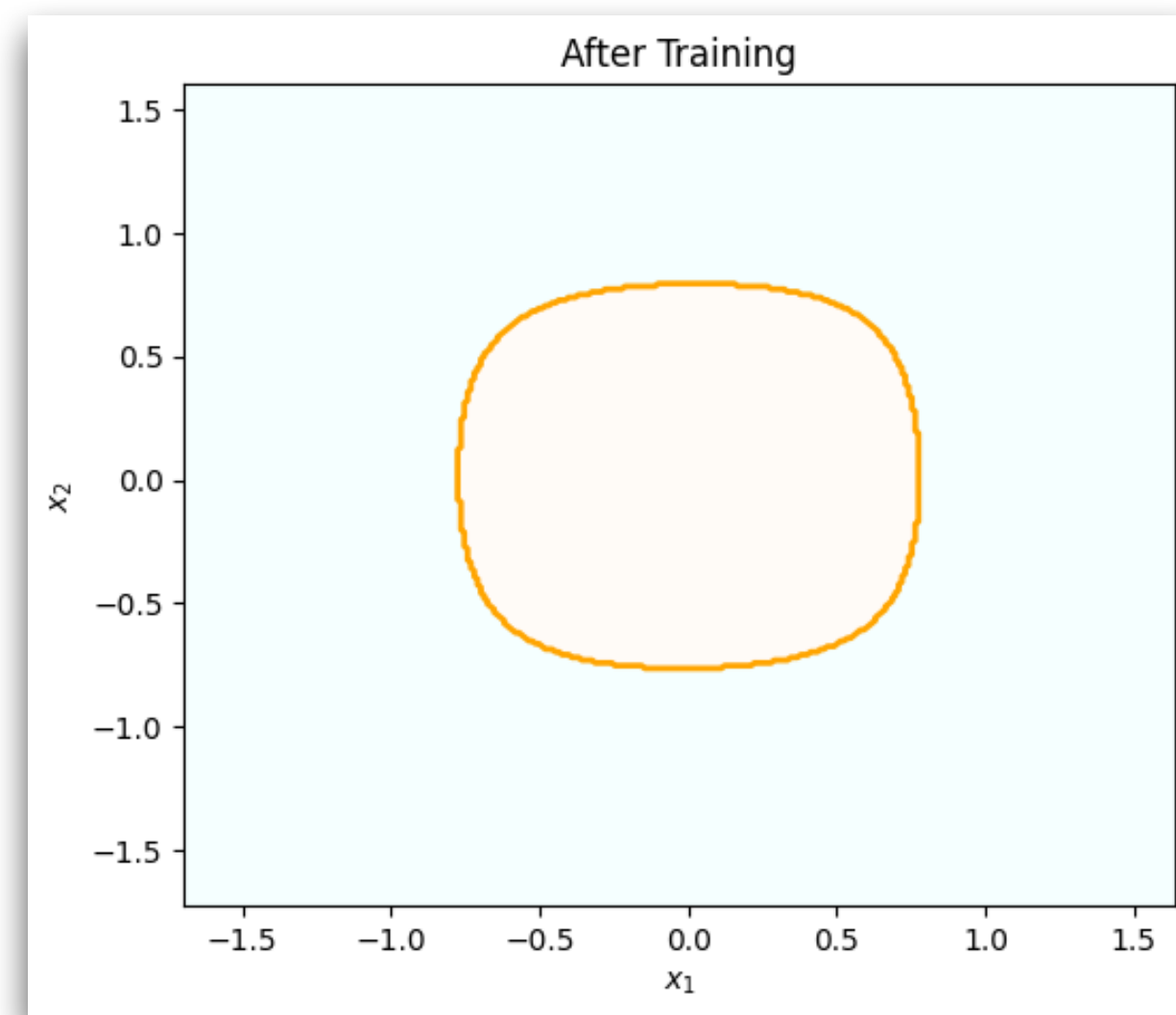


学習済みモデル $g_{\hat{\theta}}$ が出力した確率値スコア

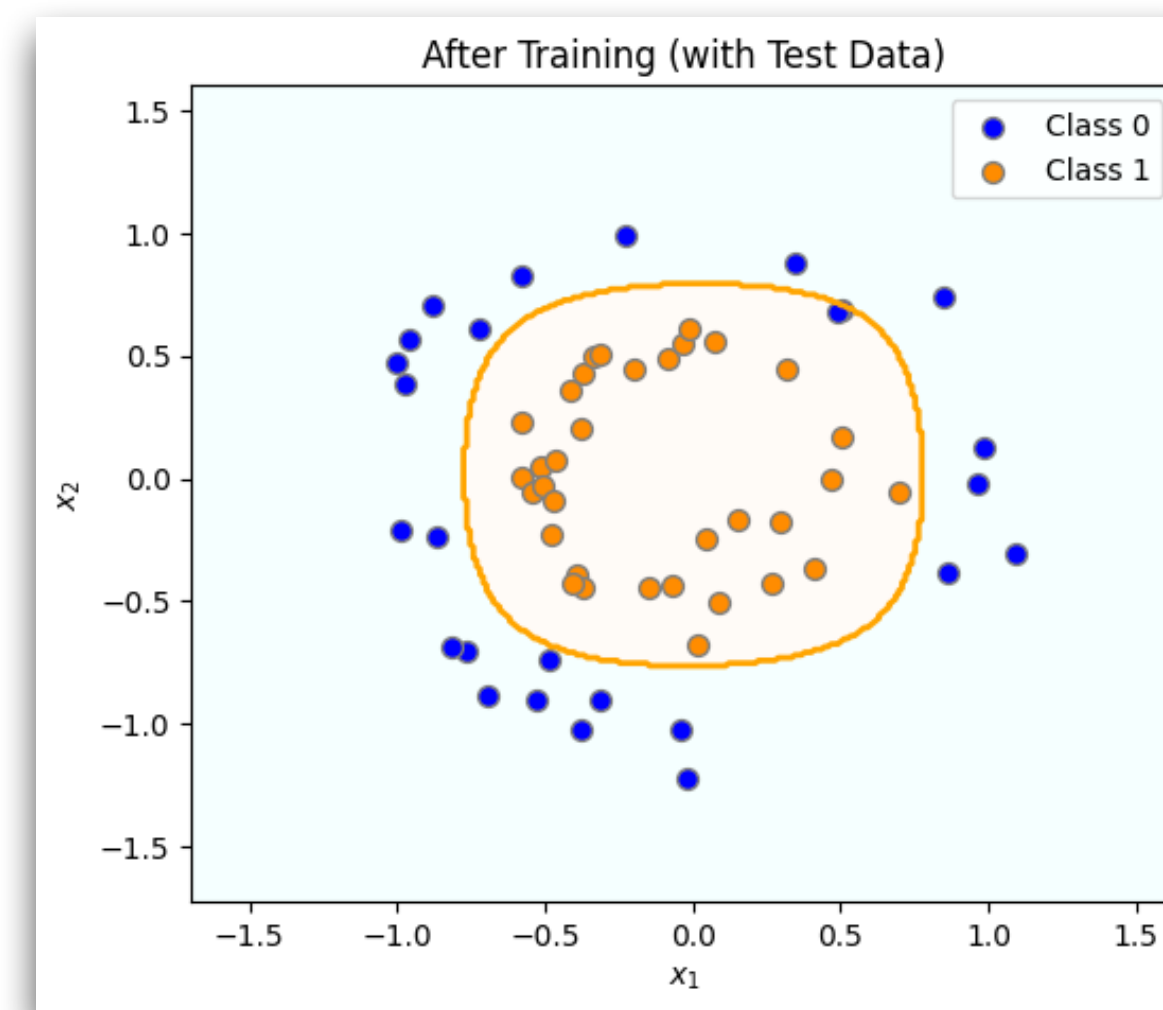
確率論的二値分類 > 実験 > 学習結果

予測器としても良さそうなものになっている

- 学習後のモデルに決定規則を合成した予測器 $f_{\hat{\theta}}(x) := \tau(g_{\hat{\theta}}(x))$ の様子



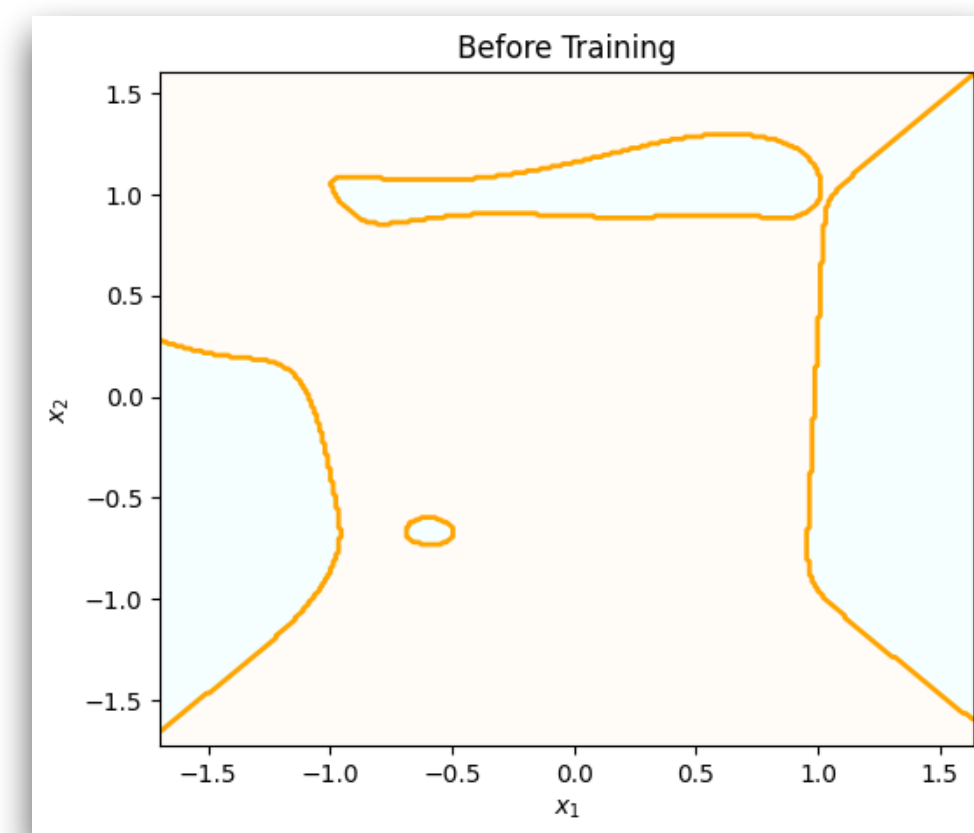
分類器としての出力
(スコアを0.5で閾値処理した結果)



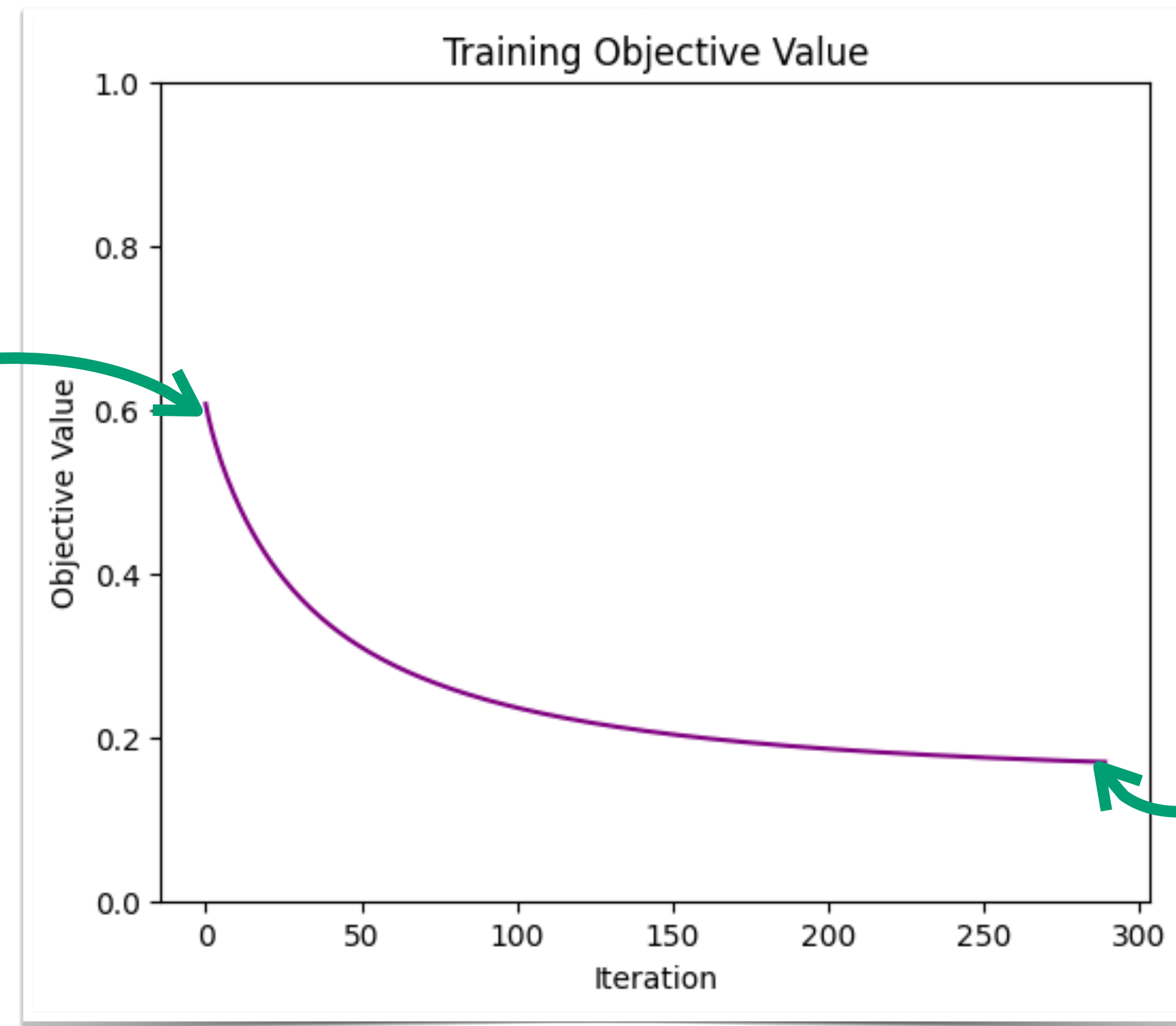
評価用データでの予測と正解ラベル

確率論的二値分類 > 実験 > 学習結果（参考）

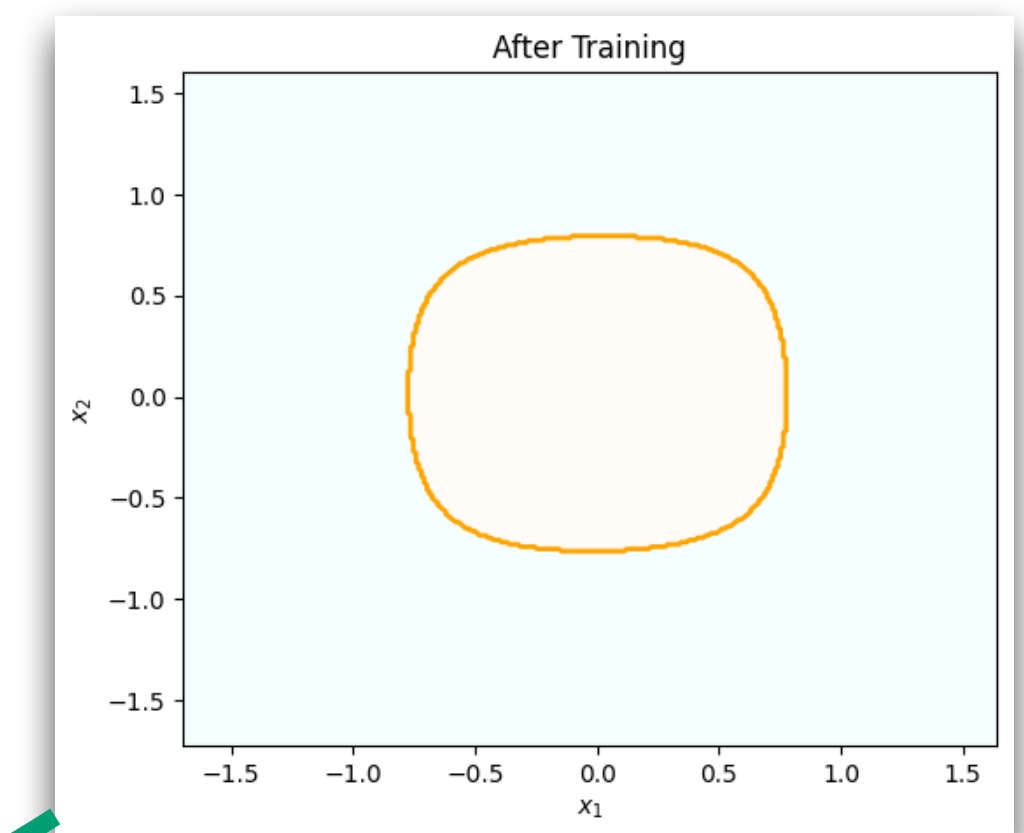
訓練前の初期化した時点での予測器と，訓練後の予測器



訓練前



目的関数値



訓練後

確率論的二値分類 > まとめ



確率論的二値分類の標準的な枠組みを説明した

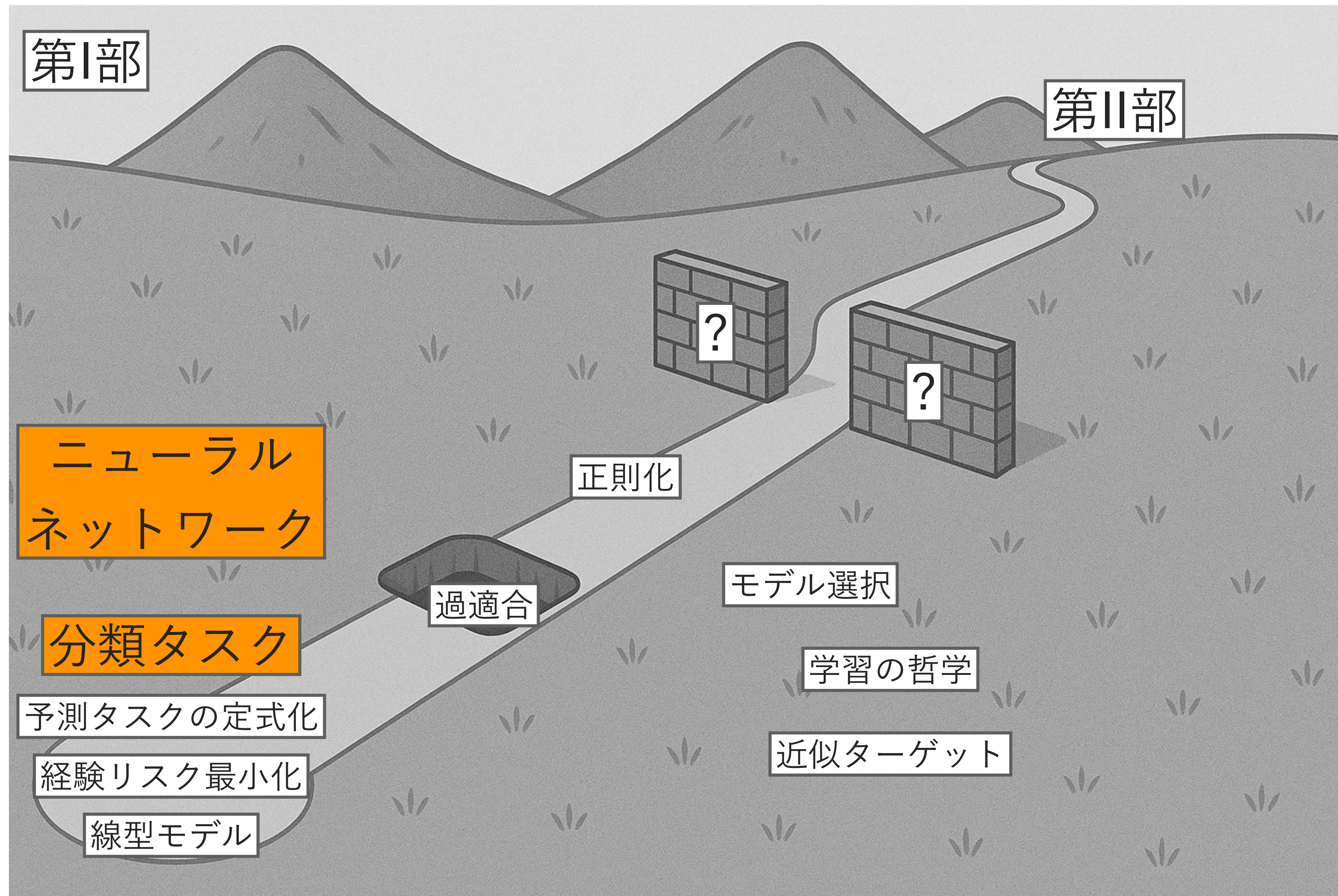
■ 確率論的二値分類

- ロジスティック関数を組み込むことでモデル $g_{\theta}(\mathbf{x})$ の出力値が確率値になるようにする
- 交差エントロピーをリスクとして学習することで $g_{\theta}(\mathbf{x}) \simeq p(y = 1 | \mathbf{x})$ となるようにする
- 最適化は勾配法ベースで行う

■ 注意

- 確率値を出力できるなら、ロジスティック関数でなくてもよい
- 確率値の近似をさせられるなら、交差エントロピーの最小化でなくてもよい
- 上手く最適化できるなら、勾配法でなくてもよい
- ただし、現在はこの組み合わせが標準的（特に工夫しない場合に使われる）

ロードマップ



多クラスの確率論的分類

講義の前提知識＞ノテーションの注意

この講義では、ベクトルの成分を表すために次の記号を使う

- 多次元の出力をするモデルを扱うにつれて、特定の成分を表す機会が増える
 - そこで、次の便利な記号を用意しておく
- （次元を問わず）ベクトル a に対して、その第 k 成分を $a[k]$ と表す.

$$a = \begin{pmatrix} 1 \\ 10 \\ 100 \\ 1000 \end{pmatrix} \text{ とするとき, } a[3] = 100$$

多クラスの確率論的分類（確率論的多値分類）



多クラス分類の場合の確率論的分類を説明

- 誤分類率 $R(f) = \mathbb{E} [\ell_{0/1}(Y, f(X))] = \mathbb{P}(Y \neq f(X))$ に対して最適な予測器の形は,

$$f^*(\mathbf{x}) = \arg \max_{1 \leq k \leq K} p(y = k | \mathbf{x})$$

- そこで, モデルで $p(y | \mathbf{x})$ を近似するよう学習し, その値が最大なクラスを出力

$$f_{\theta}(\mathbf{x}) = \arg \max_{1 \leq k \leq K} p_{\theta}(y = k | \mathbf{x})$$

確率論的多値分類＞確率値ベクトルを出力するモデル

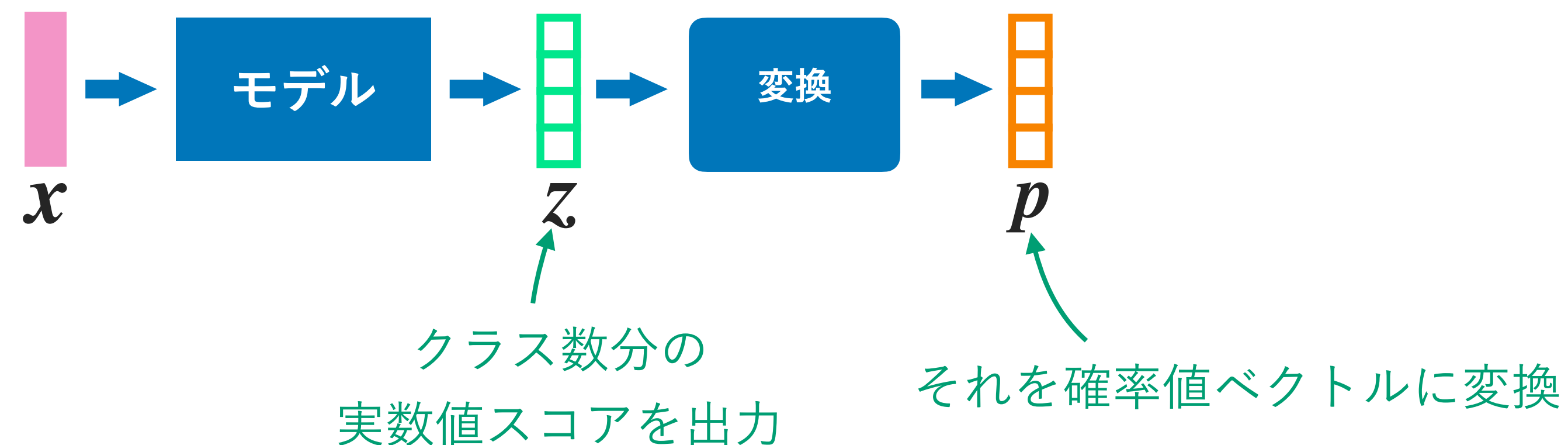
出力が確率値ベクトルになるモデル $g_{\theta}(\mathbf{x})$ を用意する必要がある

- モデルで近似したいのは、全クラスの事後確率を並べた確率値ベクトル関数

$$\mathbf{x} \mapsto \begin{pmatrix} p(y = 1 | \mathbf{x}) \\ \vdots \\ p(y = K | \mathbf{x}) \end{pmatrix}$$

成分が非負で、総和が1になる

- 対策：出力が確率値ベクトルの要件を満たすように、出力値の変換を合成



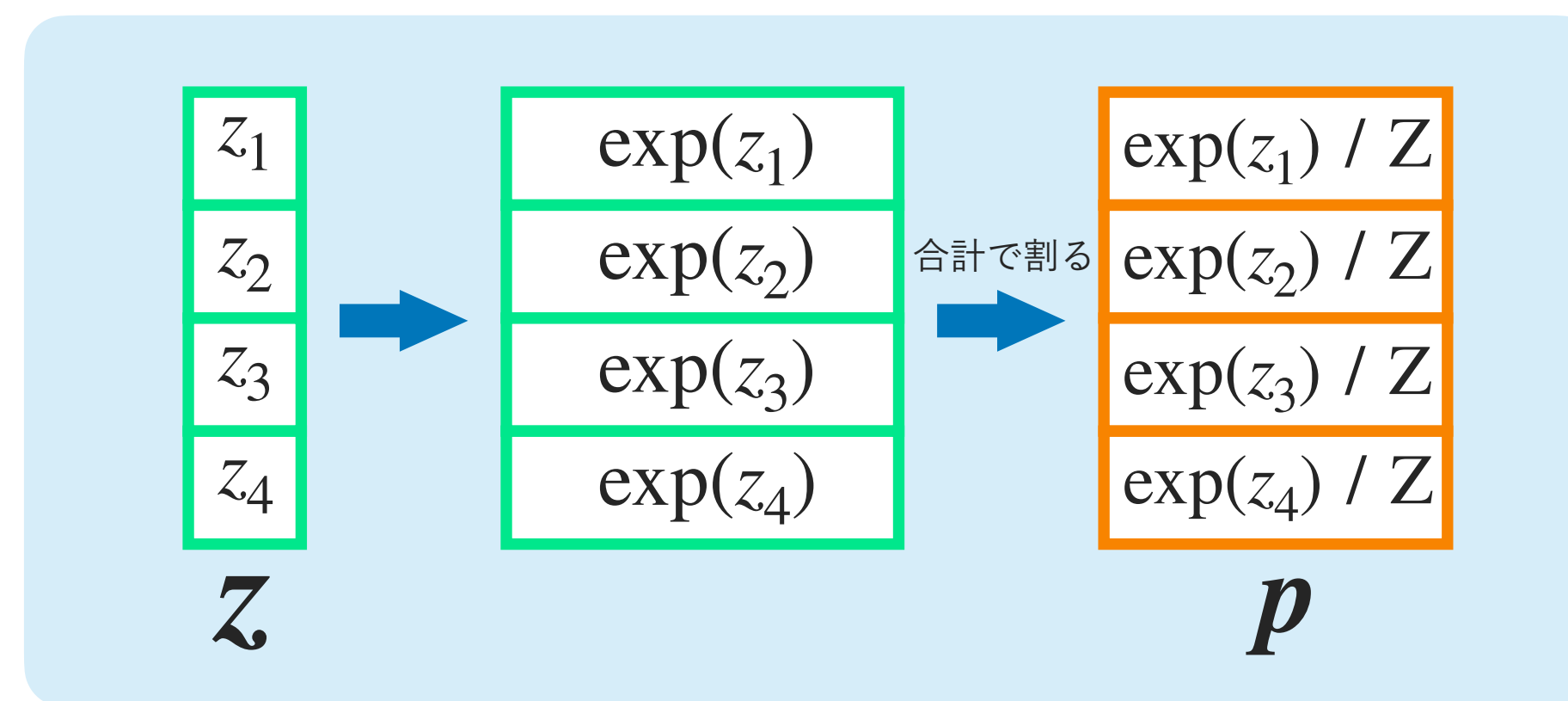
確率論的多値分類＞確率値ベクトルを出力するモデル

出力が確率値ベクトルとなるようにしたい場合，次のような変換を用いる

■ ソフトマックス関数 (softmax)

$$\text{Softmax}(z_1, \dots, z_K) = \frac{1}{\sum_{k=1}^K e^{z_k}} \begin{pmatrix} e^{z_1} \\ \vdots \\ e^{z_K} \end{pmatrix}$$

- まず $z_k \mapsto e^{z_k} \geq 0$ としておいて，総和で割るので，これは確率値ベクトルになる



4次元の実ベクトルを，4次元の確率値ベクトルに変換

$$(Z = \exp(z_1) + \dots + \exp(z_4))$$

確率論的多値分類 > ソフトマックス関数 > 補足

ソフトマックス関数は、ロジスティック関数の多クラス化とみなせる

- 2クラスの場合、ソフトマックス関数への入力も出力も2次元で、

$$\text{Softmax}(z_1, z_2) = \begin{pmatrix} \frac{e^{z_1}}{\sum_{i=1}^2 e^{z_i}} \\ \frac{e^{z_2}}{\sum_{i=1}^2 e^{z_i}} \end{pmatrix}$$

- いま、 $w = z_1 - z_2$ とおき、ロジスティック関数 $\sigma(t) = \frac{1}{1 + e^{-t}}$ を使って表すと、

$$\text{Softmax}(z_1, z_2) = \begin{pmatrix} \frac{e^{z_1}}{\sum_{i=1}^2 e^{z_i}} \\ \frac{e^{z_2}}{\sum_{i=1}^2 e^{z_i}} \end{pmatrix} = \begin{pmatrix} \frac{e^{z_1 - z_2}}{e^{z_1 - z_2} + e^{z_2 - z_2}} \\ \frac{e^{z_2 - z_2}}{e^{z_1 - z_2} + e^{z_2 - z_2}} \end{pmatrix} = \begin{pmatrix} \frac{1}{1 + e^{-w}} \\ \frac{1}{1 + e^w} \end{pmatrix} = \begin{pmatrix} \sigma(w) \\ 1 - \sigma(w) \end{pmatrix}$$

確率論的多値分類 > モデル > 予測器

ソフトマックスと最大事後確率則でモデルを作ることができる

- スコアリングモデル $s_{\theta}(x)$ を，確率値ベクトルのモデル $g_{\theta}(x)$ に変換する

$$g_{\theta}(x) = \text{Softmax}(s_{\theta}(x))$$

- $g_{\theta}(x)$ の学習は後述

- スコアリングを予測ラベルにするための決定規則には最大事後確率則を使う

$$\tau(p) = \arg \max_{1 \leq k \leq K} p_k$$

- 結局のところ，予測器は，次のようになる．

$$f_{\theta}(x) = \tau(g_{\theta}(x))$$

Softmax($s_{\theta}(x)$)



確率論的多値分類＞ベクトルを出力するモデル

モデル $s_{\theta}(x)$ としてクラス数ぶんのパラメーター線型モデルを用意

- クラスごとに線型関数 $\theta_k^{\top} \phi(x)$ を用意（特徴量変換 ϕ は共通）
- 出力値を並べれば， K 次元のベクトルを出力する関数になる

$$\begin{array}{c} x \mapsto \begin{bmatrix} \theta_1^{\top} \phi(x) \\ \theta_2^{\top} \phi(x) \\ \vdots \\ \theta_K^{\top} \phi(x) \end{bmatrix} \\ s_{\theta}(x) \end{array}$$

確率論的多値分類 > 学習方法

近似ターゲットが $p(y|\mathbf{x})$ になるように交差エントロピーをリスクとして学習

- $g_{\boldsymbol{\theta}}(\mathbf{x}) \simeq p(y|\mathbf{x})$ となるように学習したいので、最尤法ベースの学習をすればよい

$$\hat{R}(g_{\boldsymbol{\theta}}) = -\frac{1}{n} \sum_{i=1}^n \log g_{\boldsymbol{\theta}}(\mathbf{x}_i)[y_i]$$

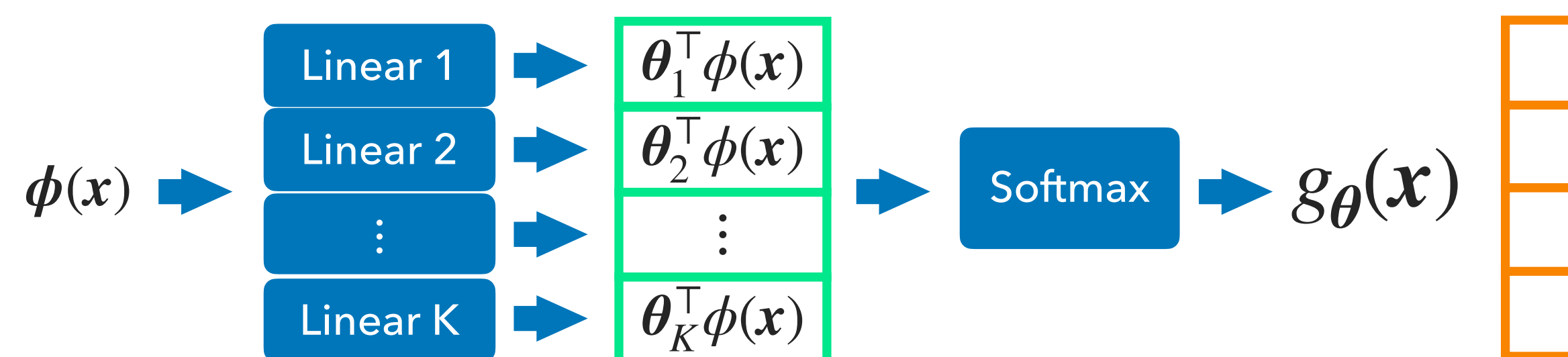
- 例) L2正則化付き最尤法

$$L(\boldsymbol{\theta}) = \hat{R}(g_{\boldsymbol{\theta}}) + \lambda \|\boldsymbol{\theta}\|^2$$

確率論的多値分類 > 多クラスロジスティック回帰

線型モデル + ソフトマックス関数 + 最尤法ベースの学習

- 多クラスロジスティックモデル (multi-class logistic model)
 - クラス毎に別々の線型モデルを用意し，出力はまとめてSoftmaxで変換



- 多クラスロジスティック回帰 (multi-class logistic regression)
 - 多クラスロジスティックモデルを最尤法ベースで学習させて， $p(y|x)$ を近似

$$R(g_\theta) = \mathbb{E} [-\log g_\theta(X)[Y]]$$

確率論的多値分類＞ソフトマックスと最尤法＞勾配

ソフトマックスと負の対数尤度を使うときの、勾配を導出する

- K 次元出力のスコアリング関数 $s_{\theta}(\mathbf{x})$ を用いて、モデル $g_{\theta}(\mathbf{x}) = \text{Softmax}(s_{\theta}(\mathbf{x}))$
- データが $(\mathbf{x}, y)$ のときの損失関数（負の対数尤度） $\ell((\mathbf{x}, y), g_{\theta}) = -\log g_{\theta}(\mathbf{x})[y]$
- θ に関する勾配 $\nabla \ell((\mathbf{x}, y), g_{\theta})$ は、 $\log$ によって Softmax の分子分母を分解すれば、

$$\nabla \ell((\mathbf{x}, y), g_{\theta}) = -\nabla s_{\theta}(\mathbf{x})[y] + \nabla \log \left(\sum_k \exp(s_{\theta}(\mathbf{x})[k]) \right)$$

$$\nabla \log \left(\sum_k \exp(s_{\theta}(\mathbf{x})[k]) \right) = \frac{\sum_k \exp(s_{\theta}(\mathbf{x})[k]) \cdot \nabla s_{\theta}(\mathbf{x})[k]}{\sum_j \exp(s_{\theta}(\mathbf{x})[j])} = \sum_k g_{\theta}(\mathbf{x})[k] \cdot \nabla s_{\theta}(\mathbf{x})[k]$$

確率論的多値分類＞ソフトマックスと最尤法＞勾配

ロジスティックモデルの負の対数尤度の勾配は，次のように書ける

■ 勾配は

$$\nabla \ell((\mathbf{x}, y), g_{\theta}) = -\nabla s_{\theta}(\mathbf{x})[y] + \sum_k g_{\theta}(\mathbf{x})[k] \cdot \nabla s_{\theta}(\mathbf{x})[k]$$

■ 特に，多クラスロジスティック回帰なら， $s_{\theta}(\mathbf{x})[k] = \boldsymbol{\theta}_k^{\top} \boldsymbol{\phi}(\mathbf{x})$ なので，

$$\nabla_{\boldsymbol{\theta}_k} \ell((\mathbf{x}, y), g_{\theta}) = (g_{\theta}(\mathbf{x})[k] - \mathbf{1}\{k = y\}) \boldsymbol{\phi}(\mathbf{x})$$

確率論的多値分類＞ソフトマックスと最尤法＞最適化

この場合も解の公式は得られないので、勾配法による最適化を行う

- 多クラスロジスティック回帰なら、 $L(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n (-\log g_{\boldsymbol{\theta}}(\mathbf{x}_i)[y_i])$ に対して、

$$\nabla_{\boldsymbol{\theta}_k} L(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n (g_{\boldsymbol{\theta}}(\mathbf{x}_i)[k] - \mathbf{1}\{k = y_i\}) \boldsymbol{\phi}(\mathbf{x}_i)$$

- 勾配法では、 $-\nabla L(\boldsymbol{\theta})$ 方向に更新する。L2正則化付きなら、重み減衰も加えて、

$$-\nabla_{\boldsymbol{\theta}_k} L(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n (\mathbf{1}\{k = y_i\} - g_{\boldsymbol{\theta}}(\mathbf{x}_i)[k]) \boldsymbol{\phi}(\mathbf{x}_i) - 2\lambda \boldsymbol{\theta}_k$$

【補足】 ここで、 $\nabla L(\boldsymbol{\theta}) = (\nabla_{\boldsymbol{\theta}_1} L(\boldsymbol{\theta}), \dots, \nabla_{\boldsymbol{\theta}_K} L(\boldsymbol{\theta}))^\top$.

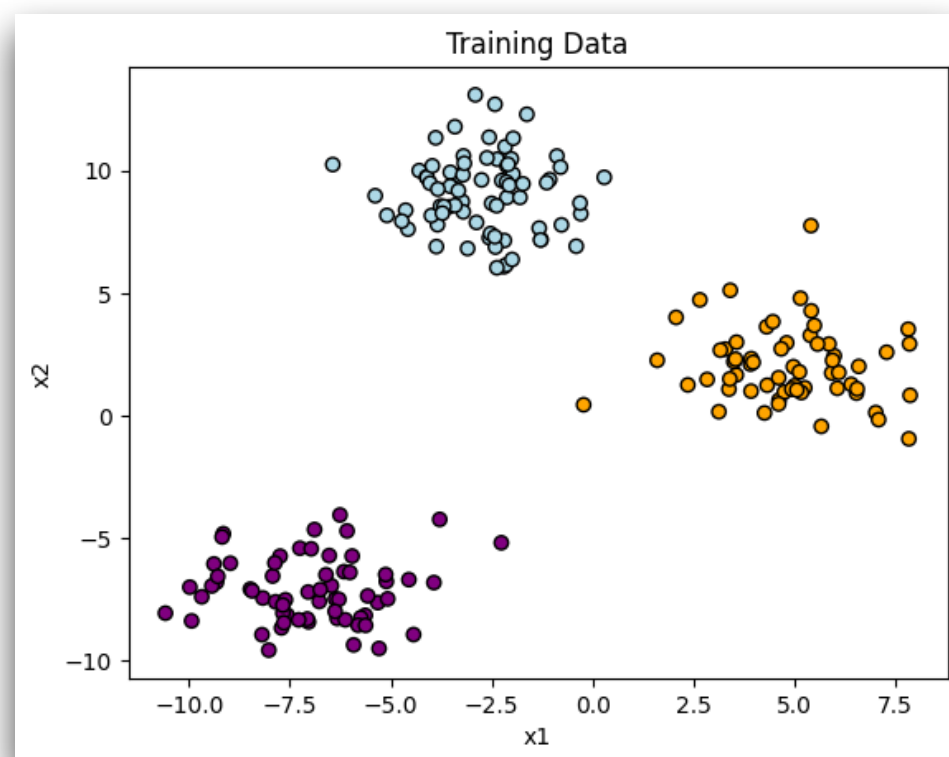
確率論的多値分類 > 実験 > 設定

L2正則化付き多クラスロジスティック回帰（勾配降下法で最適化）

タスク・学習戦略

分類タスク・教師付き学習

特徴量	ラベル
x_1	y_1
x_2	y_2
$\vdots$	$\vdots$
$\{(X_i, Y_i)\}_{i=1}^n \stackrel{\text{i.i.d.}}{\sim} p$	



訓練データ

モデル

多クラス
ロジスティックモデル

$$g_{\theta}(x) = \text{Softmax}(s_{\theta}(x))$$

$$s_{\theta}(x)[k] = \theta_k^{\top} \phi(x)$$

- 各々の中身は30次の多項式

目的関数

負の対数尤度+正則化

$$L(\theta) := \frac{1}{n} \sum_{i=1}^n \ell((x_i, y_i), g_{\theta}) + \lambda \|\theta\|^2$$

- 正則化係数 λ は $\{10^{-5}, \dots, 10^0\}$ から選択
- ハイパーパラメータ選択は早期停止時点の検証損失値を使用

最適化手法

勾配降下法

$$\hat{\theta} \leftarrow \hat{\theta} - \eta \nabla L(\hat{\theta})$$

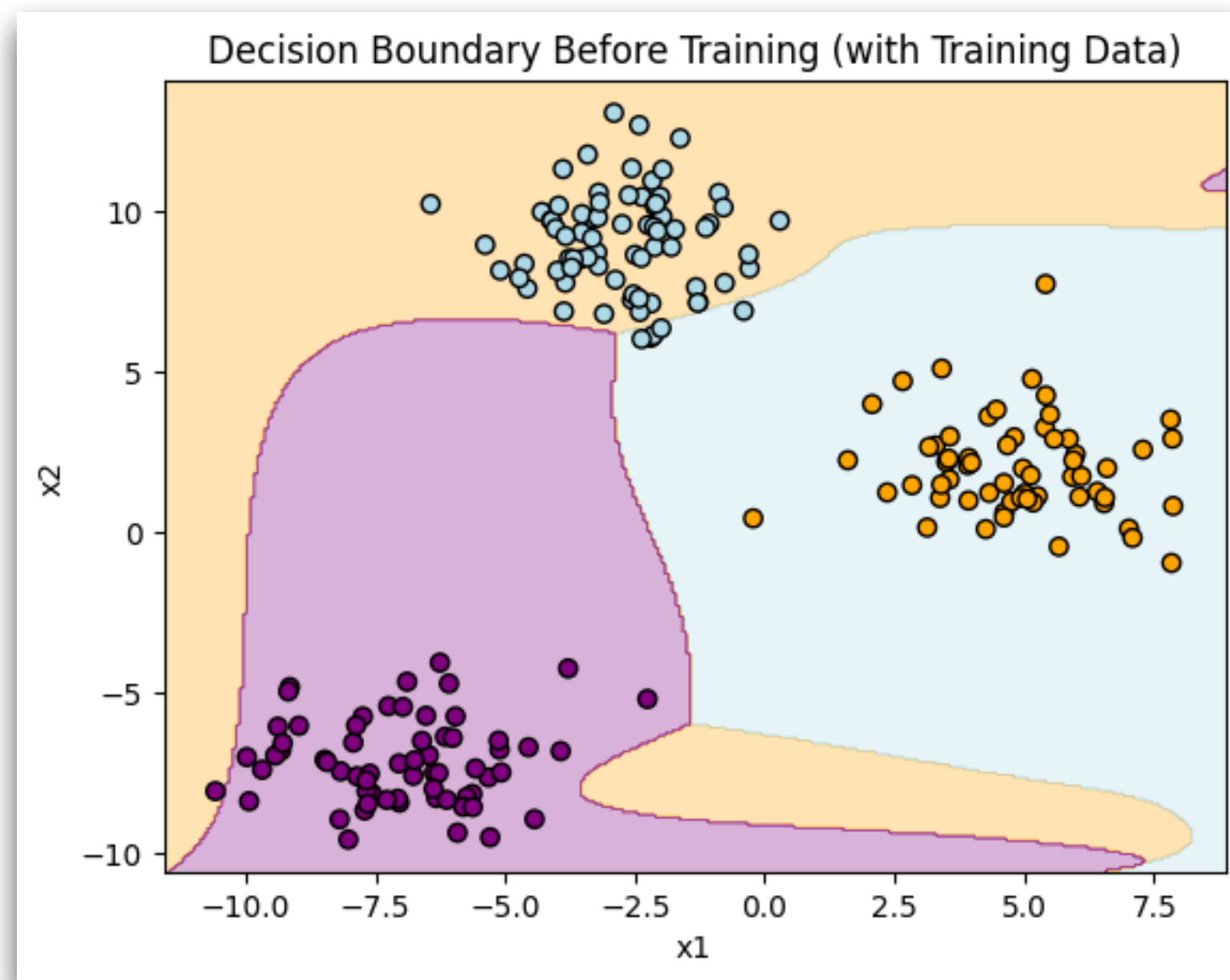
- 学習率 η は 0.1
- 更新回数上限10000回
- 早期停止あり（直近10回の改善幅が 10^{-3} 未満で停止）

【補足】 学習の安定化のために、 ϕ の高次の項は、各項の次数 r に応じて、それぞれ予め 10^{-r} 倍したものとした（多項式モデルとしての表現力には影響なし）。

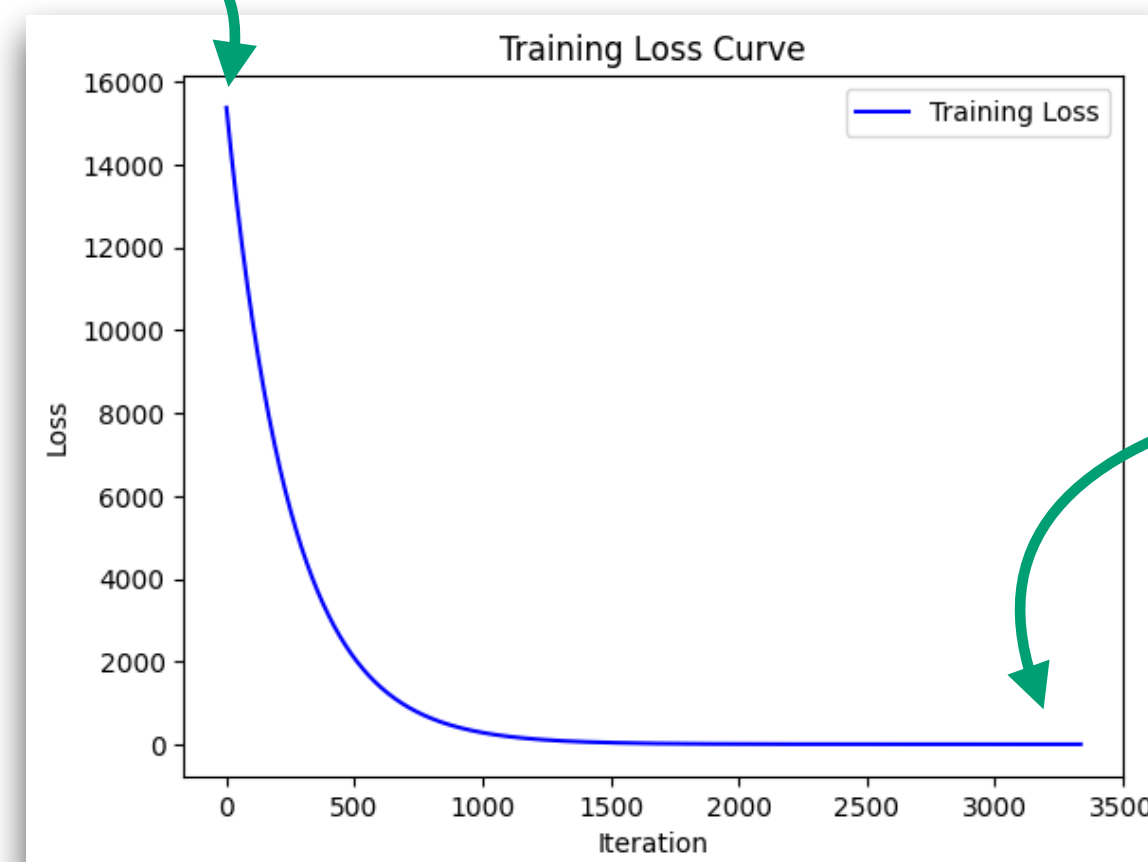
【実験ソースコード】 [番号]_multiclass_classification.ipynb

確率論的多値分類＞実験＞結果

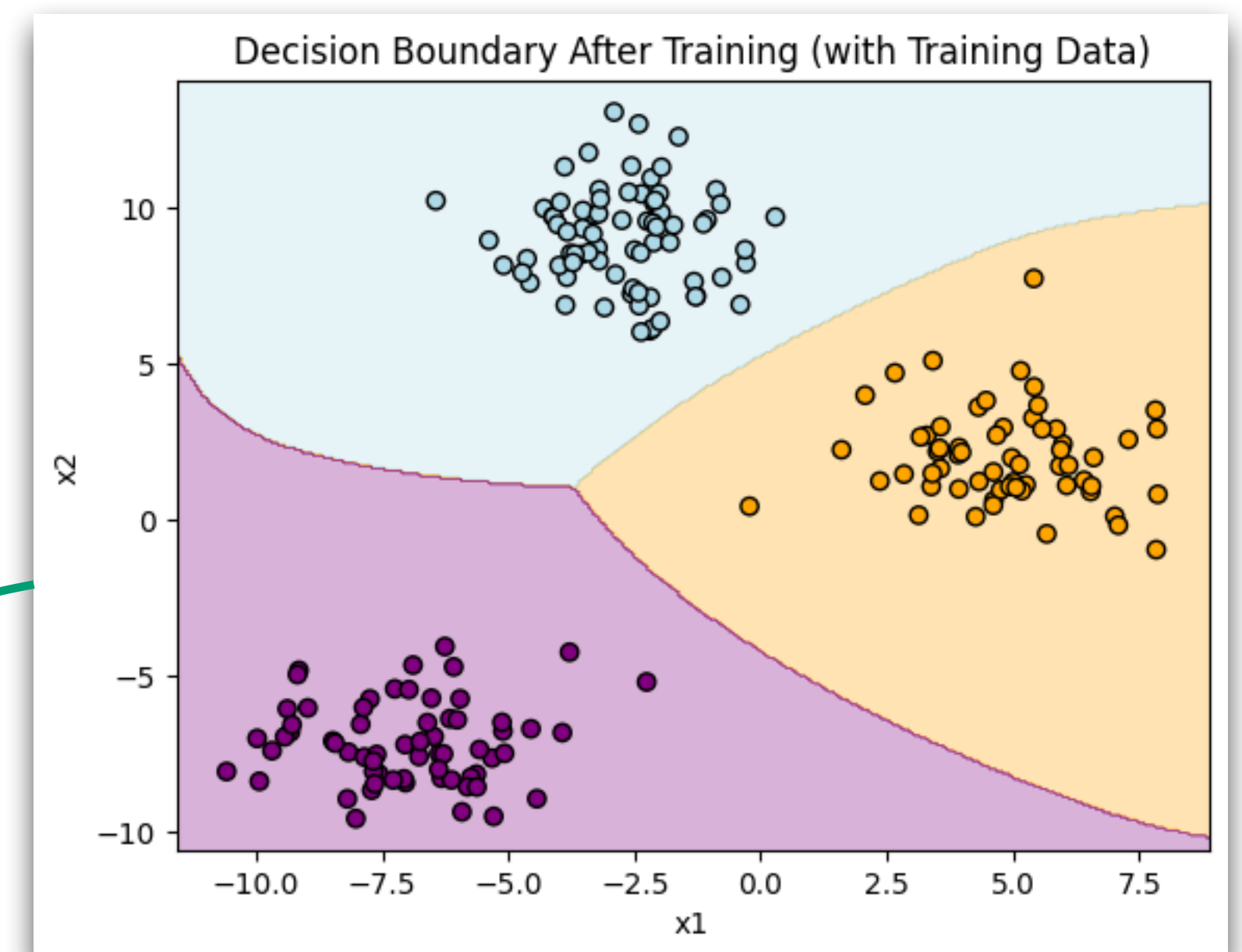
訓練前の初期化した時点での予測器と，訓練後の予測器



訓練前のモデルと訓練標本



訓練誤差

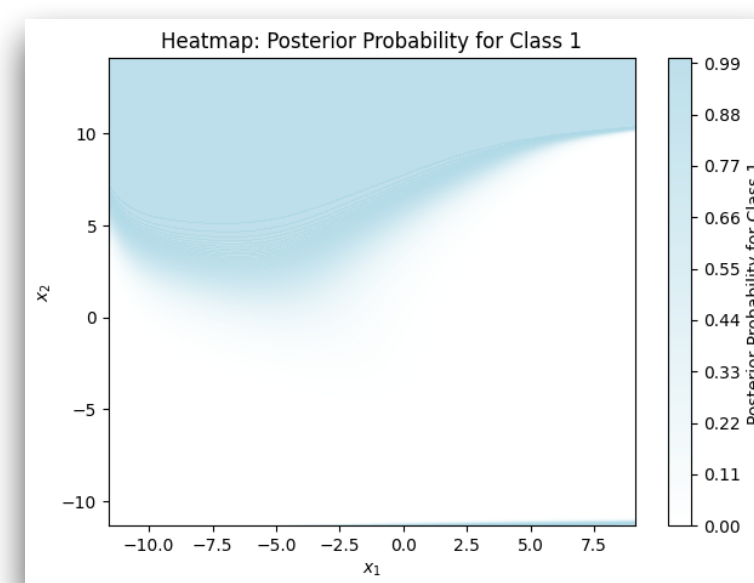
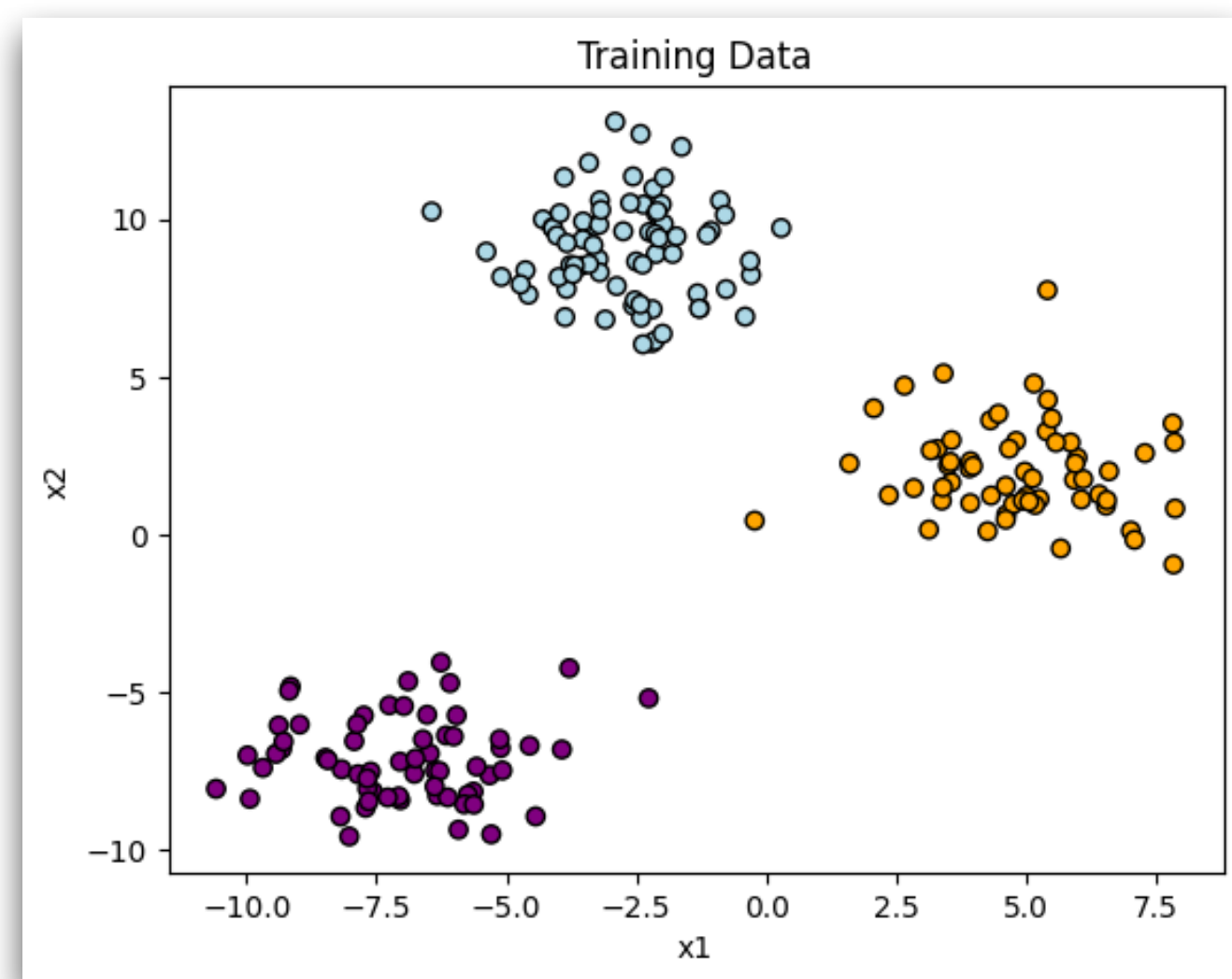


訓練後のモデルと訓練標本

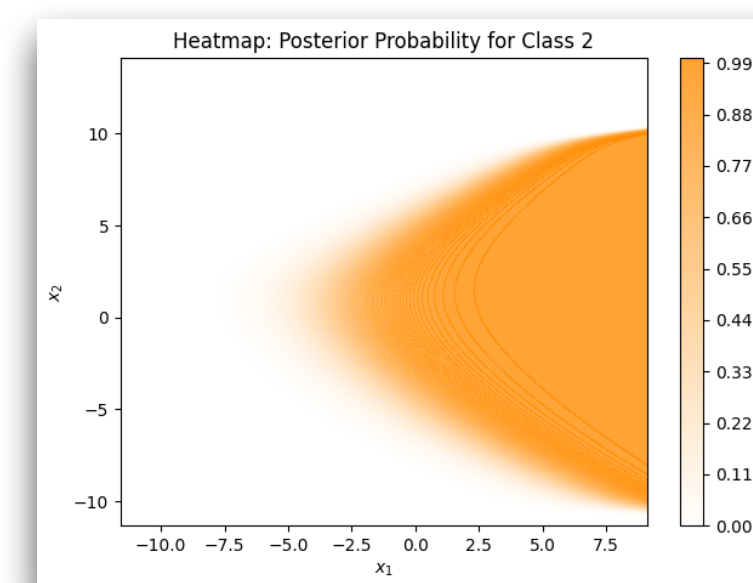
確率論的多値分類 > 実験 > 結果

妥当そうなスコアリングモデルが学習できている

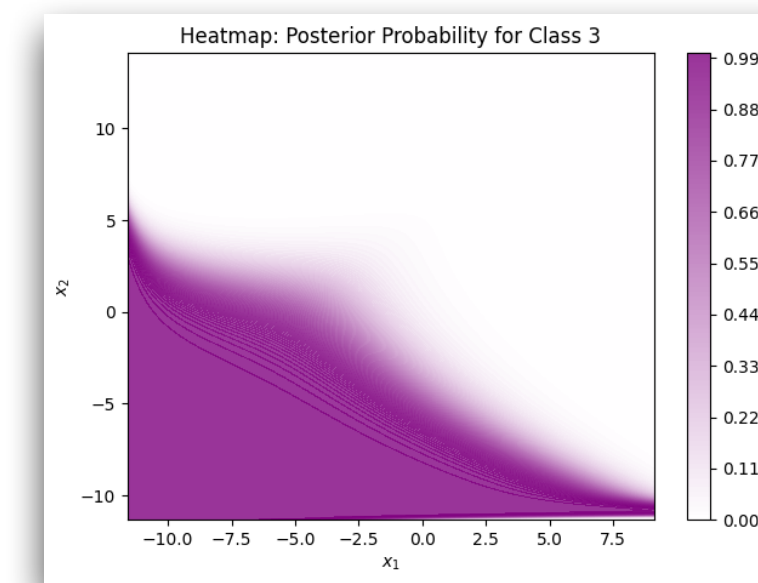
- 各クラスの主要な集団のあたりでは1に近く、他のクラスの領域では0に近くなる
 - その境目付近ではグラデーションがついている



クラス1



クラス2



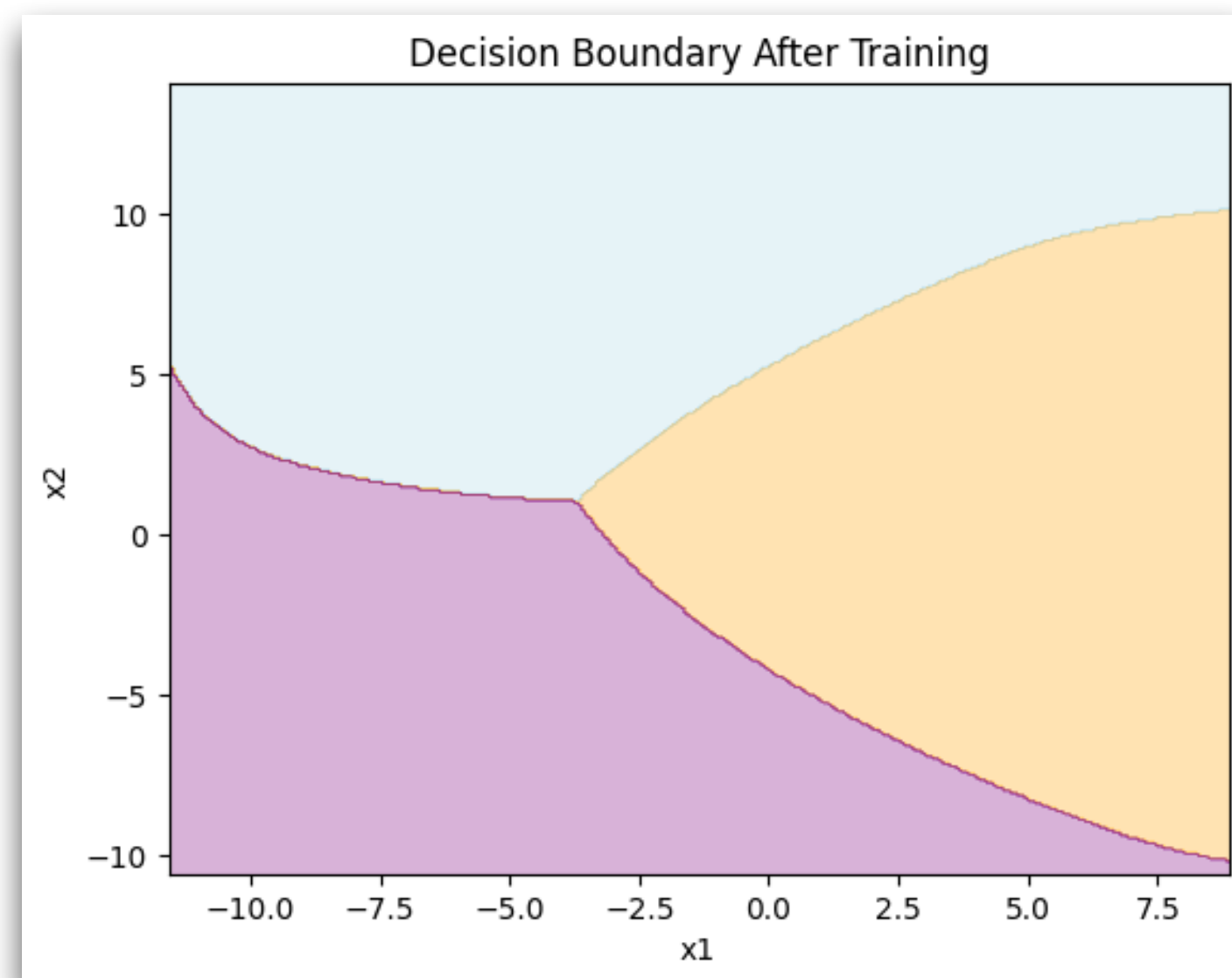
クラス3

学習済みモデル $g_{\hat{\theta}}$ が出力した確率値スコア

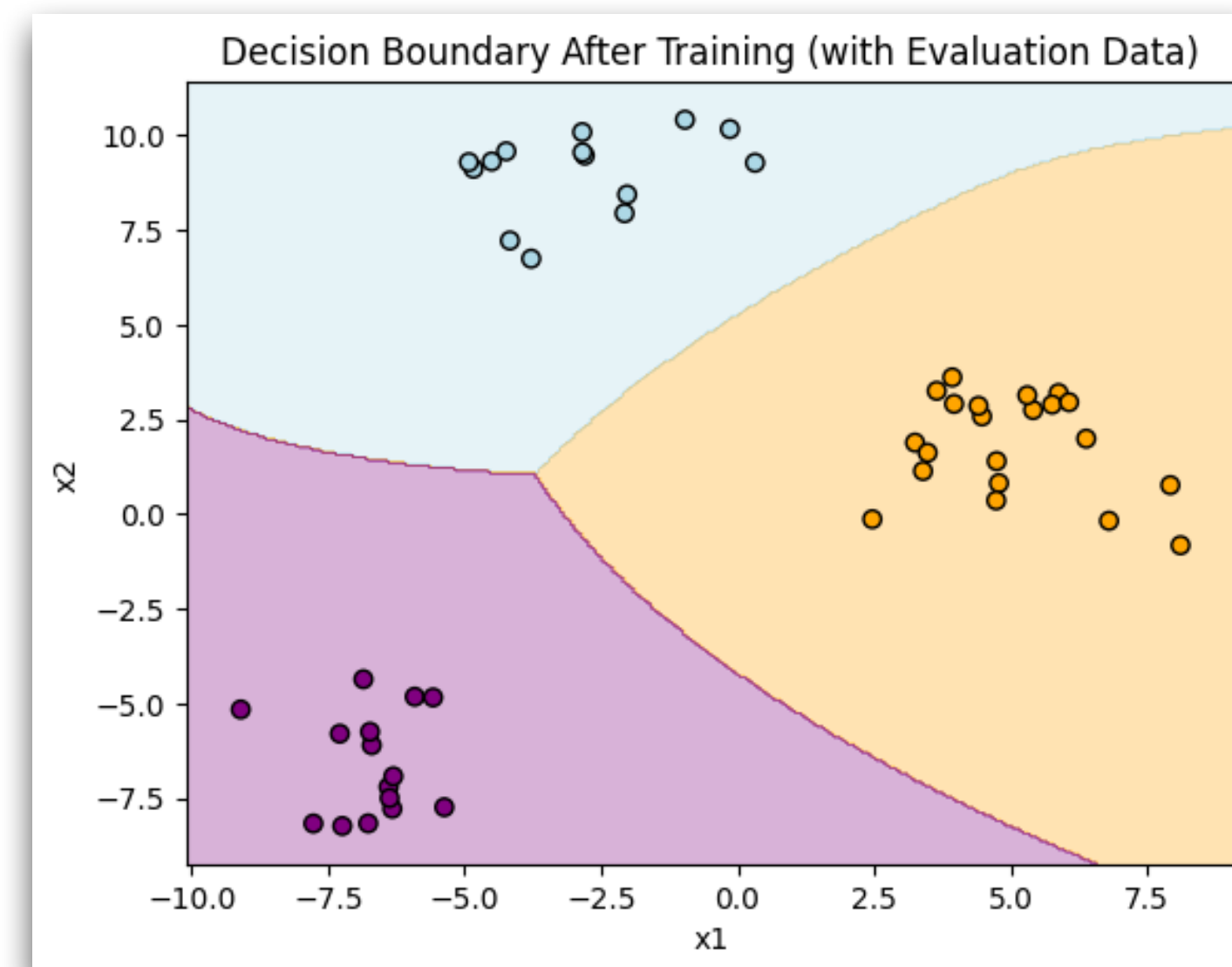
確率論的多値分類＞実験＞結果

予測器としても良さそうなものになっている

- 学習後のモデルに決定規則を合成した予測器 $f_{\hat{\theta}}(\mathbf{x}) := \tau(g_{\hat{\theta}}(\mathbf{x}))$ の様子



分類器としての出力
(スコア最大値を与えるラベルを予測)



評価用データでの予測と正解ラベル

確率論的多値分類 > まとめ



確率論的多値分類の標準的な枠組みを説明した

■ 確率論的多値分類

- ソフトマックスを使うことでモデルの出力値が確率値になるようにする
- 交差エントロピーをリスクとすることで、モデルに $p(y|\mathbf{x})$ を近似させる
(最尤法ベースの学習)
- 予測ラベルの選択をする決定規則としては、最大事後確率則を使う
- 最適化は勾配法ベースで行う

■ もちろん、この組み合わせでなくとも良いという注意は二値分類のときと同じ

確率論的多値分類＞ソフトマックス関数＞補足



ソフトマックスという名前は, $\arg \max$ に似ていることからきている

- LogSumExpという関数とSoftmaxの関係を使って説明する

確率論的多値分類 > ソフトマックス関数 > LogSumExp

LogSumExp関数というものを考える

- 「対数・総和・指数」という関数 (log-sum-exponential)

$$\text{LogSumExp}(z_1, \dots, z_K) = \log \left(\sum_{k=1}^K \exp(z_k) \right)$$

- これは max に似ている
 - 例) $\text{LogSumExp}(2, 5, 8, 0) \simeq 8.0513$ となり, おおよそ最大値 8 に一致

演習課題



実際、次が成立する

- 次を示せ.

$$\max\{z_1, \dots, z_K\} \leq \text{LogSumExp}(z_1, \dots, z_K) \leq \max\{z_1, \dots, z_K\} + \log(K)$$

確率論的多値分類 > ソフトマックス関数 > LogSumExp

SoftmaxとLogSumExpの関係

- SoftmaxはLogSumExpの勾配になっている

$$\frac{\partial}{\partial z_k} \text{LogSumExp}(z_1, \dots, z_K) = \text{Softmax}(z_1, \dots, z_K)[k]$$

- 理由：

- Softmaxの定義から, $\text{Softmax}(\mathbf{z})[k] = \exp(z_k - \text{LogSumExp}(\mathbf{z}))$
- 裏を返せば, $\text{LogSumExp}(\mathbf{z}) = z_k - \log \text{Softmax}(\mathbf{z})[k]$.
- これと, $\frac{\partial}{\partial z_k} \text{Softmax}(\mathbf{z})[k] = \text{Softmax}(\mathbf{z})[k] \cdot (1 - \text{Softmax}(\mathbf{z})[k])$ から上記が従う.

確率論的多値分類＞ソフトマックス関数＞性質

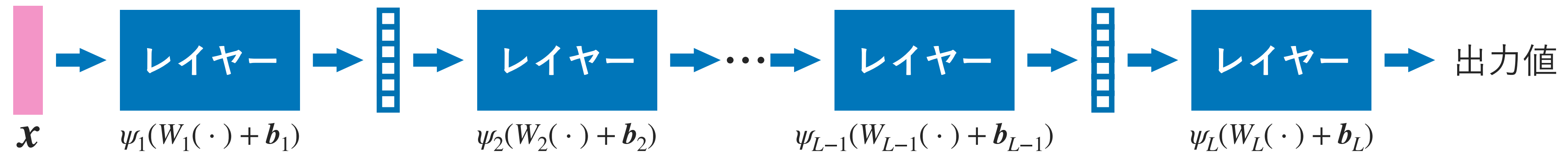
LogSumExpはmaxに似ており，その勾配であるSoftmaxはargmaxに似ている

- max を最も上昇させる入力の微小変化は，最大値の変数の方向
 - 例) $\max(2, 5, 8, 0) = 8$ であり，第3成分を増加させなければ上昇しない
- LogSumExp (maxに似ている) の，勾配であるSoftmaxは，arg max に似ている
 - 例) $\text{Softmax}(2, 5, 8, 0) \approx [0.002, 0.047, 0.950, 0.000]$ となり，最大値の第3成分がほぼ1，それ以外がほぼ0となっている → 最大値成分を指し示すベクトル

ニューラルネット＞モデルの構造

順伝播型ニューラルネットワークの構造

- レイヤーという変換を積み重ねたもの



- 別の見方：ユニットという部品で構成されている（○で表す）

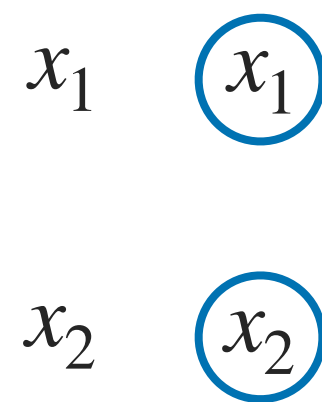


- レイヤーごとに、 W の列の数（= b の次元）と同じ個数のユニットを並べる
- 入力のところにも、入力特徴量の次元と同じ個数のユニットを並べる

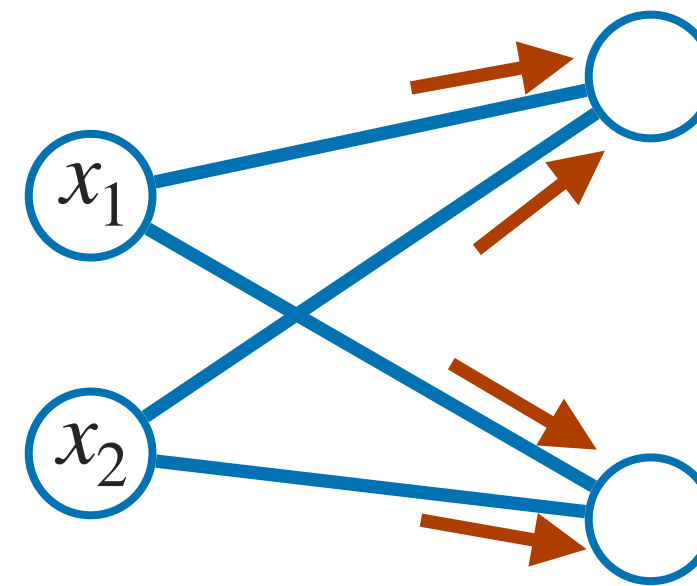
ニューラルネット＞モデルの構造

ユニットという見方で説明すると，ニューラルネットの計算の流れは……

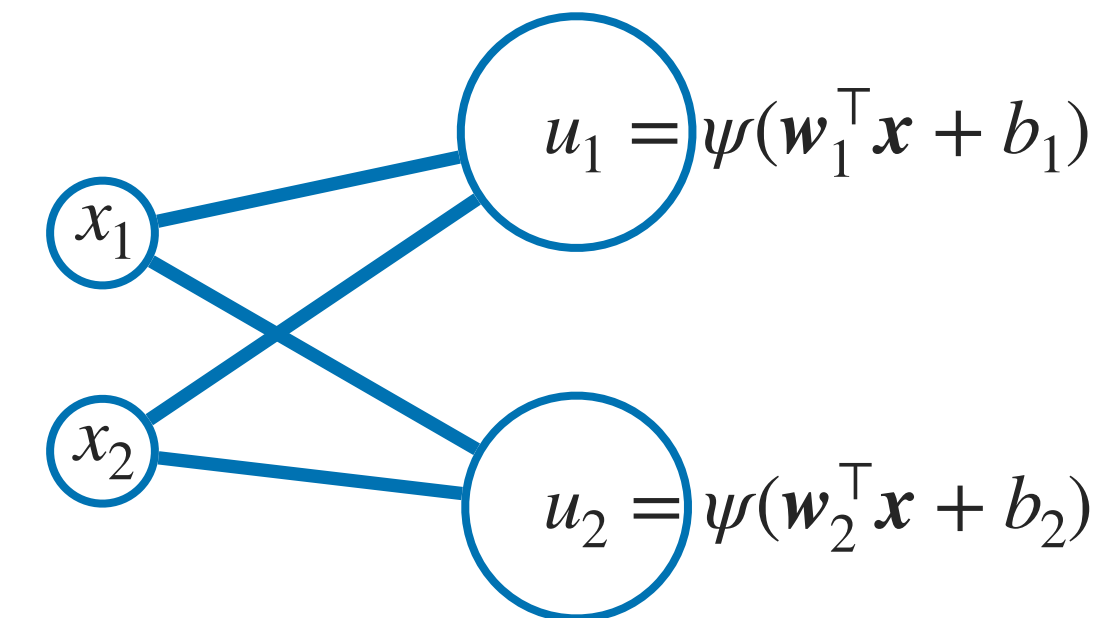
- ユニットは自分の活性値を保持し，次のユニットに値を渡す



まず入力特徴量の値が
入力層のユニットに
そのままセットされる



ユニットの現在の値は
後続のユニットへ渡される



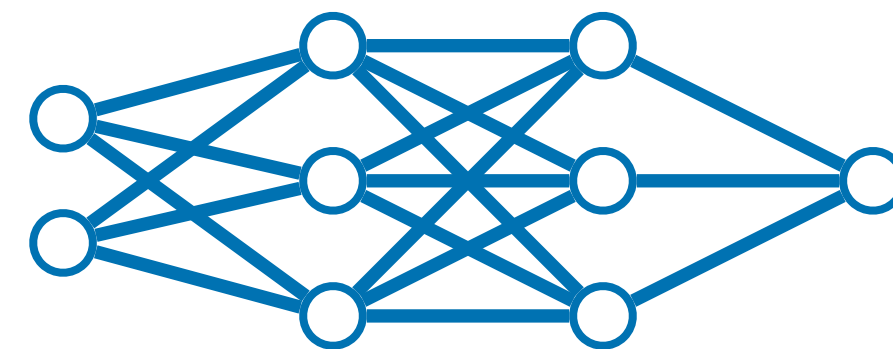
受け取った値から
計算された活性値が
保持される

- ユニットが保持する「現在の値」がまた後続のユニットに渡されていく

ニューラルネット＞モデルの構造

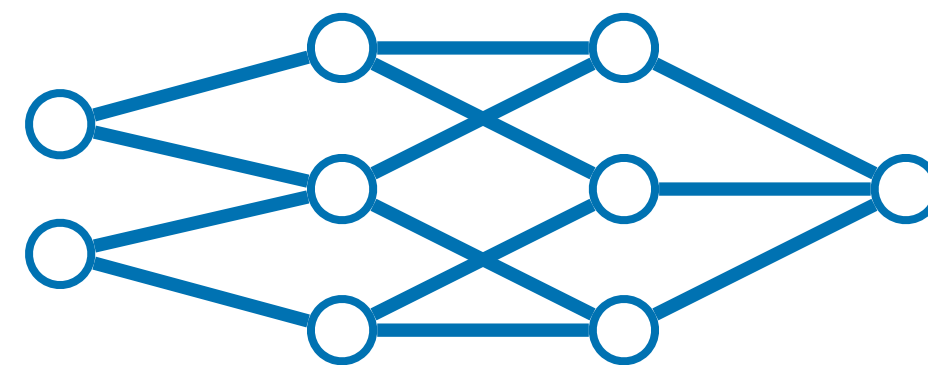
ユニットの繋ぎ方は自由

- 順伝播型ニューラルネットのレイヤーは全結合層だった
 - 前の層の**全ての**ユニットから，今の層の**全ての**ユニットに線が繋がっている



全結合

- ネットワークの設計のパターンはアーキテクチャ（architecture）と呼ばれる



あえて一部だけ結合

ニューラルネットワーク

ニューラルネットワーク



次は、ニューラルネットワークとよばれる柔軟なモデルを解説します

- ニューラルネットワークというモデルは、非線形モデルの主要なものの一つであり、実用的にも広く使われている。
- どういう学習の仕組みになっているかを理解しておくと、ここまでの内容と、近年のニューラルネットワークを用いた高度な機械学習モデルに関する話がつながって、裏の仕組みがイメージしやすくなる

講義の前提知識 > ノーテーションの注意

この講義では、1変数関数は断りなく多次元のベクトルに適用できるとする

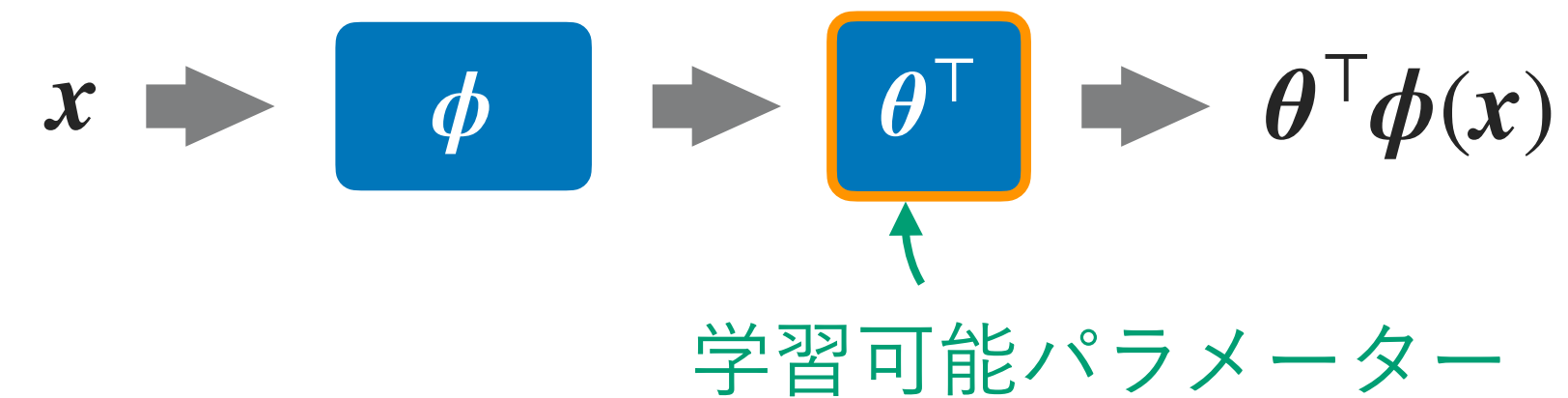
- 多次元の出力をするモデルを扱う上で、便利な記号を用意しておく.
- ベクトル $\boldsymbol{a}$ に対して、1変数関数 ψ を成分ごとに適用したものを $\psi(\boldsymbol{a})$ で表す.

$$\psi(\boldsymbol{a}) := \begin{bmatrix} \psi(a_1) \\ \psi(a_2) \\ \vdots \\ \psi(a_d) \end{bmatrix}$$

ニューラルネット > 複雑なモデルと特徴量

ニューラルネットのポイントは、変換の階層性

- 線型モデルでは、出力値へのパラメーターの関与は、出力直前の一箇所だけ



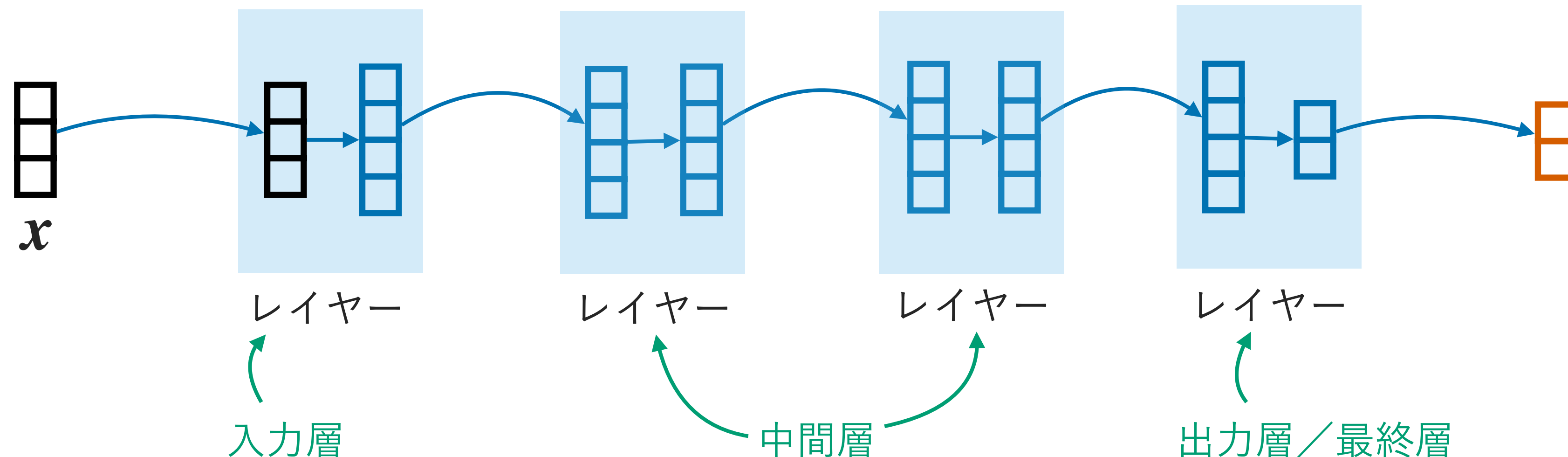
- ニューラルネットワーク (neural network, NN) では……



ニューラルネット > 基本の構成要素 > 順伝播型NN

ニューラルネットワークの中でも、最もプレーンなもの

- 順伝播型ニューラルネットワーク (Feed-forward neural network, FNN)
 - レイヤー (layer, 層) とよばれる部品を直列に繋げて複雑な関数を作る



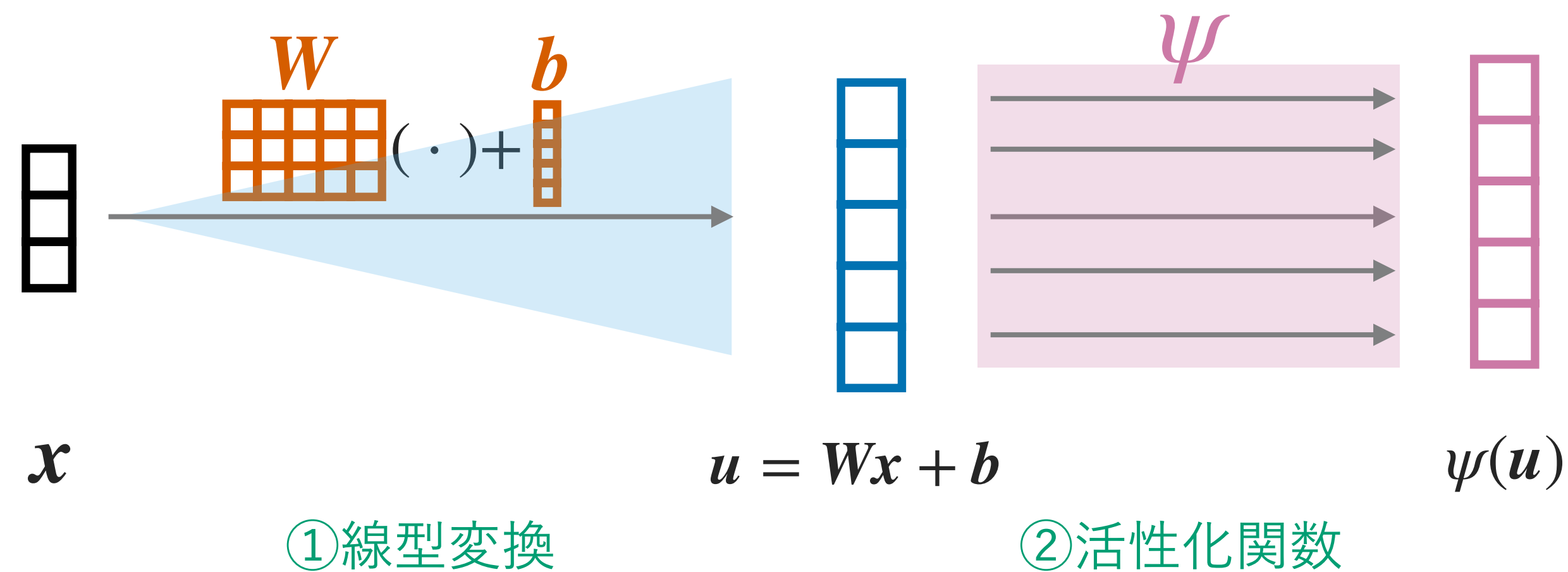
【用語】 特別な味付けがないという比喻で、「バニラ」ニューラルネットワーク (vanilla neural network) とも呼ばれる。多層パーセプトロン (Multi-layer Perceptron, MLP) とも呼ばれる。伝播は、伝搬 (でんぱん) ではなく伝播 (でんぱ) であることに注意。

ニューラルネット > 基本の構成要素 > レイヤー

各レイヤーの中身は、「線型変換を施してから活性化関数」でできている

■ 各レイヤー内の計算は $x \mapsto \psi(Wx + b)$ (W, b はパラメーター, ψ は固定)

- 線型変換 $x \mapsto Wx + b$ の後, 共通の ψ を成分毎に適用 $\psi(u) = [\psi(u_1), \dots, \psi(u_r)]^\top$
- W は重み, b はバイアス, ψ は活性化関数 (activation function) という



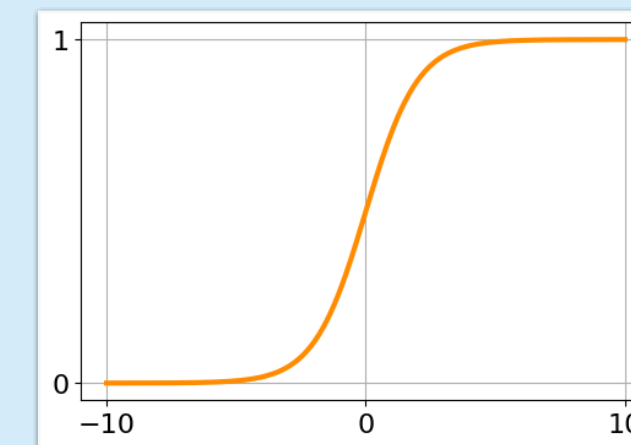
ニューラルネット > 基本の構成要素 > 活性化関数

よく使われる活性化関数（他にも多数提案されている）

Sigmoid

（標準ロジスティック関数）

$$\psi(u) = 1/(1 + \exp(-u))$$

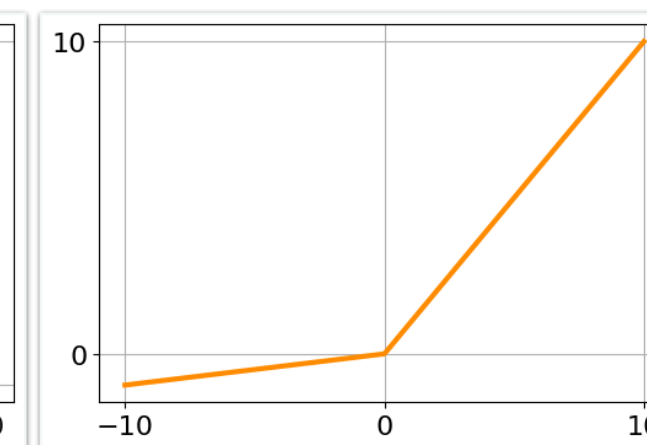
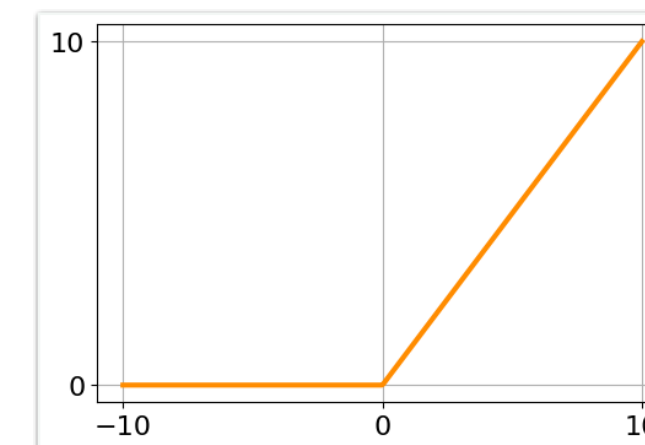


ReLU

$$\psi(u) = \max\{0, u\}$$

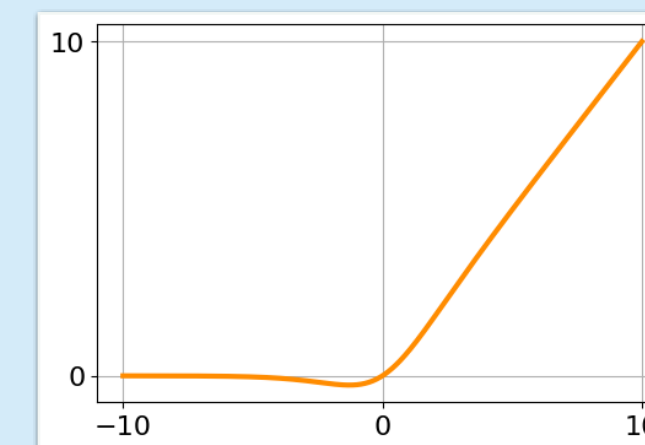
Leaky ReLU

$$\psi(u) = \max\{\alpha u, u\} \quad (0 \leq \alpha < 1)$$



Swish

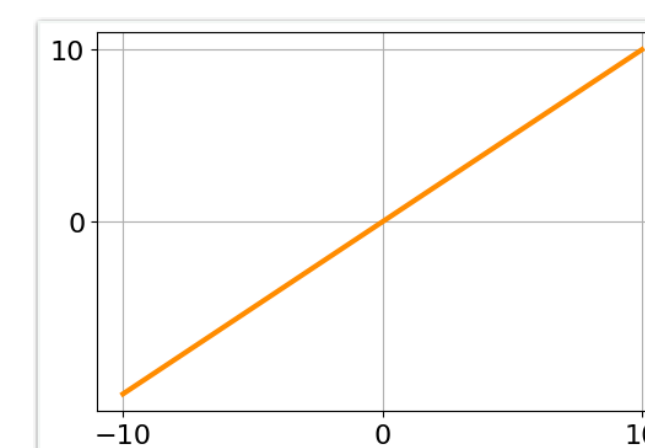
$$\psi(u) = u/(1 + \exp(-\beta u))$$



Linear activation

$$\psi(u) = u$$

（恒等写像）

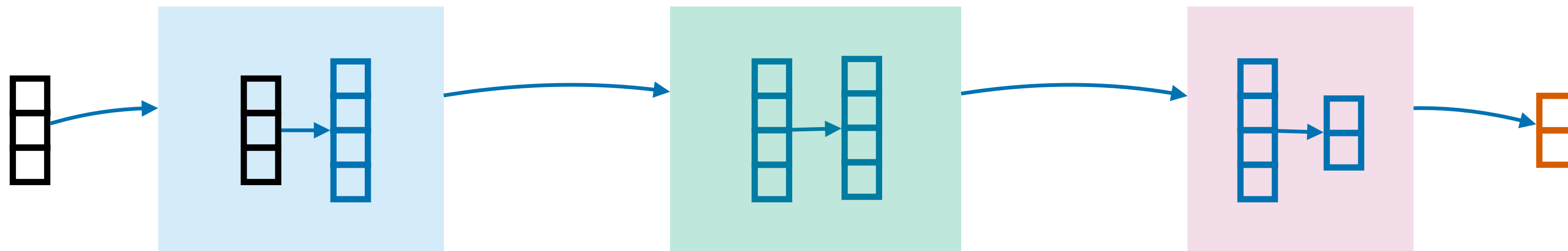


ニューラルネット > 基本の構成要素 > 順伝播型NN

このモデルは階層性がポイント

- 一つ一つのレイヤーは、さほど複雑な処理は行わない
 - 線型変換 + 成分ごとの活性化関数
 - しかし、それを多数重ねることで、全体としては複雑な関数を表現
- 各レイヤー内の処理には調整可能なパラメーター (W, b) が付いている

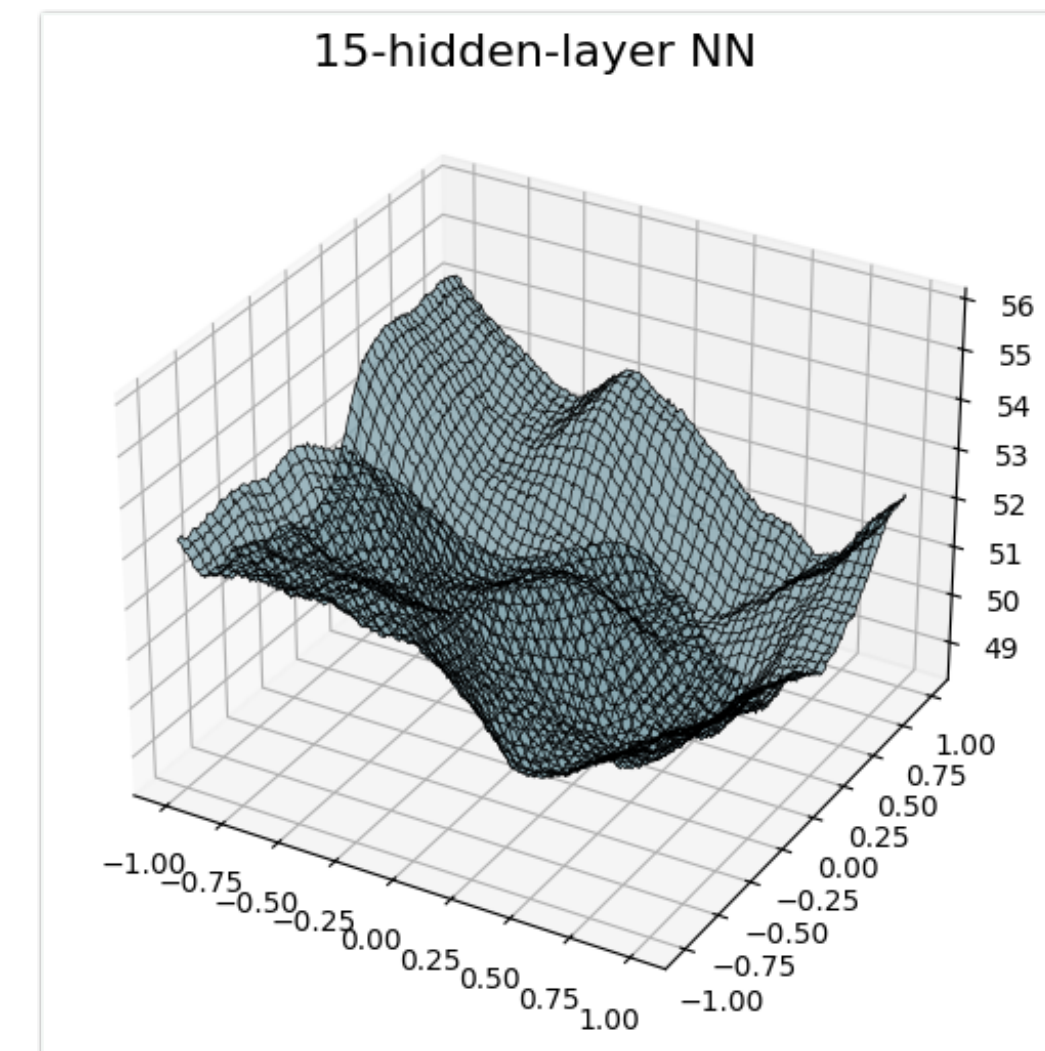
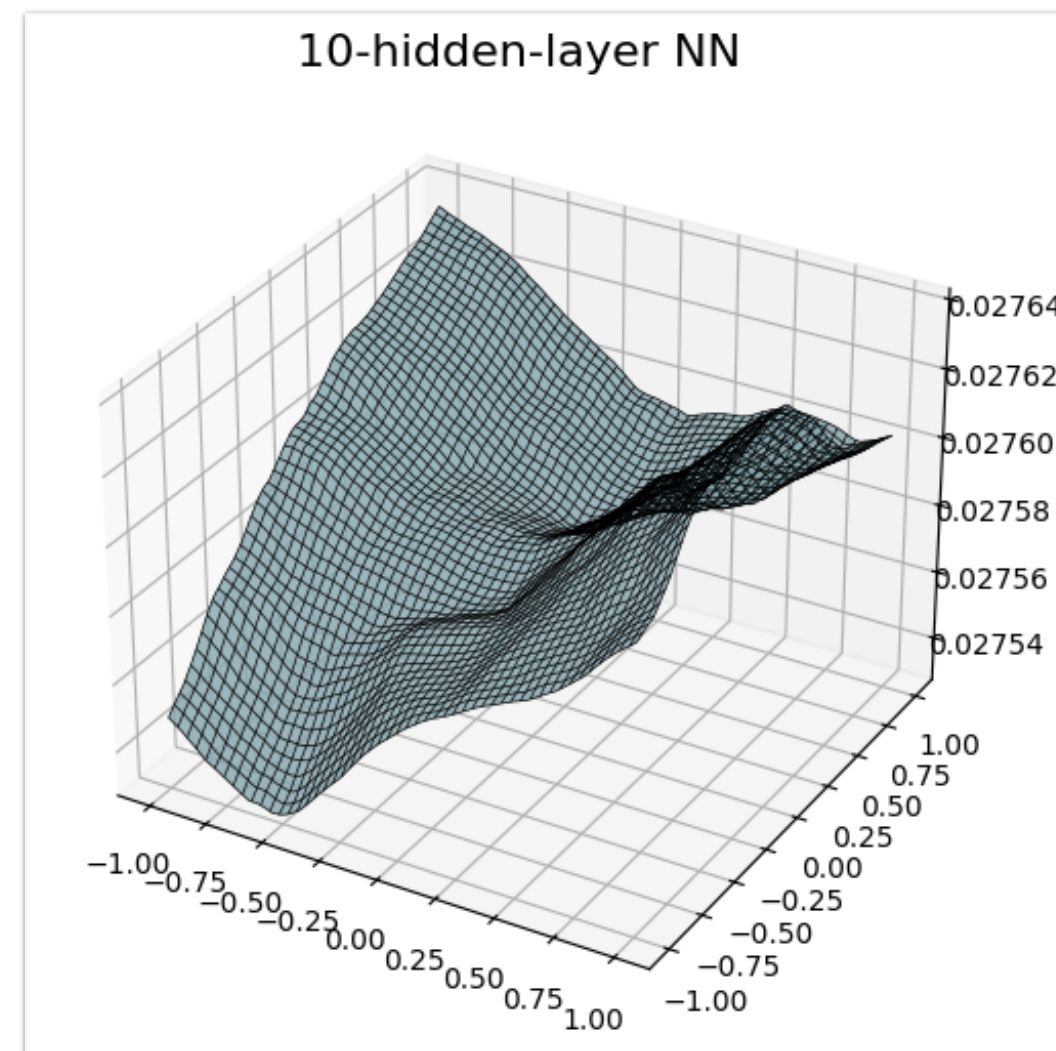
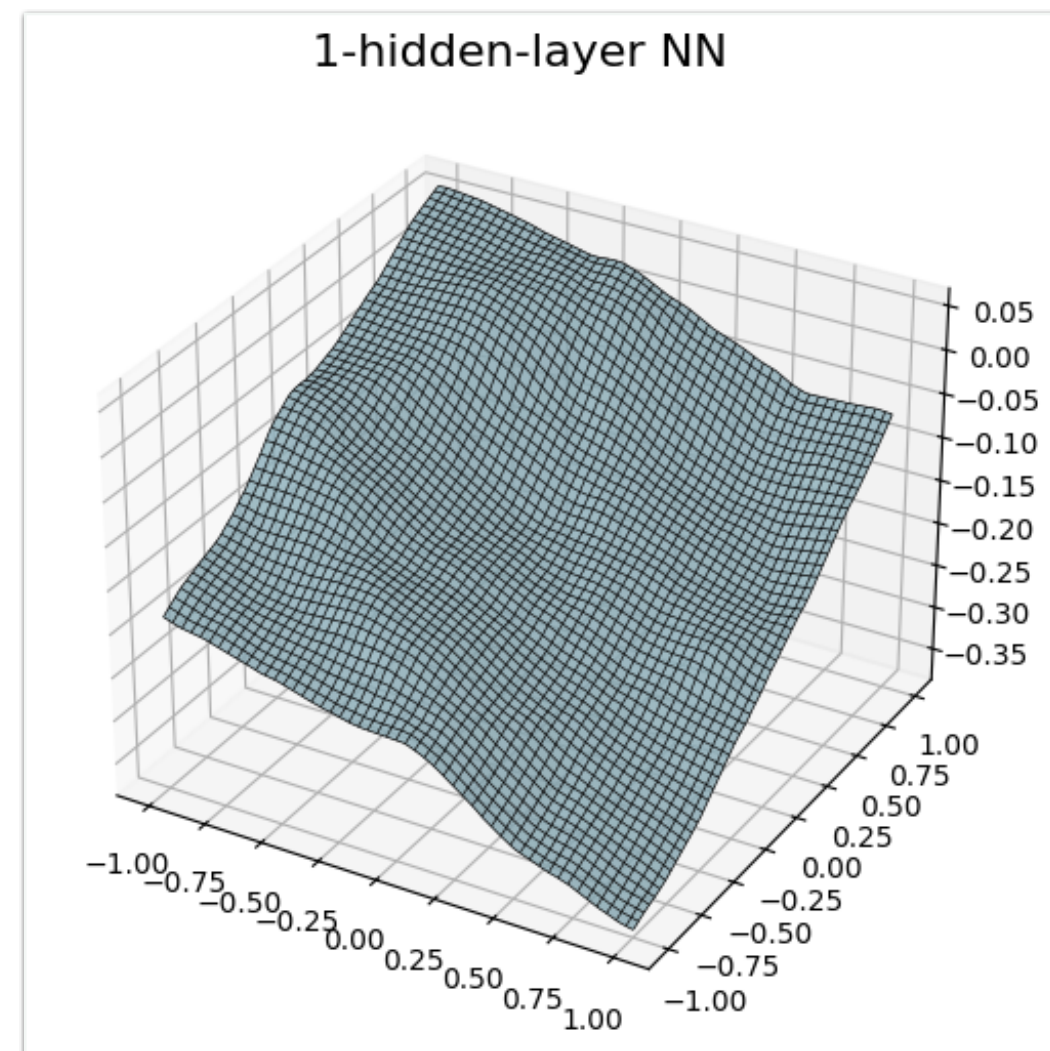
$$x \mapsto \psi_1(W_1(x) + b_1) \mapsto \psi_2(W_2(\psi_1(W_1(x) + b_1) + b_2) \mapsto \psi_3(W_3(\dots$$



ニューラルネット > 基本の構成要素

隠れ層を増やしていくと，関数として複雑化していく

- 既存の層は変えずに，ランダム初期化した層を順次追加していった場合

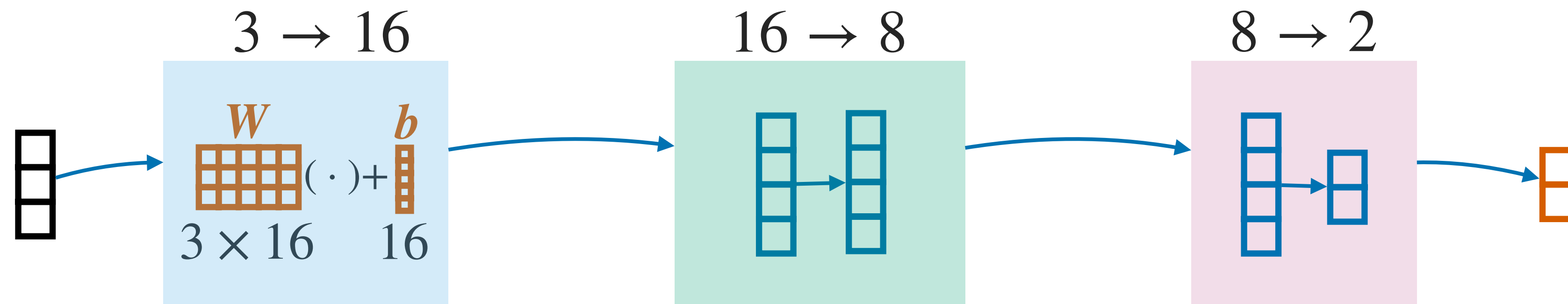


レイヤーの数

ニューラルネット > 基本の構成要素 > 順伝播型NN

パラメーターの形状：前の層の出力を受け取れて、次の層に渡せる必要がある

- 入力層・出力層のパラメーターは、モデルとしての入出力次元に合わせる
- 中間層の出力の次元数は任意に選択可能（ハイパーパラメーター）
 - 例）3次元入力→**16次元**→**8次元**→2次元出力
 - 隣り合うレイヤー間では、出力を受け渡せる必要がある



【注意】 入力層・出力層の次元数は、受け付ける入力次元数・欲しい出力次元数に合わせる必要がある。

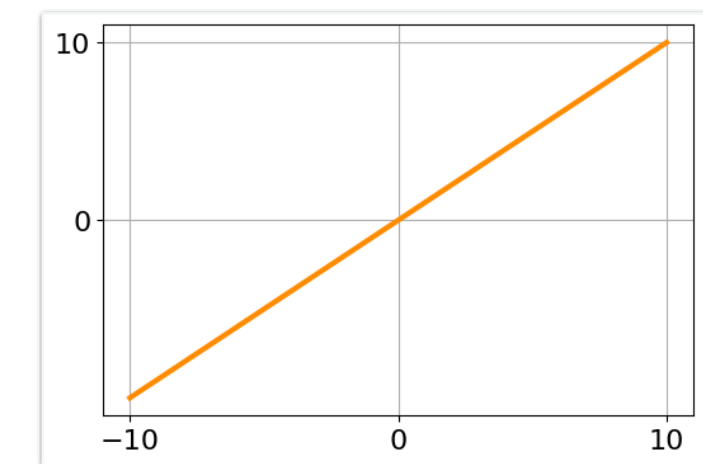
ニューラルネットワーク > 基本の構成要素 > 出力層

出力層の活性化関数は、確率値にしたいか否かなどで決まる

■ 出力層の活性化関数では、通常は以下が用いられる

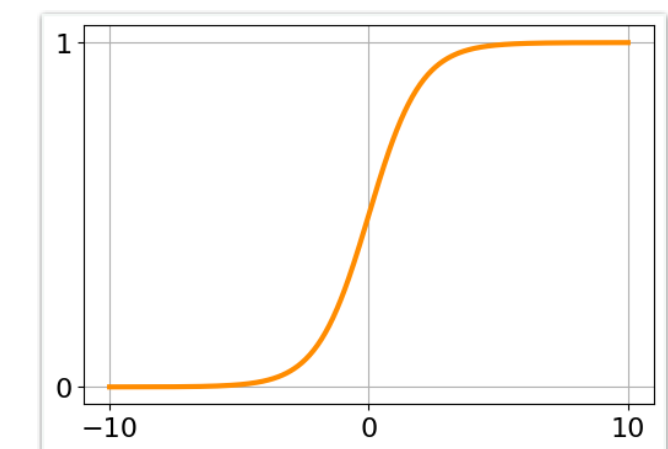
- 回帰 → 実数出力にしたいので恒等活性化関数

$$\psi(u) = u$$



- 2クラス確率論的分類 → シグモイド関数（ロジスティック関数）

$$\psi(u) = \text{Sigmoid}(u)$$



- 多クラス確率論的分類 → ソフトマックス関数

$$\psi(u) = \text{Softmax}(u)$$

ニューラルネット＞基本の構成要素＞活性化関数の役割



活性化関数は省いてはいけない？

- 活性化関数がないければ、いくつ層を重ねても、結局1つの層に縮約できてしまう

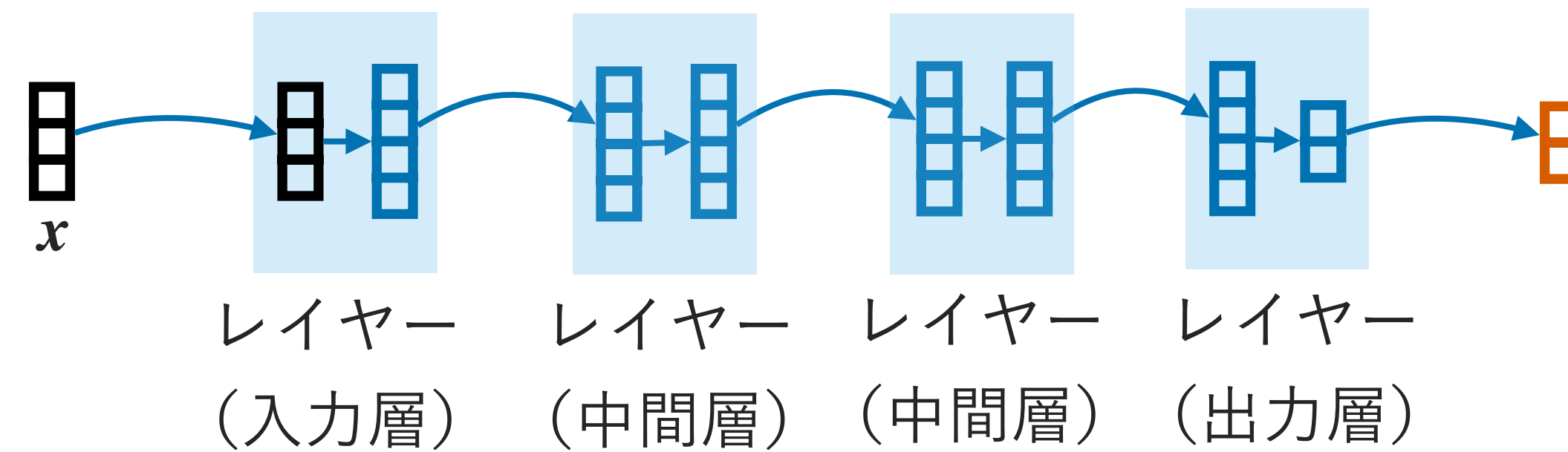
$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

- つまり、活性化関数が無ければ、1層分の複雑さの関数しか表すことができない
- 線型（+定数）よりも複雑な関数を表せるようにするのが活性化関数の役割
 - ゆえに、活性化関数のことを「非線型性」（nonlinearity）と呼ぶこともある

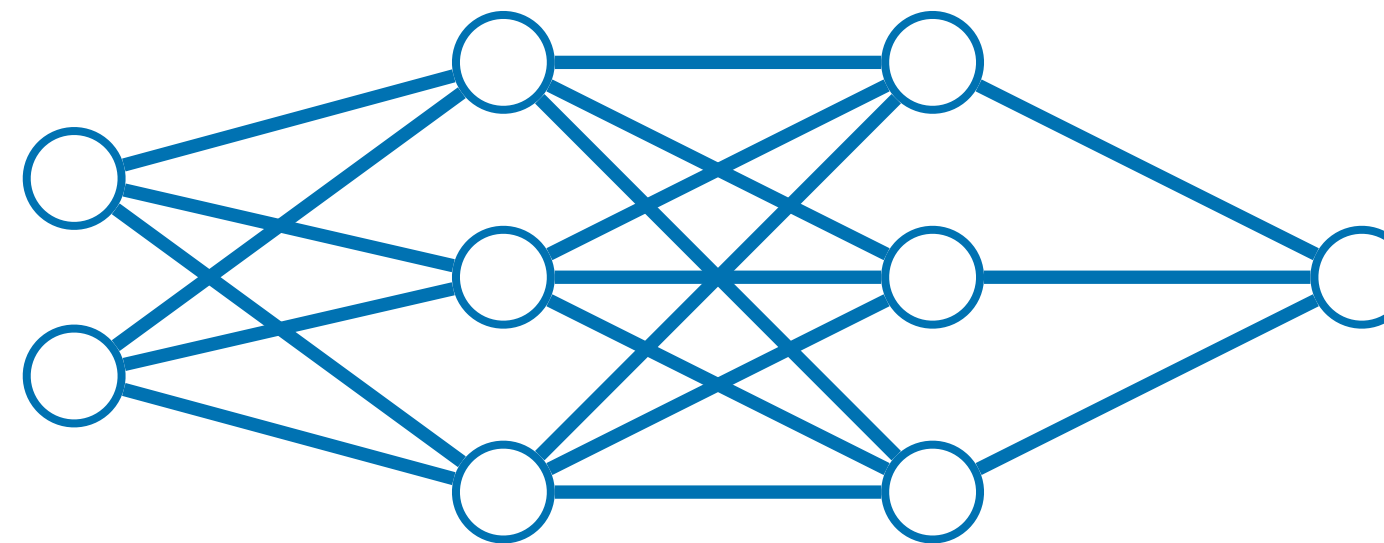
ニューラルネット＞基本の構成要素＞ユニット

レイヤーを細かく見ると，ユニットを多数繋げたものという見方もできる

- 順伝播型NNは，ベクトルを変換するレイヤーを積み重ねたモデル



- レイヤーは，複数の**ユニット (unit, 素子)** からなる



ニューラルネット > 基本の構成要素 > ユニット

ユニットは、レイヤーをさらに詳しく見たもの

- 重み行列 W , バイアス b , 活性化関数 ψ からなるレイヤーは,

$$W = \begin{bmatrix} \mathbf{w}_1^\top \\ \vdots \\ \mathbf{w}_r^\top \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ \vdots \\ b_r \end{bmatrix} \quad \text{とすると,} \quad \psi(W\mathbf{x} + b) = \begin{bmatrix} \psi(\mathbf{w}_1^\top \mathbf{x} + b_1) \\ \vdots \\ \psi(\mathbf{w}_r^\top \mathbf{x} + b_r) \end{bmatrix}$$

- これは、次の形の**ユニット (unit, 素子)** を r 個並べたものともみなせる
 - w を**重み(weight)**, b を**バイアス(bias)**, $\psi: \mathbb{R} \rightarrow \mathbb{R}$ を**活性化関数**として

$$\mathbf{x} \mapsto \psi(\mathbf{w}^\top \mathbf{x} + b)$$

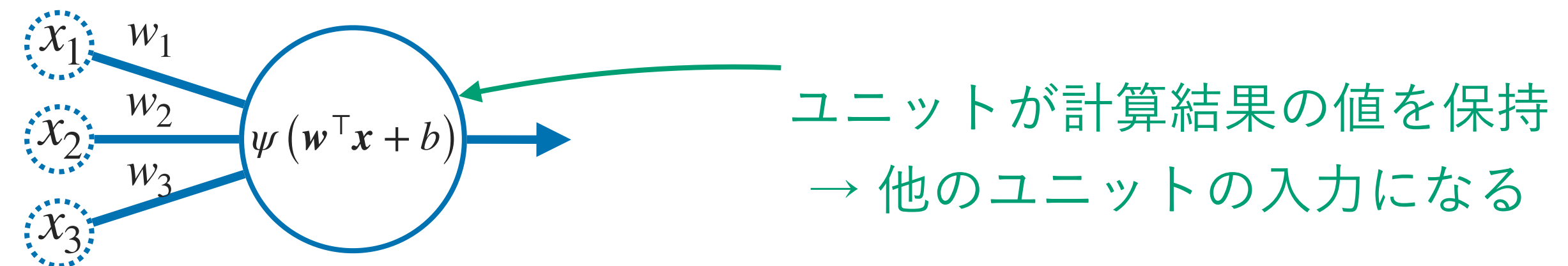
【用語】 「重み」は $\mathbf{w}^\top \mathbf{x} = w_1 x_1 + \dots + w_p x_p$ という重み付き和の重みであることから。「バイアス (偏り)」は、出力の値を $+b$ だけ「ずらす」ことから。

ニューラルネット > 表現図式と用語 > ユニット

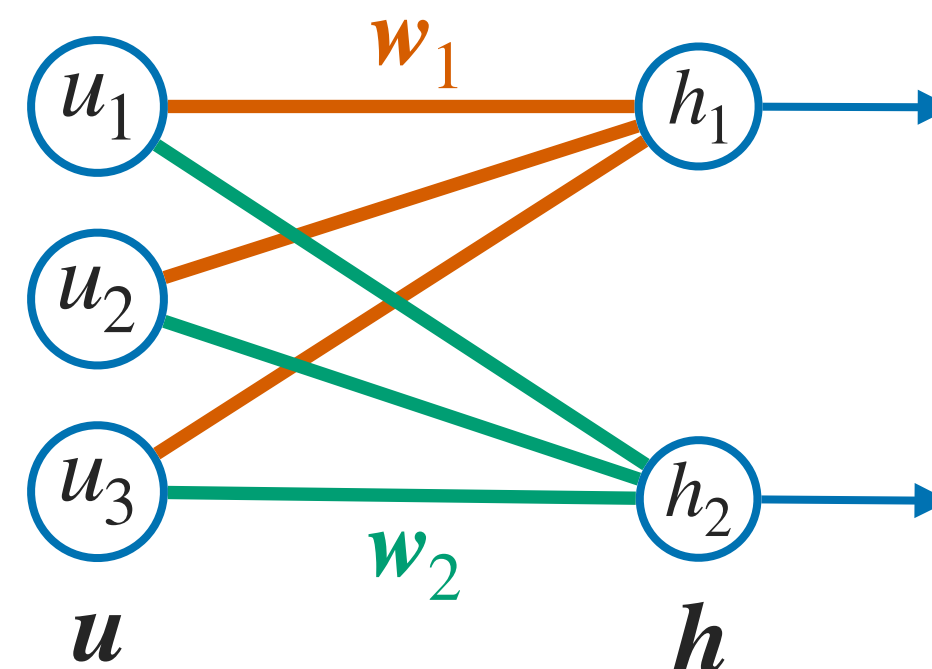
ユニットを表す図式

- ユニット $x \mapsto \psi(w^\top x + b)$ の図（重み w ，バイアス b ，活性化関数 ψ ）

- 入力を受け取るとユニットの「現在の値」 = **活性値 (activation)** が決まる



- ユニットの活性値は，次のユニットたちに渡される（重み付き和で）

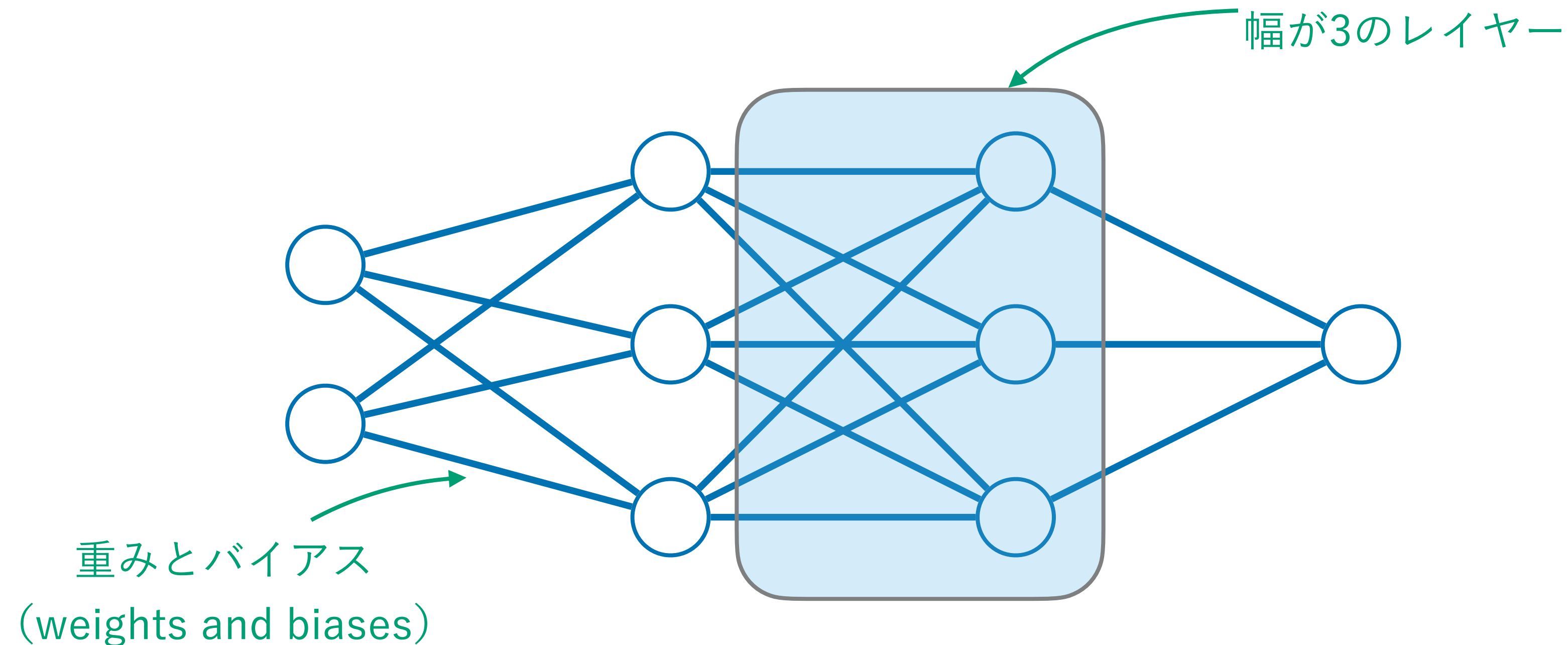


$$\begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} \psi(w_1^\top u + b_1) \\ \psi(w_2^\top u + b_2) \end{bmatrix}$$

ニューラルネット＞表現図式と用語＞用語

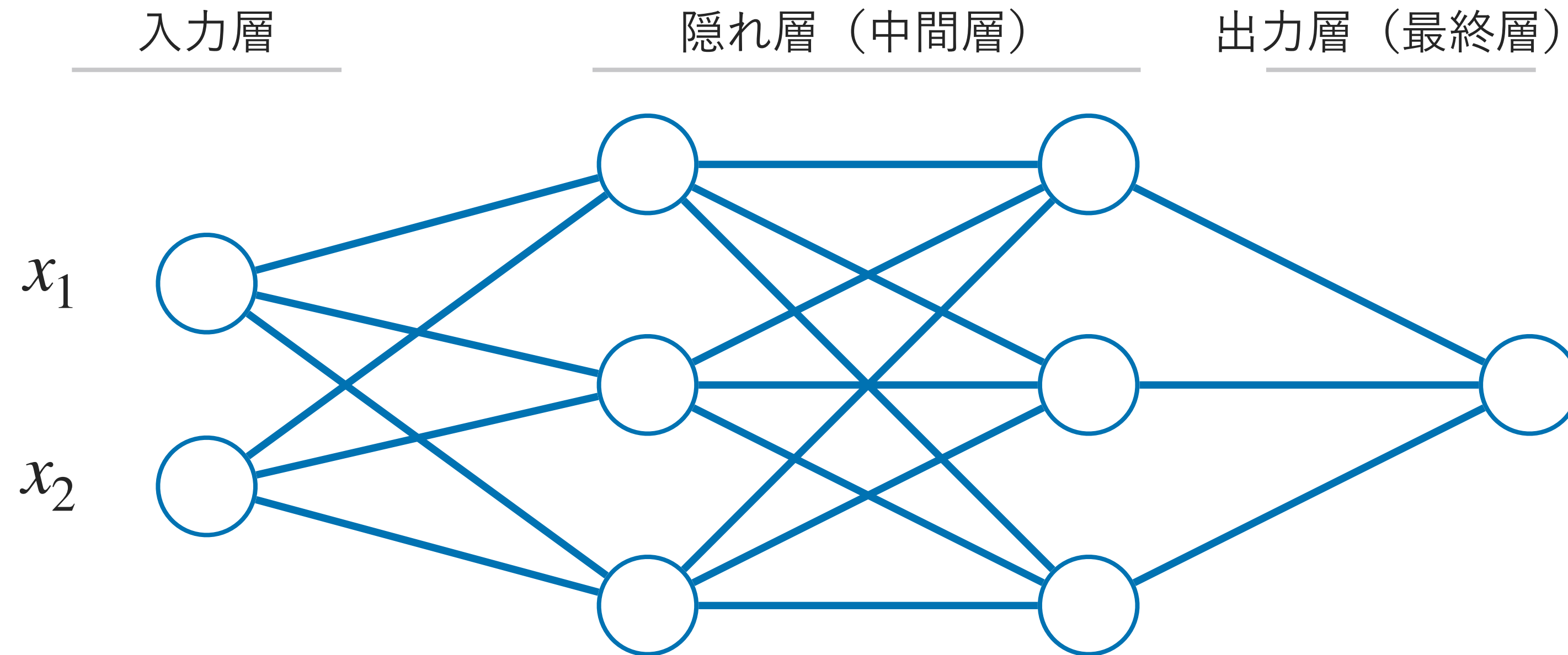
ユニットを複数並べたものがレイヤーとなる

- 並べた個数はレイヤーの幅 (width) という



ニューラルネット＞表現図式と用語＞全体

よく使われる図式



隠れ層が2層ある → 隠れ2層のFNN
(two-hidden-layer FNN) という

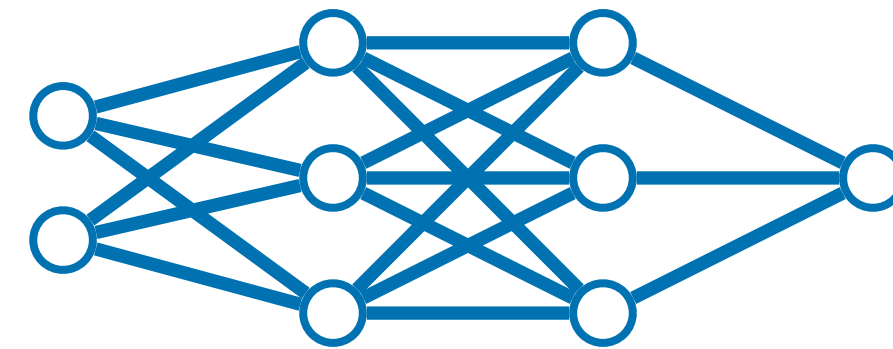
入力層を除いて3層ある → 3層FNN
(three-layer FNN) ともいう

ニューラルネット > アーキテクチャ



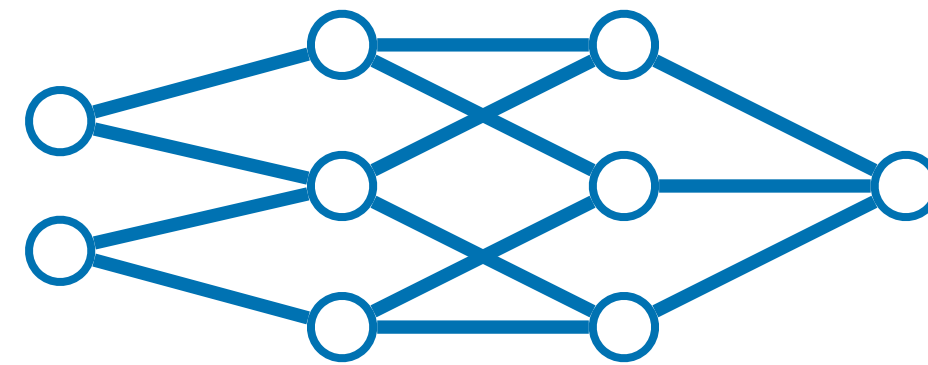
ユニット単位で見れば，結合の仕方はもっと自由に設計してもよい

- 今回扱ったレイヤーは，前の層の**全ての**ユニットから，今の層の**全ての**ユニットに線が繋がっているので，**全結合層 (fully-connected layer)** と呼ばれる



全結合

- 例えば，次のようなネットワークを考えてもよい



あえて一部だけ結合

- ネットワークの設計のパターンは**アーキテクチャ (architecture)** と呼ばれる

ニューラルネット＞モデルの構造



ニューラルネットワークの人気の理由の一つは、その柔軟さにある

- 応用分野ごとに、問題に合った形のモデルを生み出すことができる
 - 画像に適した構造, テキストに適した構造, ……が研究・開発されている
- 応用分野ごとの適したアーキテクチャ選定は、学習における事前知識の典型例

ニューラルネットワークの最適化① 確率的勾配法

ニューラルネット＞最適化



NNの最適化に関する話を解説

- ニューラルネットワークは，構造の自由度などから有力なモデルの一つ
- しかし，パラメーターが複雑に絡み合うので，最適化は独自の工夫が凝らされる
 - 確率的勾配法
 - 誤差逆伝播
 - という基本の考え方2つを中心に解説

ニューラルネット＞最適化＞取扱いの難点①

データも大規模になりがち

- 複雑なデータに対応するために、NNのパラメーター数は大規模になる
 - パラメーターが多いので、訓練データも大規模にしないと性能が良くならない
- インターネット時代の到来により、大規模な訓練データが収集されるように
 - 2009年のImageNet：画像データが320万枚（その後1400万枚以上に拡張）
 - 2023年のRefinedWeb：テキストデータが数千億単語（数兆単語のデータのうち一部だけが公開）

【参考】 Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. 2009 IEEE Conference on Computer Vision and Pattern Recognition, 248–255. <https://doi.org/10.1109/CVPR.2009.5206848>

【参考】 Penedo, G., Malartic, Q., Hesslow, D., Cojocaru, R., Cappelli, A., Alobeidli, H., Pannier, B., Almazrouei, E., & Launay, J. (2023). The RefinedWeb dataset for Falcon LLM: Outperforming curated corpora with web data, and web data only (arXiv:2306.01116). arXiv. <https://doi.org/10.48550/arXiv.2306.01116>

ニューラルネット > 最適化



基本的には勾配法ベースの手法で最適化を行う

- 基本的に大域的最適解を求める解の公式はないので、勾配法で最適化する

- 復習) 勾配降下法のパラメーター更新式

$$\hat{\theta} \leftarrow \hat{\theta} - \eta \cdot \nabla L(\hat{\theta})$$

- ただし、ニューラルネットワークの性質に合わせて、経験則や理論解析に基づく様々な工夫が取り込まれている

- 単なる勾配降下法ではなく、かなり複雑な最適化手法が発達
- 最適化手法に取り入れる工夫も、帰納バイアスの一端を担う

ニューラルネット > 最適化 > 大規模化への対応

NNの学習に使うデータは大規模 → 訓練時の計算量を制御する工夫が必要

- 経験リスクに基づく目的関数

$$L(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell((\mathbf{x}_i, y_i), g_{\boldsymbol{\theta}})$$

- この勾配は

$$\nabla L(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \nabla \ell((\mathbf{x}_i, y_i), g_{\boldsymbol{\theta}})$$

- 例えば10億件のデータがあると、1回の更新だけでも10億件の勾配計算
 - あまりに多い場合、そもそもコンピューター1台には入りきらず実行不可能

ニューラルネット > 最適化 > 大規模化への対応

リスクのサンプル近似のために、必ずしも全てのデータを用いる必要はない

- ミニバッチ確率的最適化 (mini-batch stochastic optimization method)
 - 毎ステップ、データの一部を **ミニバッチ (mini-batch)** として取得して学習
 - **バッチサイズ (batch size)** = ミニバッチ内のデータ数 $|B|$ は1件～数百件
 - ミニバッチ B 上での訓練損失を小さくするように学習

$$\tilde{L}(\theta) = \frac{1}{|B|} \sum_{(x_i, y_i) \in B} \ell((x_i, y_i), g_{\theta})$$

- 通常のミニバッチの作り方
 - 訓練データをシャッフルし、バッチサイズごとに分割しておく

【用語】 ミニバッチではなく全部の訓練用データを一度に使う、と言いたい場合には「フルバッチ学習」や「バッチ学習」ということがある。

ニューラルネット > 最適化 > ミニバッチ確率的勾配法

ミニバッチ確率的勾配降下法 (mini-batch stochastic gradient descent)

- パラメーター θ を適当に初期化 ($\hat{\theta} \leftarrow \theta$)
- 以下を必要なだけ繰り返す (1周を**エポック**と呼ぶ)
 - 以下をすべてのミニバッチ B について実行する
 - ▶ ミニバッチ B で作った目的関数 $\tilde{L}$ の、現在の $\hat{\theta}$ における勾配 $\nabla \tilde{L}(\hat{\theta})$ を計算
 - ▶ 勾配の反対方向にパラメーターを少し更新 (学習率 $\eta > 0$)

$$\hat{\theta} \leftarrow \hat{\theta} - \eta \nabla \tilde{L}(\hat{\theta})$$

- 早期停止条件を満たしていたら中断
- 最終的な $\hat{\theta}$ を使用する

【用語】 エポック (epoch) は、同じ訓練データを何回学習に使ったかを数えたものとも言える。1エポック内では、各データ点はちょうど1回学習に使われる。

ニューラルネット > 最適化 > ミニバッチ確率的勾配法

通常の勾配法に加えて，次のような項目を決める必要がある

- パラメーター θ を適当に初期化 ($\hat{\theta} \leftarrow \theta$)

- 以下を必要なだけ繰り返す（1周を**エポック**と呼ぶ）
 - データをシャッフルし，ミニバッチ $B_1, B_2, \dots$ に分割


 - 以下をすべてのミニバッチ $B_1, B_2, \dots$ について実行する
 - ▶ ミニバッチ B で作った目的関数 $\tilde{L}$ の，現在の $\hat{\theta}$ における勾配 $\nabla \tilde{L}(\hat{\theta})$ を計算
 - ▶ 勾配の反対方向にパラメーターを少し更新 $\hat{\theta} \leftarrow \hat{\theta} - \eta \nabla \tilde{L}(\hat{\theta})$ （学習率 $\eta > 0$ ）
 - 早期停止条件を満たしていたら中断
- 最終的な $\hat{\theta}$ を使用する

【用語】エポック（epoch）は，同じ訓練データを何回学習に使ったかを数えたものとも言える。1エポック内では，各データ点はちょうど1回学習に使われる。

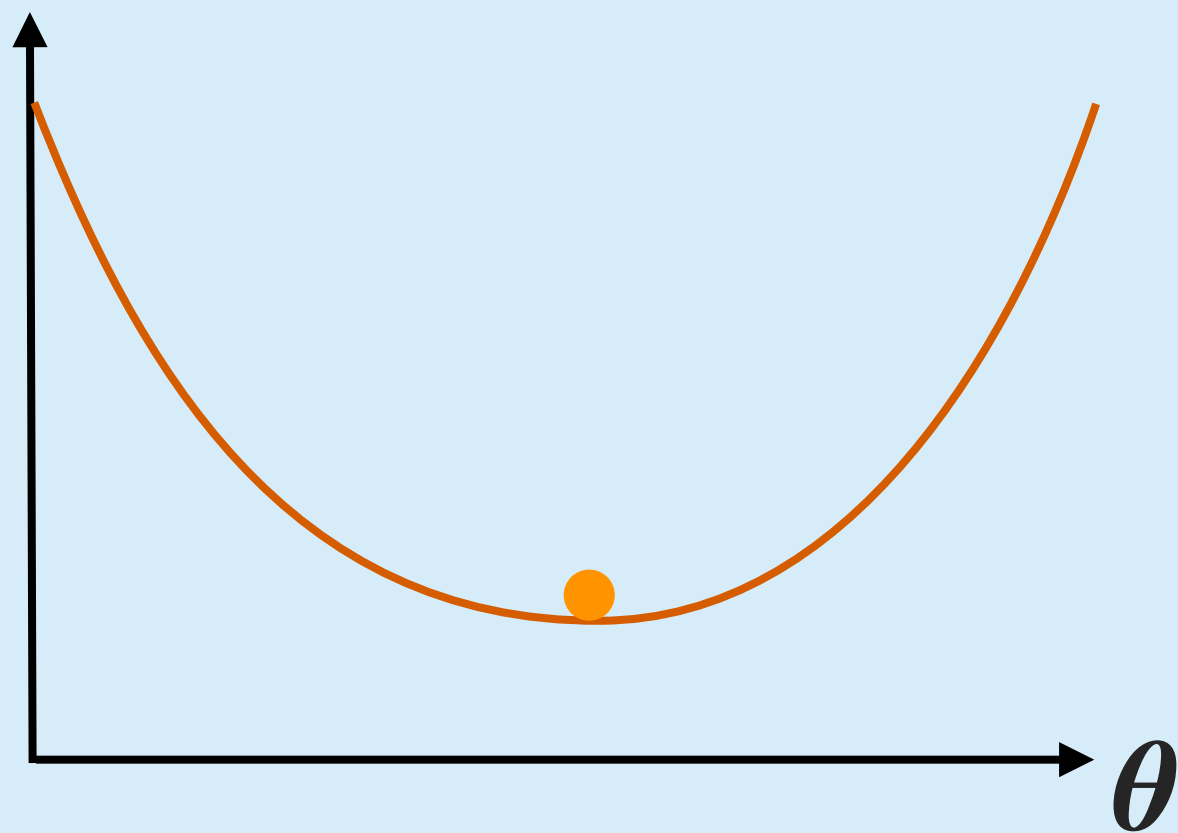
ニューラルネット＞最適化＞取扱いの難点②

目的関数の形状が（線型モデルなどと比較して）かなり複雑

線型モデルの最小二乗法など

凸最適化問題

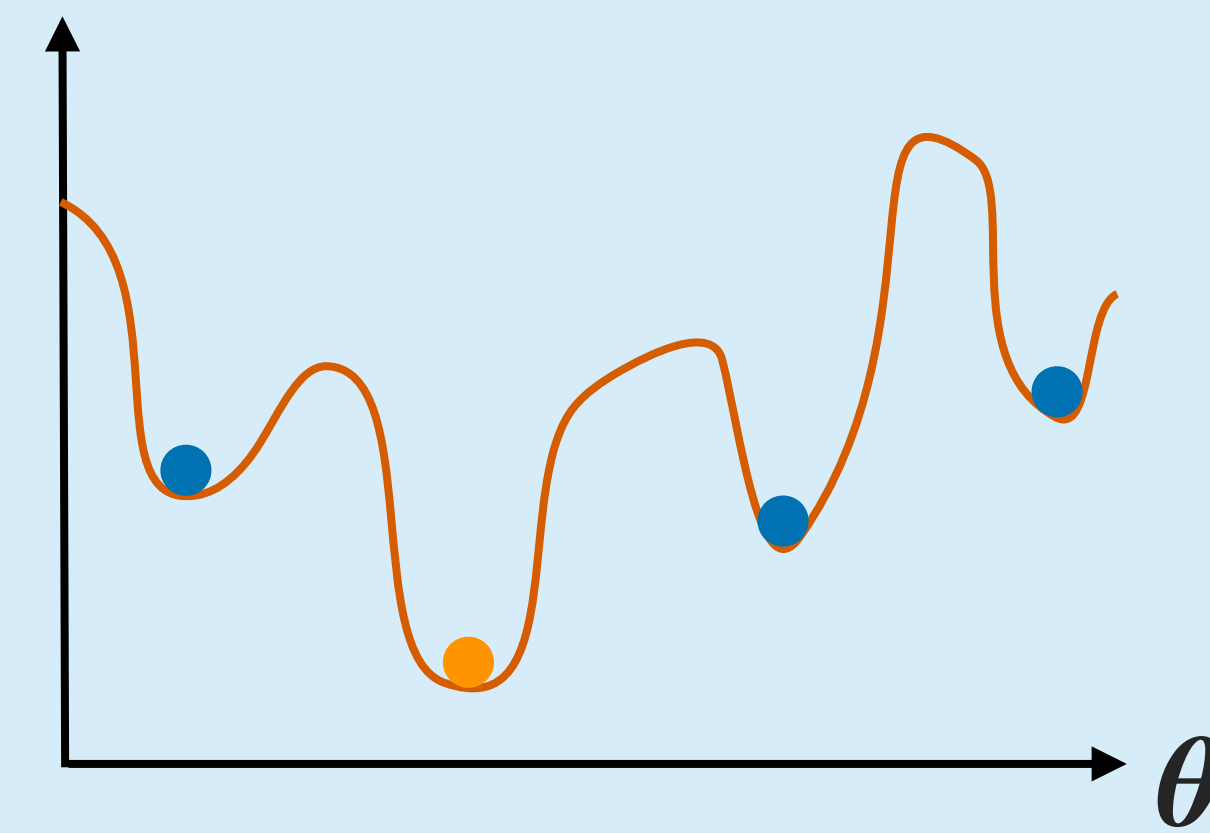
（大雑把に言って）解は一つで、
見つけやすい



ニューラルネットなど

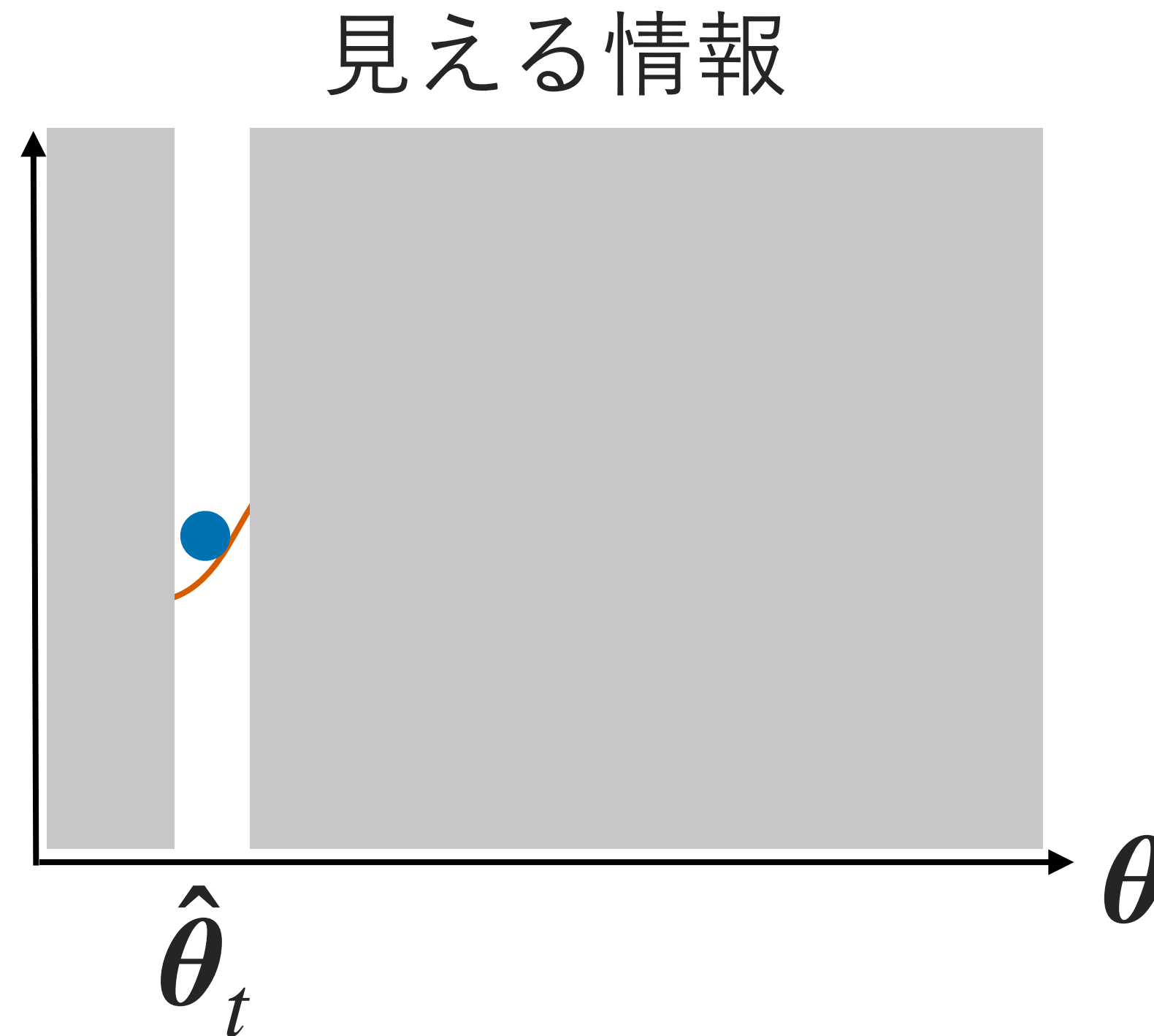
非凸最適化問題

局所的最適解が多数あり、大域的
最適解を求めるのが困難



局所的最適化手法

勾配法では基本的に「今いる場所の情報」だけでパラメーターを更新



現在地での勾配情報だけを頼りに
パラメーターを更新

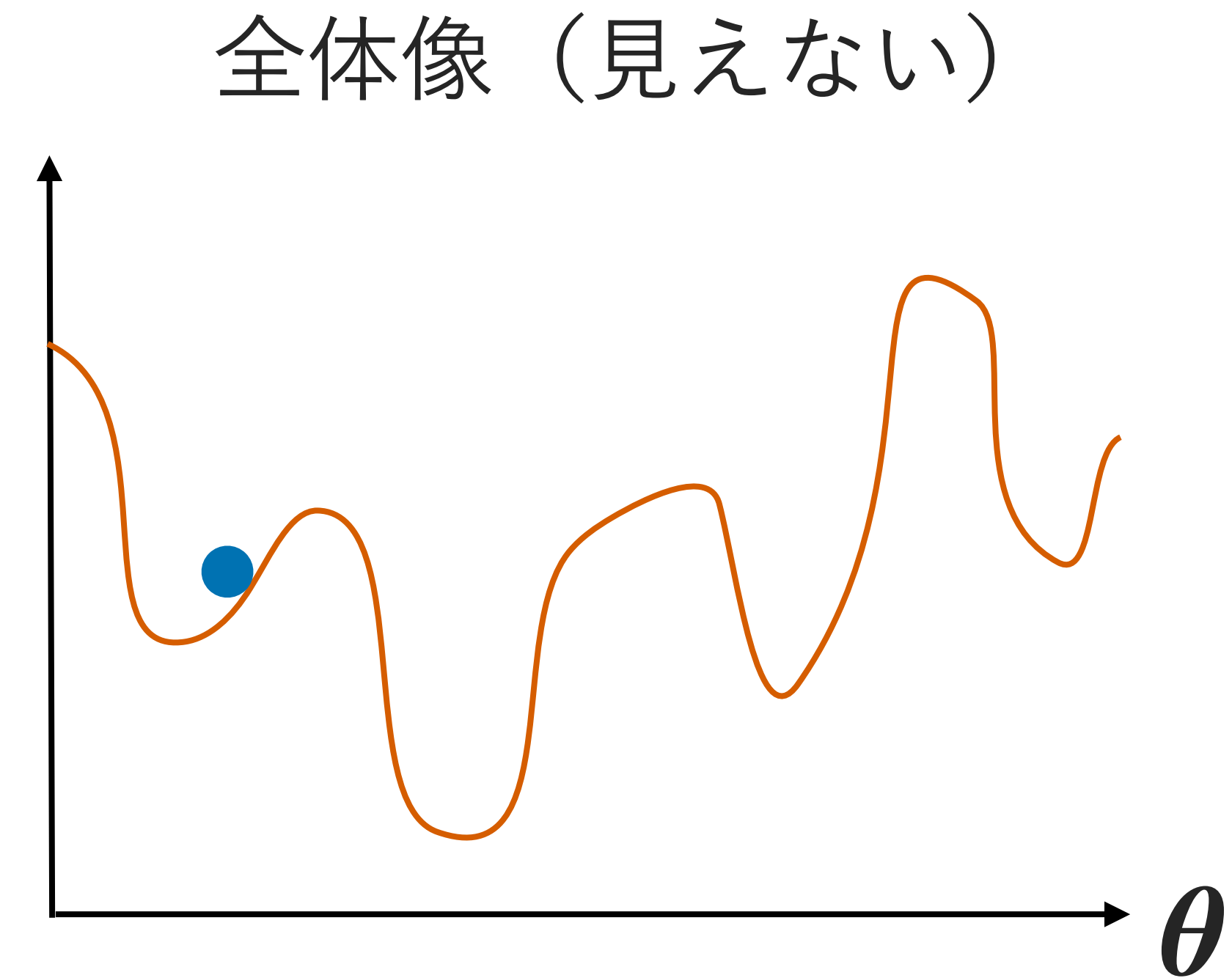
【補足】全体像は、あらゆるパラメーター値を試してその時の損失を記録していけば把握可能だが、高次元のパラメーターでそれは現実的には実行不可能。

【補足】1次元だと、このように一旦上がって下がる必要があるなど、一段と難しい。高次元では逆に経路があって最適化しやすいこともある。

【補足】モメンタム法と呼ばれる工夫を取り入れた勾配法などでは、現在地だけでなく、それまでの勾配情報の履歴も加味して更新する。

局所的最適化手法

勾配法では基本的に「今いる場所の情報」だけでパラメーターを更新



損失値が低い θ に移動したい

ニューラルネット＞最適化＞複雑な構造を想定した工夫

実際には、色々な工夫を組み込んだ勾配ベースの最適化法が使われる

- 学習率の調整や、慣性項（momentum）を更新則に組み込むなどの工夫
- AdamやMuonやCosine Annealing Scheduleなどの工夫がある（詳細は略）

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t .

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

return θ_t (Resulting parameters)

【参考】 Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. Proceedings of 3rd International Conference for Learning Representations. ICLR 2015, San Diego. 【用語】 Adamという名前は、適応的なモーメント推定（adaptive moment estimation）に由来。

【参考】 Defazio, A., Yang, X.A., Khaled, A., Mishchenko, K., Mehta, H., Cutkosky, A., 2024. The road less scheduled. Presented at the The Thirty-eighth Annual Conference on Neural Information Processing Systems.

ニューラルネット＞最適化＞まとめ



ニューラルネットワークの最適化は、複雑で様々な工夫が必要

■ 主な複雑さ

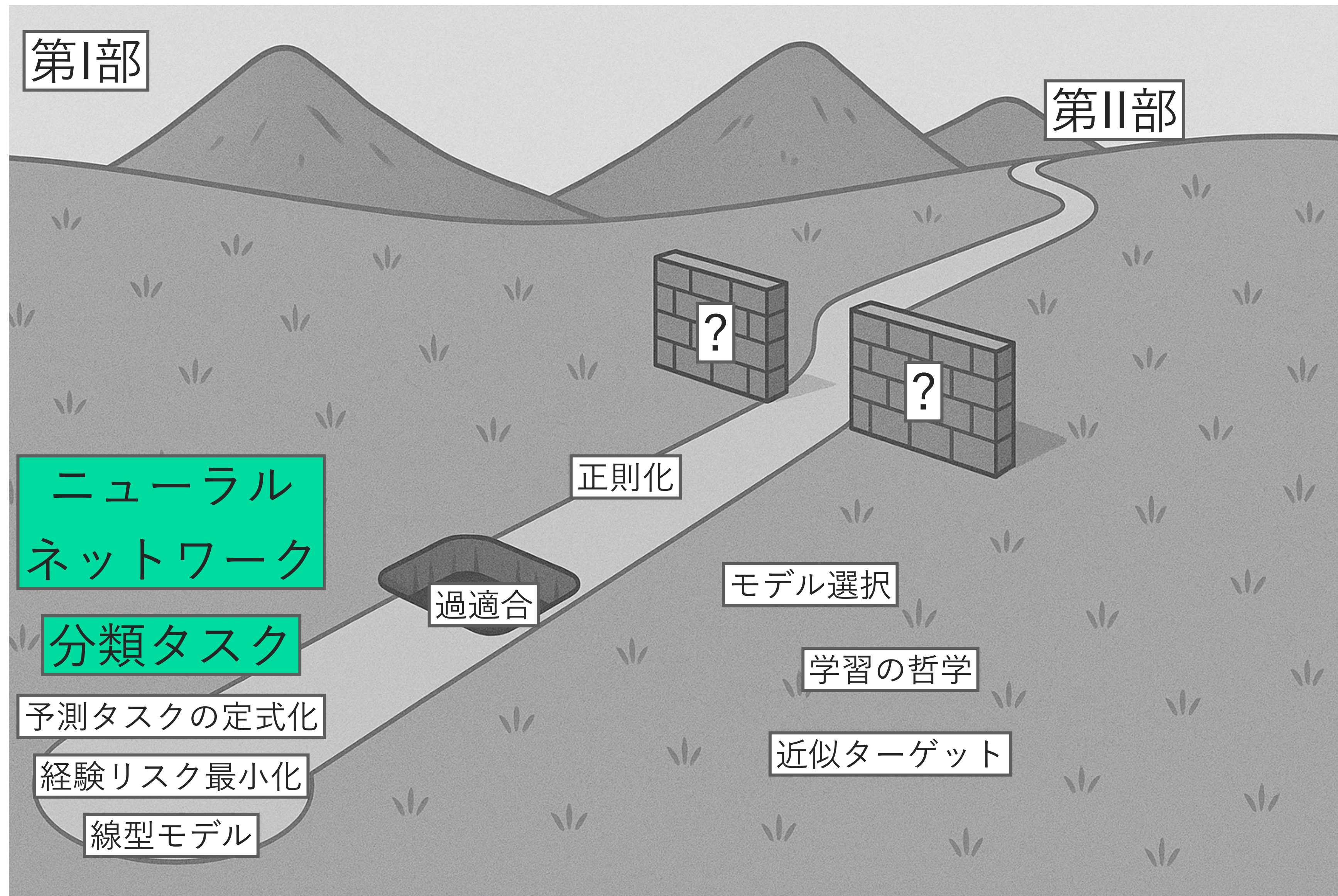
- データ量が大规模
- 目的関数の”地形”が複雑（局所的最適解が存在するなど）

■ 主な解決方針

- 確率的勾配法により一度に全データを扱わなくてよい
- 局所的最適解を避けるための様々な工夫を凝らした最適化手法が開発（実用上はその時点で評判が良いものを選ぶのがよい）

【補足】線型モデルと異なり局所的最適解が存在するなどの違いがあることは確かだが、近年では、パラメーターを大量に持つニューラルネットワークの目的関数は逆に良い性質を持つのではないかと、などの観点から研究が進んでいる（そうでないと、実用上、高い性能を発揮するモデルが実際に得られている＝最適化が実践的には上手くできていることの説明がつかない、という面がある）。

ロードマップ



第7回

手嶋毅志 (Takeshi Teshima)



質問と回答 (第7回 2025)

加算対象ではないその他の質問・コメント

授業が進むにつれて使われている式が複雑になってきたように感じて、少し難しいと思ったので復習したいと思います。

そうですね。一度分かってしまえば簡単に見えてくるのではないかと思いますので、ぜひ資料を見ながら復習して頂ければと思います。なお、数式の複雑さは現在より大きくは上がりません。

今ゼミで回帰モデルについて学んでいる最中[…], 今日の内容である、確率論的多値分類などはゼミでは学習していないことなのですがこの授業で学んでとても興味深いと感じました。

そういった感想、嬉しいです！ゼミで回帰を学ばれているときは、目的とするタスクが予測ではなく何らかの分析という場合もあります。ぜひタスクを問いながら学んでみてください。

加算対象ではないその他の質問・コメント

ソフトマックス関数が出力した数値を確率として扱うことは可能ですか。

確率値であるための要件は単に「0以上1以下であること」，確率値ベクトルなら「各成分が0以上で足して1であること」です。故に，1成分だけ取り出した $\text{Softmax}(\mathbf{z})[k]$ は，確率値です。

よく使われる活性化関数において、それぞれどういった特徴がありどういったものに使うのに適していますか。

前回の資料の通りで，最終層では欲しい出力値の要件（実数？確率値？確率値ベクトル？）に応じて選ぶことが多いです。現在は，中間層ではReLUやその亜種が使われることが多いです。

加対象の質問

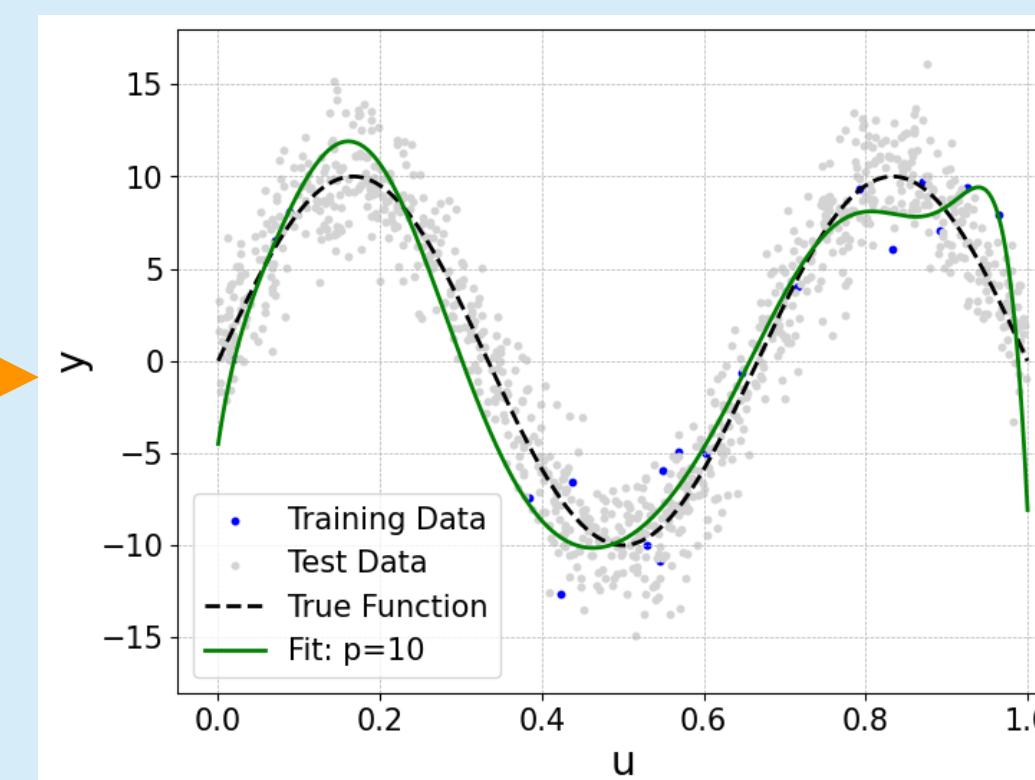
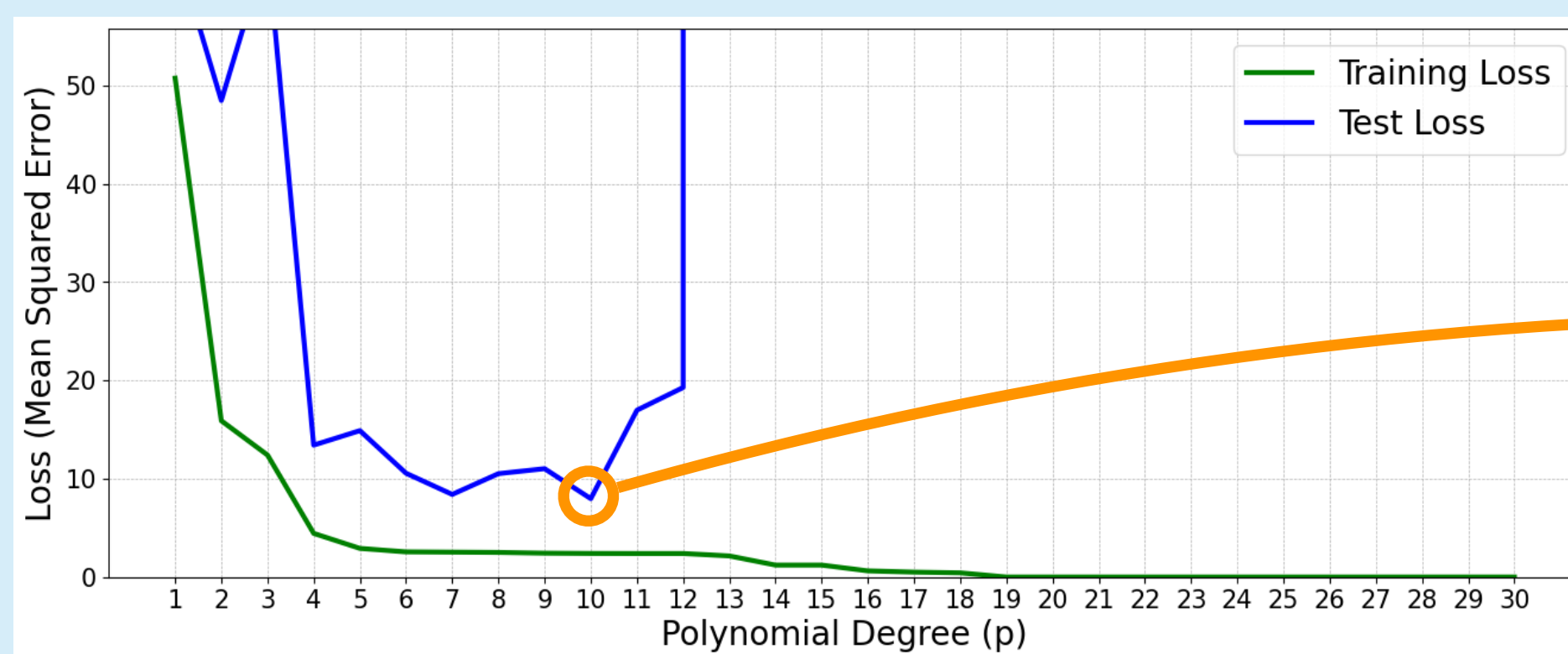
分野ごとに適したニューラルネットワークの研究は、どこまで進んでいるのですか。

第II部でいくらか出てきます。一本道ではないので現在も様々な探索の試みがありますが、おおよそ1970年代から現在までの研究の蓄積でかなり進んでいます。画像ではCNN系やVisionTransformer系がさまざま。テキストではRNN系やTransformer系が発展。音声でもCNN系、RNN系、Transformer系が探索されています。グラフニューラルネットというものや、可逆ニューラルネットという特殊な性質を持ったものもあります。拡散モデルと呼ばれるアーキテクチャや、エンコーダー・デコーダー型というアーキテクチャ、推薦系ならTwo-tower型というアーキテクチャもよく使われます。ちなみに順伝播型NNはこれらとよく併用されます。

加点对象の質問

（第2回のリアクションペーパーでの質問）過適合現象を説明したスライド64ページ目のグラフを見ると、10次のときがTest Lossが一番小さく良いように見えるのですが、授業で紹介された4次方が良いのでしょうか。10次だと複雑すぎるのですか？

左図についてのご質問ですね（よく見ていますね！）。10次での様子は右図です。Test Lossの数値のとおり，実際に10次で問題ないです。



加点对象の質問

数ヶ月ほど前にGeminiが「死んでください」と言ったり、最近では画像生成AIを使ってわいせつな画像を作成し販売していた人が逮捕されたりとAI関連のニュースがありました。なぜこのようなことが起こるのかを説明してほしいです。人間が無理にAIに出力させようとしているようですが、出力する前に不適切な言葉や画像を出力しないように制御するシステムはないのですか？

いずれも生成系タスクを解くAIの、社会的影響に関しての話題ですね。

「リアルな生成ができること ≠ 社会的規範や社会的通念を認識していること」なので起こります。なお、どちらの事例も、問題か否かは社会的規範に依存します。どうやって社会的規範を遵守させるかは、完全には解決していない課題ですが、改善は進められています。詳しくは第13回で扱います。

加対象の質問

勾配降下法における早期停止の利点は何ですか。

①時間の節約

事前に決めたステップ数だけ律儀に更新しなくても、モデルの改善がみられなくなってきたらアルゴリズムを止めてしまってもいいですね。時間の節約になります。

②正則化

ある程度パラメーターの多いモデルなら、ステップを踏むほど学習が進み、いずれ過適合し始めます（経験則）。勾配降下法の更新回数は（ざっくり）モデルの複雑さに対応します。ここで、早期停止条件として検証損失などの「モデル選択規準」に基づくルールを用いれば、結果として正則化の効果が得られます。

出席確認

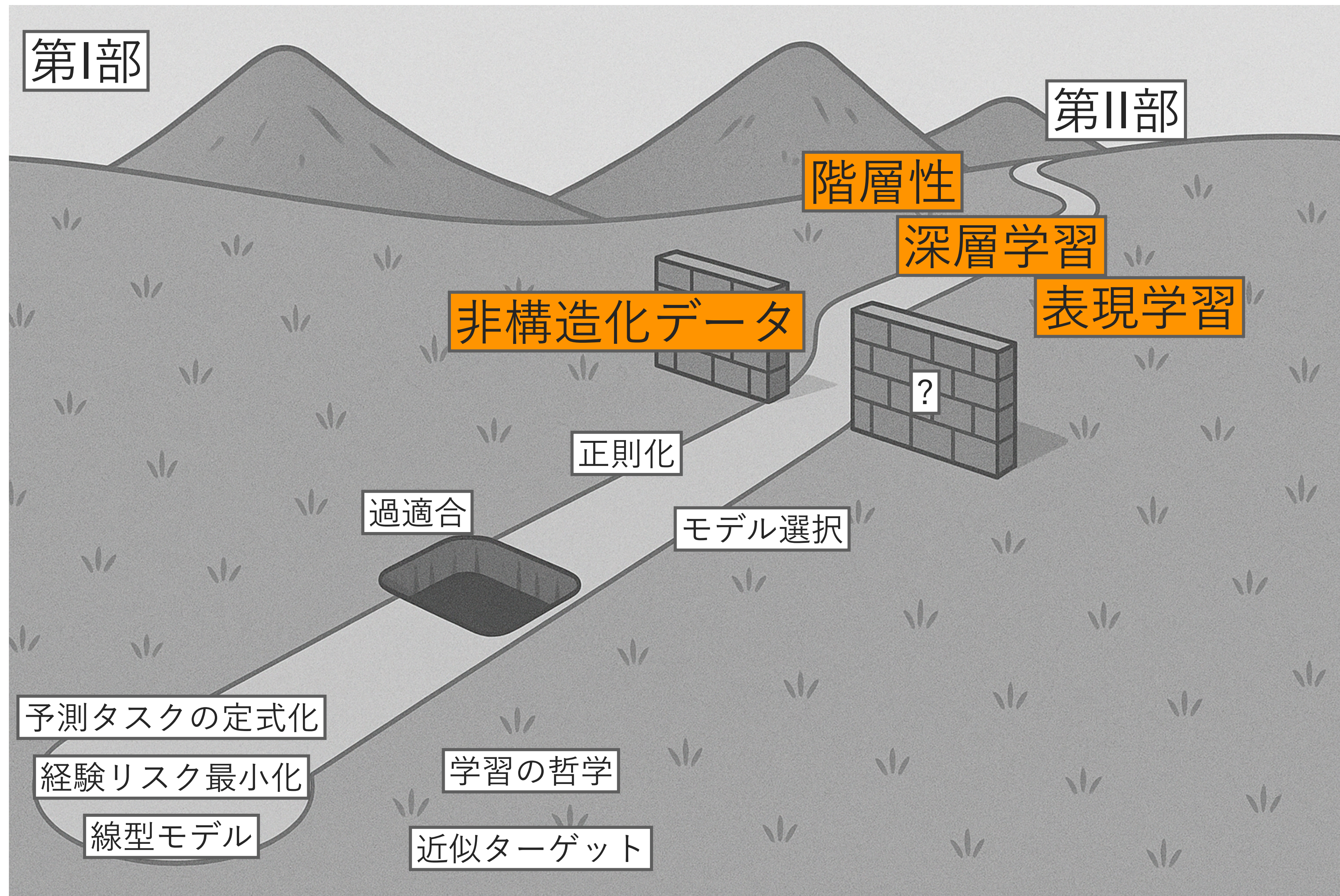
本日の出席確認番号は……

2025

告知

- 小テストは月曜日に公開します
- モバイル出席のリアクションペーパーシステムは運用していません（今回から設定をオフにしました）。
 - これまでに**出席のほうのシステムに送っていただいたリアクションペーパー**は、目を通しています。これまでに送っていただいた分の加点・無加点の判定は、LETUSのほうと同じように扱います。（これまでの回での回答で結果的にほぼカバーできていました）
 - 引き続き、質問はLETUSの「質問投稿フォーム」のほうへお願いします。

ロードマップ



ニューラルネットワークの最適化② 誤差逆伝播

ニューラルネット > 最適化



基本的には勾配法ベースの手法で最適化を行う

- 基本的に大域的最適解を求める解の公式はないので、勾配ベースの方法で最適化

$$\hat{\theta} \leftarrow \hat{\theta} - \eta \cdot \nabla L(\hat{\theta})$$

- これには、目的関数のパラメーターに関する勾配を計算する必要がある
 - 例) ソフトマックス + 負の対数尤度の場合

$$\nabla \ell((\mathbf{x}, y), g_{\theta}) = -\nabla s_{\theta}(\mathbf{x})[y] + \sum_k g_{\theta}(\mathbf{x})[k] \cdot \nabla s_{\theta}(\mathbf{x})[k]$$

- スコアリングモデルの出力成分 $s_{\theta}(\mathbf{x})[k]$ の勾配 $\nabla s_{\theta}(\mathbf{x})[k]$ を計算する必要がある

ニューラルネット > 最適化 > 勾配計算



パラメーターが複雑に絡み合っているので、微分を愚直に計算したくはない

- 例えば3層のモデル ($\psi_1 = \psi_2 = \text{ReLU}$, $\psi_3 = \text{Softmax}$ とする) なら,

$$g_{\theta}(\mathbf{x}) = \psi_3(W_3\psi_2(W_2\psi_1(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3)$$

- 目的関数

$$L(\theta) := -\frac{1}{n} \sum_{i=1}^n \log g_{\theta}(\mathbf{x})[y_i]$$

- このとき、現在のパラメーターの具体的な値 $\theta = (W_1, \mathbf{b}_1, W_2, \mathbf{b}_2, W_3, \mathbf{b}_3)$ における,
 $L(\theta)$ の $W_1, \mathbf{b}_1, W_2, \mathbf{b}_2, W_3, \mathbf{b}_3$ のそれぞれに関する微分 (勾配) を計算せよ.

ニューラルネット > 最適化 > 勾配計算の例

手計算できる例で説明する

- 1次元特徴量での回帰を考える.
 - モデルは次の順伝播型NNとする

$$g_{(w,b)}(x) = \sigma(wx + b)$$

- データは1点 (x, y) だけとして, 目的関数は二乗誤差とする

$$L(w, b) = (y - g_{(w,b)}(x))^2$$

- 勾配 $\nabla L(w, b)$ を計算せよ.

ニューラルネット > 最適化 > 勾配計算の例 > 手計算



試しに $L(w, b) = (y - \sigma(wx + b))^2$ の w による微分の計算手順を考えてみる

- まず外側に2乗があるので, $(x^2)' = 2x$ を使って,

$$(y - \sigma(wx + b))^2 \rightsquigarrow 2(y - \sigma(wx + b)) \cdot (\text{中身の}(y - \sigma(wx + b))\text{の微分})$$

- 続いて中身の $(y - \sigma(wx + b))$ を w で微分するから,

$$(y - \sigma(wx + b)) \rightsquigarrow -\sigma'(wx + b) \cdot x \text{ のようになって,}$$

- 残る $\sigma'(wx + b)$ の部分は, 確か $\sigma'(u) = \sigma(u)(1 - \sigma(u))$ だったから, まとめると

$$\frac{\partial L(w, b)}{\partial w} = -2(y - \sigma(wx + b))\sigma(wx + b)(1 - \sigma(wx + b))x$$

- b による微分も同じように, …… (少し大変)

ニューラルネット > 最適化



パラメーターが多段階で変換に絡むため、勾配の式は複雑になる

- しかし、実はシステムチックに勾配を計算する方法がある
 - それは、入力 x から損失値が出てくるまでの計算順をまず考えて、その後に、それを遡りながら微分を計算するという方法

ニューラルネット＞最適化＞機械的に勾配を計算する方法

まずは、入力から損失の値が出てくるまでの計算手順を図にする

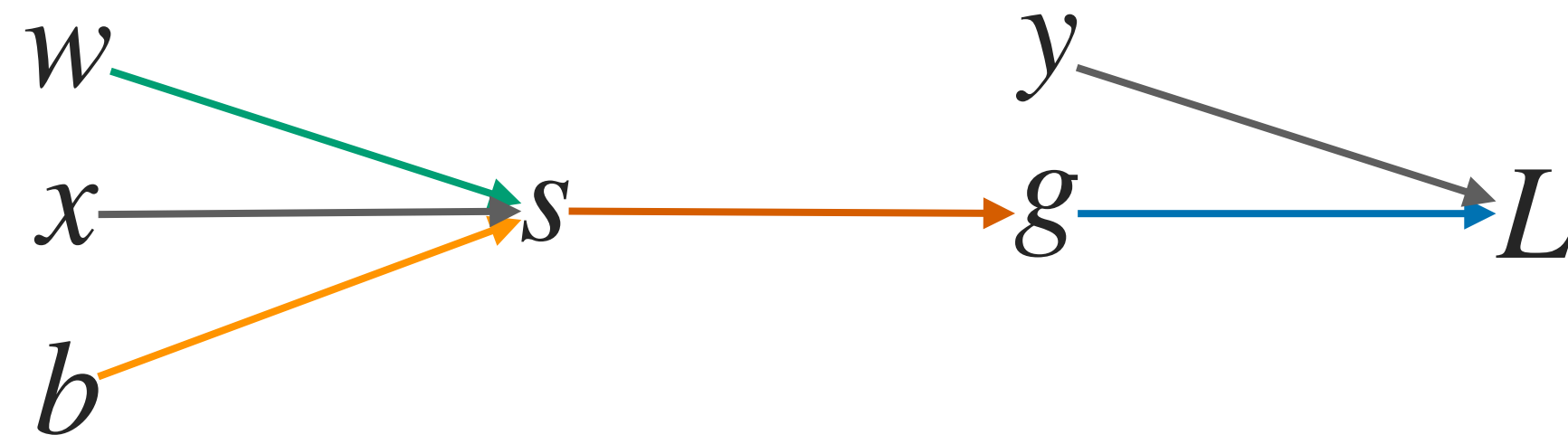
- モデルと目的関数の計算式は次のとおりだが……

$$L(w, b) = (y - g_{(w,b)}(x))^2 \quad (\text{ここで } g_{(w,b)}(x) = \sigma(wx + b))$$

- 途中の計算結果の値にも次のように記号を割り振っておいて……

$s = wx + b$ を計算し, $g = \sigma(s)$ を計算し, $L = (y - g)^2$ を計算

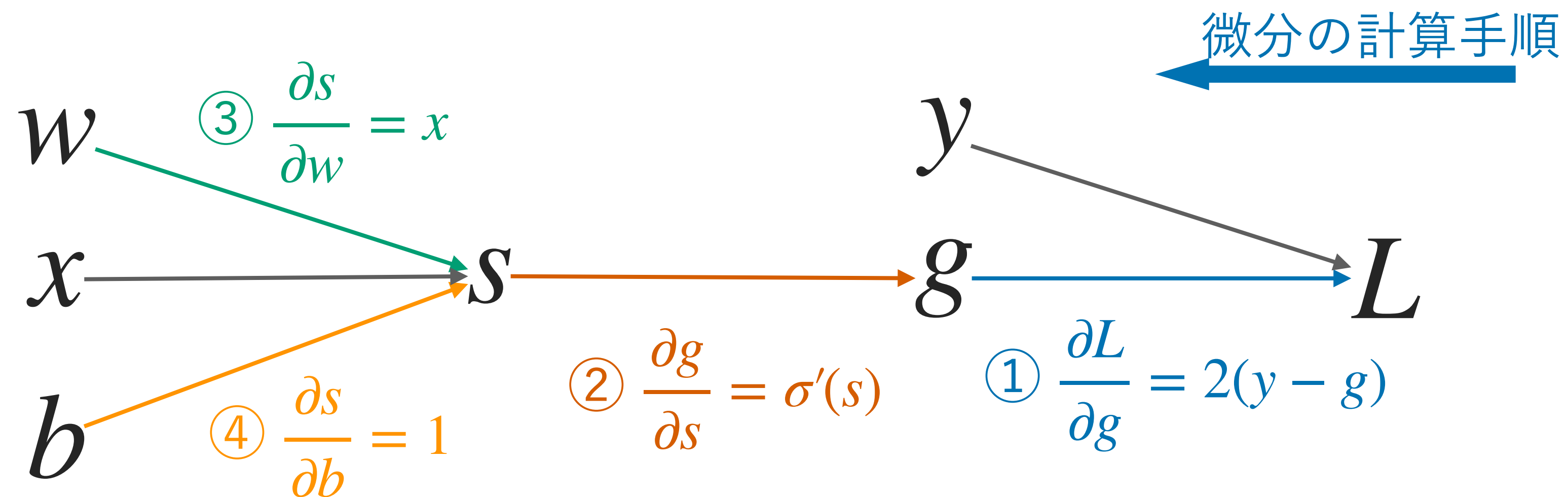
- 計算手順の通りに、入力やパラメーターから損失値 L が決まるまでを図にする



ニューラルネット > 最適化 > 機械的に勾配を計算する方法

この計算順を描いた図の「矢印」を逆順にたどりながら微分を計算する

- 例えば $g \rightarrow L$ の矢印では, L の g での偏微分 $\frac{\partial L}{\partial g} = 2(y - g)$ を計算 (他も同様)



- 最後に掛け算で $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial g} \frac{\partial g}{\partial s} \frac{\partial s}{\partial w}$, $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial g} \frac{\partial g}{\partial s} \frac{\partial s}{\partial b}$ を得る ($\frac{\partial L}{\partial g} \frac{\partial g}{\partial s}$ は再利用)

ニューラルネット > 最適化 > 計算グラフと自動微分



もっと複雑なニューラルネットでも全く同じことができる

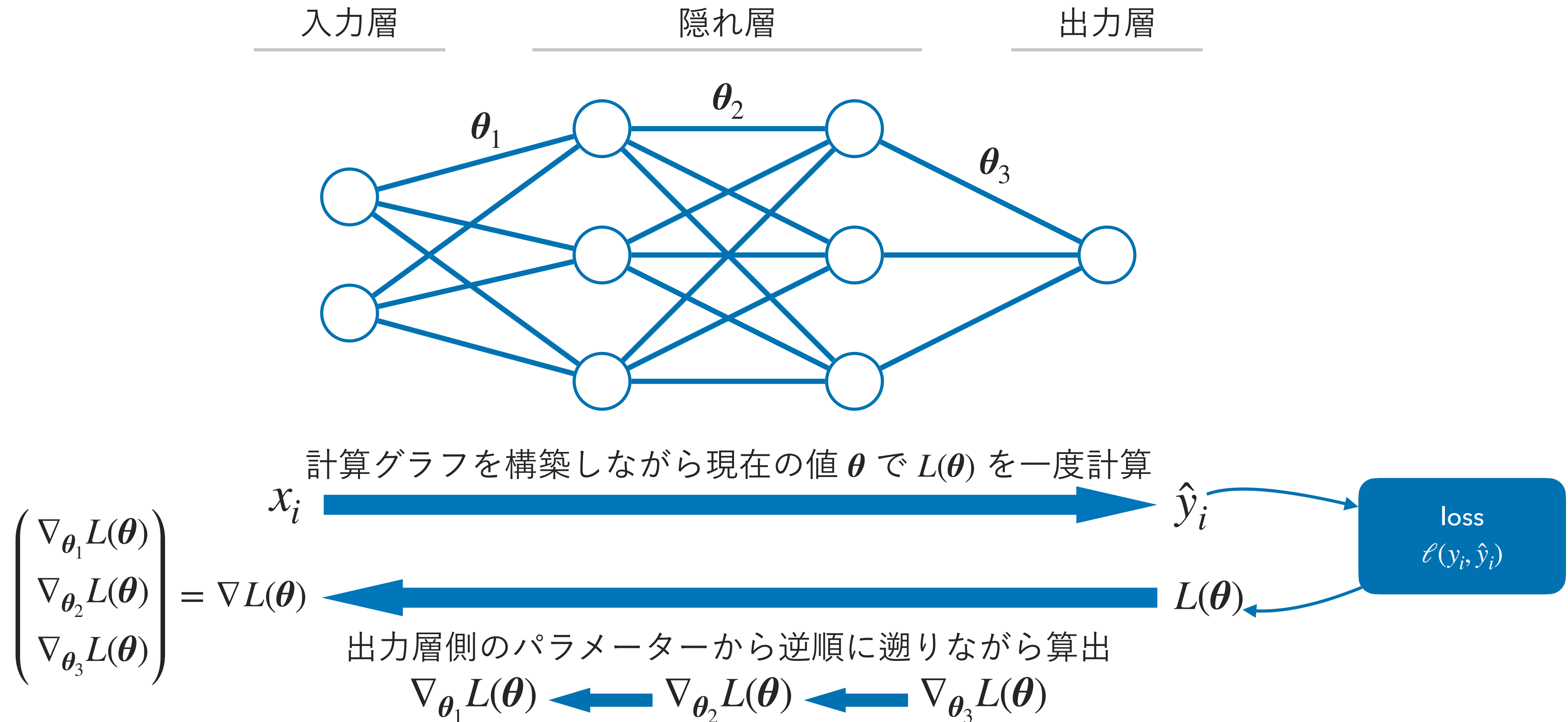
- このようにパラメーターやデータの変数を，計算の依存関係に従って矢印で繋いだ図を，**計算グラフ**（**computation graph**）という。
- 一般の計算グラフでも，全く同じように，システムチックに勾配計算をできる
 - これを**自動微分**（**automatic differentiation**）という
 - 全結合層である必要もなく，どんなアーキテクチャでも勾配を計算できる
 - 今では「微分可能な計算グラフ」の全般がニューラルネットと呼ばれる

【補足】どんな構造でも勾配をシステムチックに導出できるとはいえ，その勾配の計算効率が良いかどうかは別の話。

【補足】計算グラフとは，変数を頂点とする非巡回有向グラフで，各頂点に親の変数を引数とする関数が割り当てられているもの。なお，非巡回有向グラフとは，頂点と有向辺の集まりで，周回できる道がないもののこと。

ニューラルネット > 最適化 > 誤差逆伝播法 (Backpropagation)

自動微分によって $\nabla L(\theta)$ を出力層側から順に遡りながら算出



【用語】 損失関数 ≡ 誤差の情報を、入力側の層にめがけて伝えているように見えるので、誤差逆伝播法という。

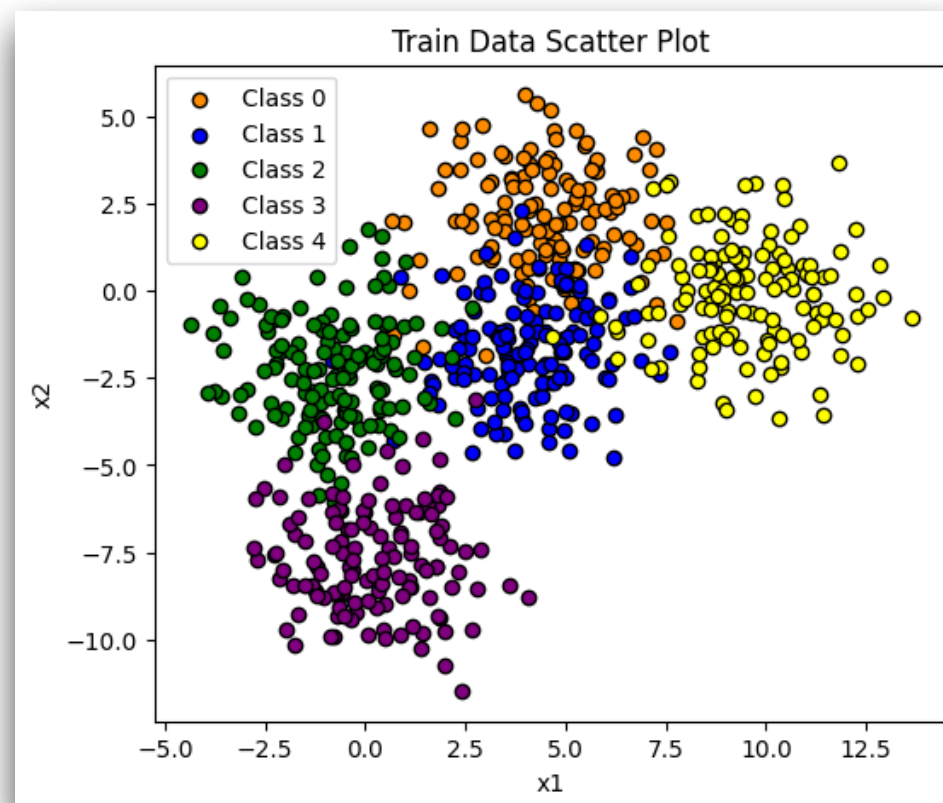
ニューラルネット＞実験＞設定

多クラス分類をニューラルネットで学習

タスク・学習戦略

5クラス分類・教師付き学習

特徴量	ラベル
x_1	y_1
x_2	y_2
$\vdots$	$\vdots$
$\{(X_i, Y_i)\}_{i=1}^n \stackrel{\text{i.i.d.}}{\sim} p$	



訓練データ

モデル

$g_{\theta}(x)$: 順伝播型NN

- 各層のユニット数は以下.
 - ▶ 入力層: 2 (入力次元)
 - ▶ 隠れ層①: 64
 - ▶ 隠れ層②: 32
 - ▶ 出力層: 5 (クラス数)
- 活性化関数はReLU, 最終層のみSoftmax.

目的関数

負の対数尤度

$$L(\theta) := \frac{1}{n} \sum_{i=1}^n \ell((x_i, y_i), g_{\theta})$$

- モデル選択基準は検証損失値 (早期停止 = 学習エポック数のみ選択)
- L2正則化はなし

最適化手法

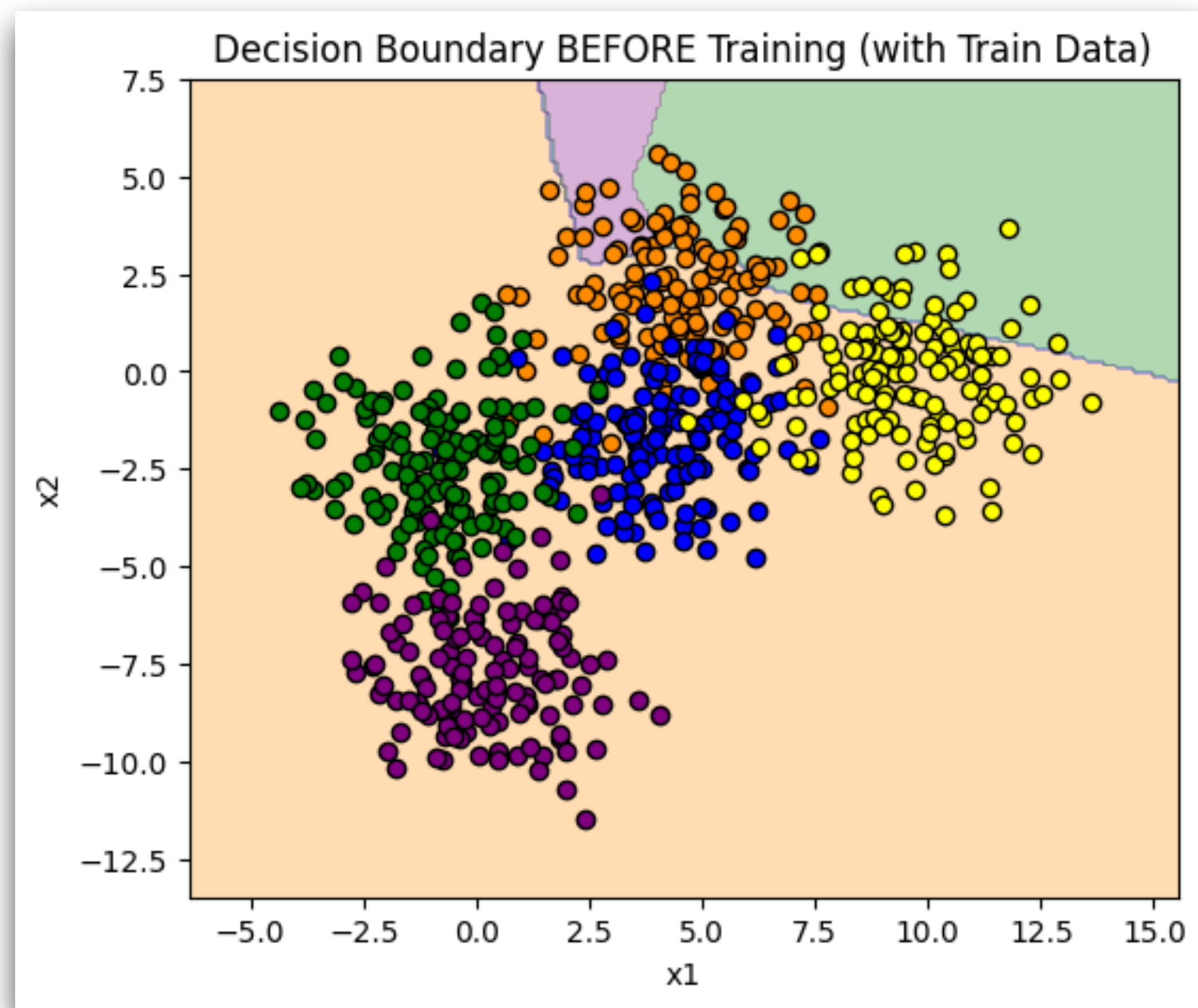
Adam

(慣性項を用いる確率的勾配法の一つ)

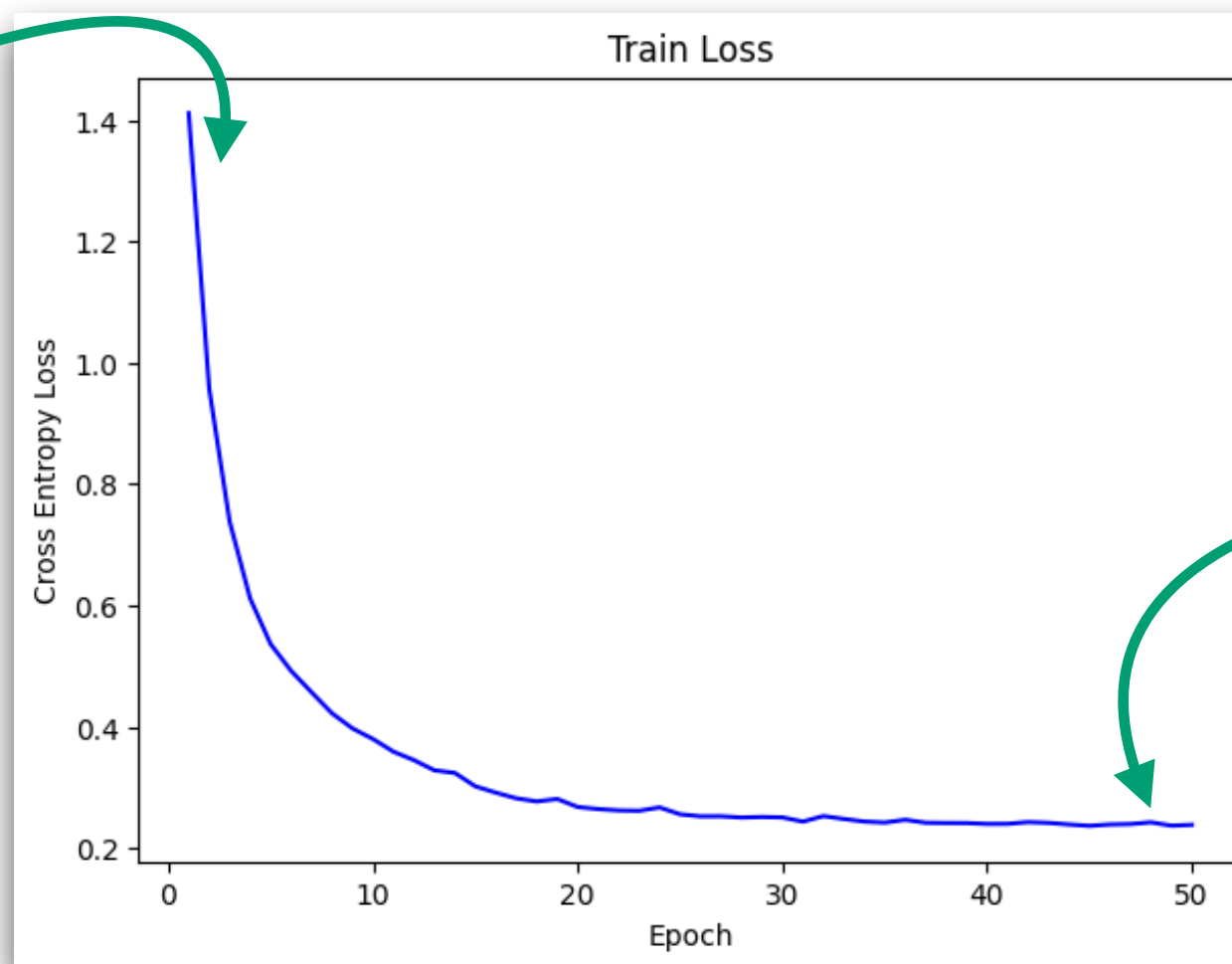
- バッチサイズは 32
- 学習率 η は 10^{-3}
- エポック数上限 100
- 早期停止 (直近10エポックの改善幅が0以下で停止)
- 初期値 $\hat{\theta}$ はHeの初期化

ニューラルネット＞実験＞結果

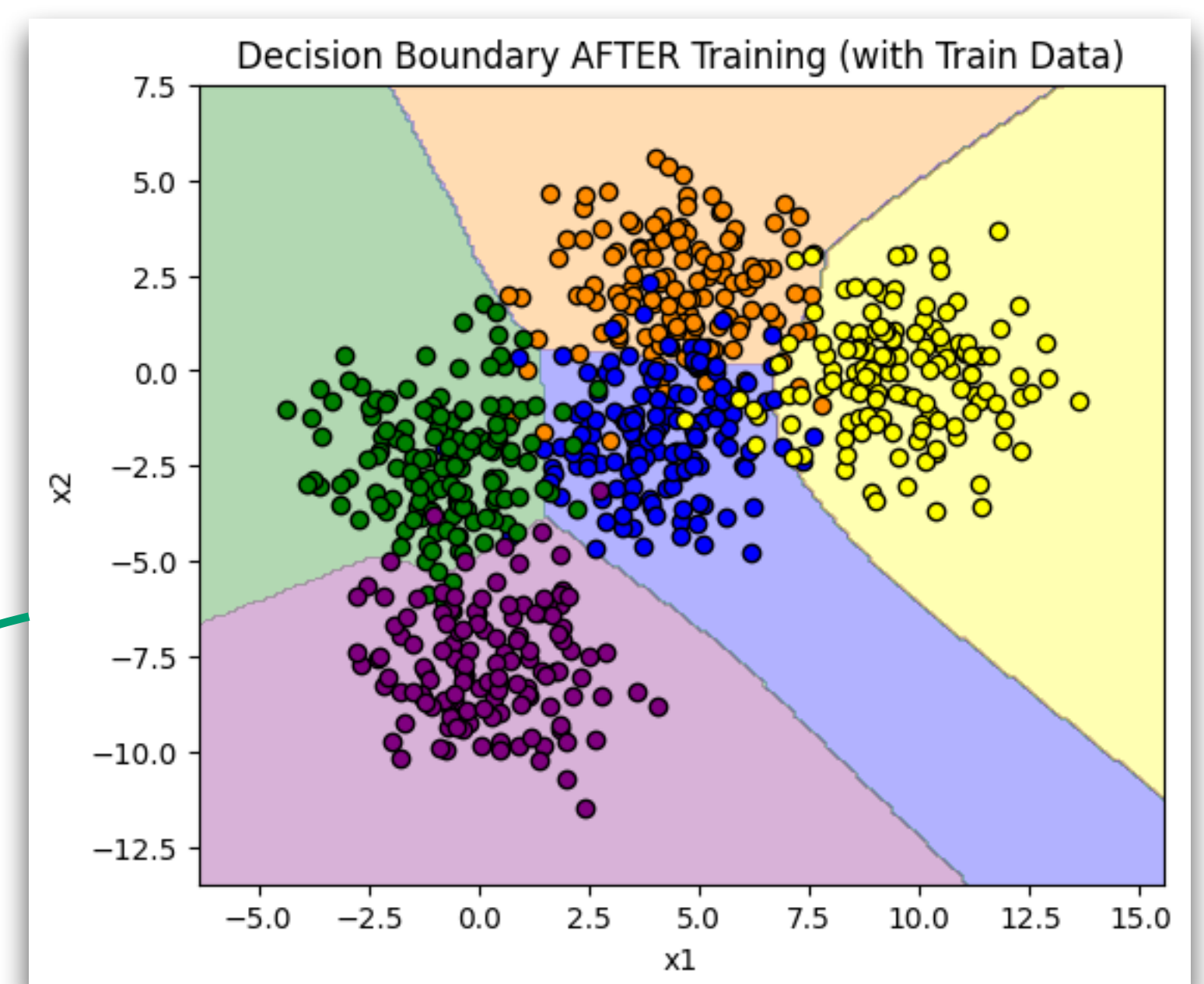
訓練前の初期化した時点での予測器と，訓練後の予測器



訓練前のモデルと訓練標本



訓練誤差

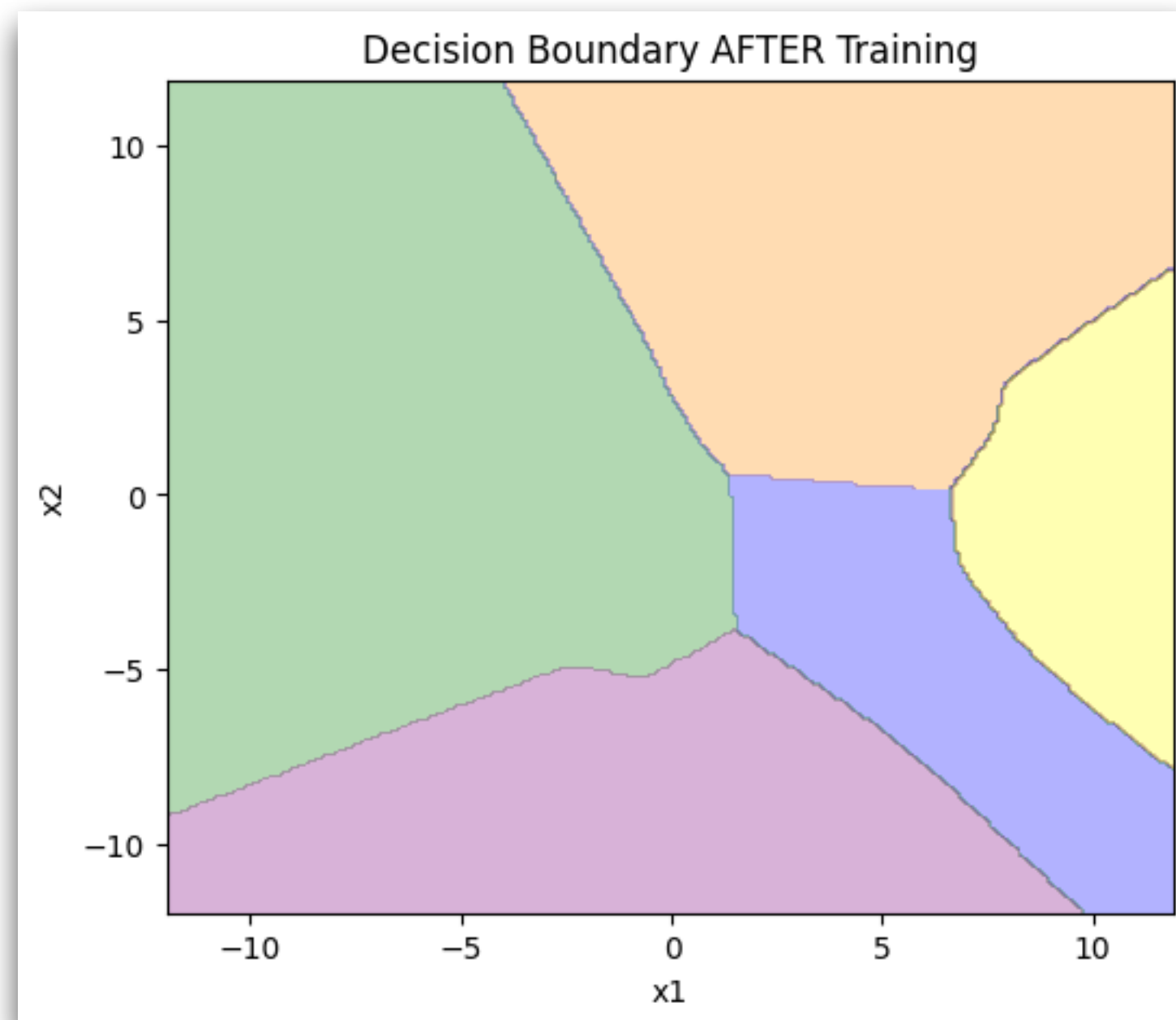


訓練後のモデルと訓練標本

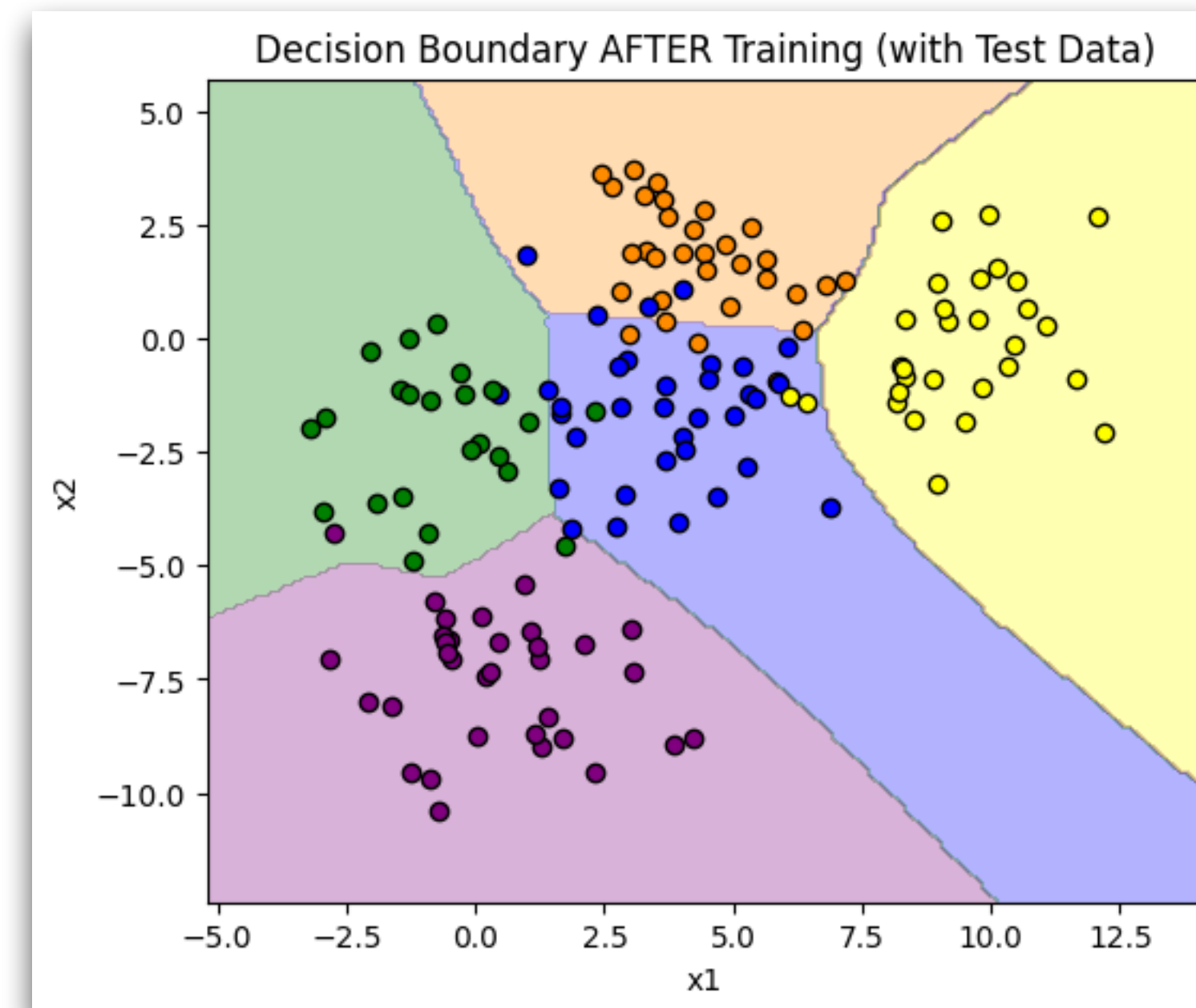
ニューラルネット > 実験 > 学習過程

予測器としても良さそうなものになっている

- 学習後のモデルに決定規則を合成した予測器 $f_{\hat{\theta}}(x) := \tau(g_{\hat{\theta}}(x))$ の様子



分類器としての出力
(スコア最大値を与えるラベルを予測)



評価用データでの予測と正解ラベル

ニューラルネット > まとめ

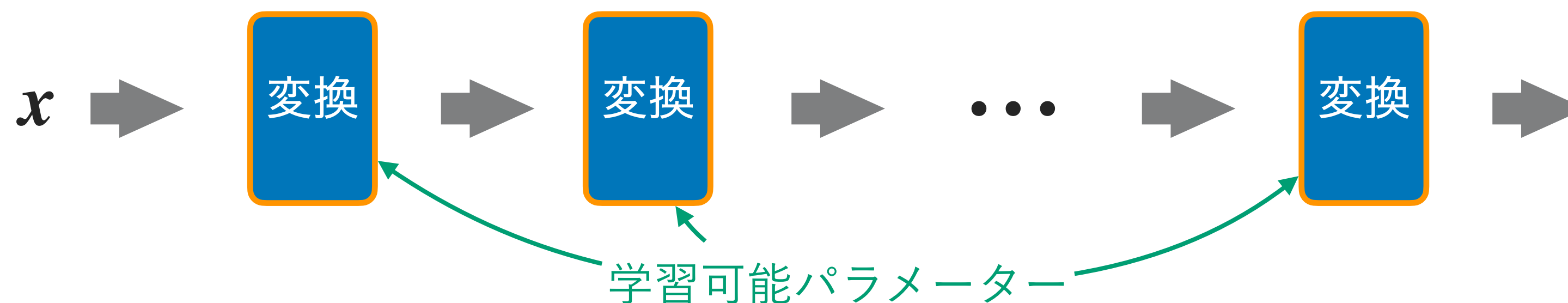


ニューラルネットは階層性と構造の自由度がポイント。最適化は大変。

- ニューラルネットワークというモデルは、非線形モデルの主要なものの一つであり、実用的にも広く使われている。
 - 順伝播型NNは、線型変換→非線型変換→……を積み重ねて複雑な関数を表現
 - 自由度の高さを活かして様々なアーキテクチャのNNが開発されている
- 最適化は誤差逆伝播法の考え方を使えば、複雑なアーキテクチャのNNでも汎用的に勾配を計算できて、勾配ベースの最適化手法が適用できる
 - ただし、ニューラルネットの最適化は高難易度で、様々な工夫を要する
 - 実際には、最新時点で評判の良い最適化手法を調べて使うことから

これまでのあらすじ

- ニューラルネットワークという柔軟な道具を手に入れた



- 最終層の手前までを特徴写像 ϕ だと思えば、特徴量変換も含めて一気通貫で学習可能なモデルともいえる
- 対して線型モデルでは、 ϕ で特徴量に変換 $\rightarrow \theta^\top$ を掛けて出力 $\rightarrow \theta^\top \phi(x)$
- これは人類にとって重要な道具になった。一体なぜ重要になったのか？